PNEUMONIA DETECTION USING RESNET

PneumoScan AI: an explainable ResNet-ensemble system for chest X-ray screening

---

A MINI PROJECT REPORT

submitted for

CSDX530 – DEEP LEARNING

(CAT-2 Mini Project)

Submitted by

UMER BASITH G

Register Number: 230171601192

Under the guidance of

Mrs. Mahalakshmi R

Assistant Professor / CSE

Department of Computer Science and Engineering

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

B.S. ABDUR RAHMAN CRESCENT INSTITUTE OF SCIENCE AND TECHNOLOGY

Vandalur, Chennai

NOVEMBER 2026

B.S. ABDUR RAHMAN CRESCENT INSTITUTE OF SCIENCE AND TECHNOLOGY

VANDALUR, CHENNAI

BONAFIDE CERTIFICATE

Certified that this mini project report titled **"PNEUMONIA DETECTION USING RESNET"** is the bonafide work of **UMER BASITH G** (Register Number: **230171601192**), who carried out the project work under my supervision for the course **CSDX530 – Deep Learning** (CAT-2 Mini Project) in the Department of Computer Science and Engineering, B.S. Abdur Rahman Crescent Institute of Science and Technology, during November 2026.

Certified further that, to the best of my knowledge, the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Mrs. Mahalakshmi R
FACULTY IN-CHARGE
Assistant Professor
Department of Computer Science and Engineering
B.S. Abdur Rahman Crescent Institute of Science and Technology
Vandalur, Chennai

SIGNATURE

HEAD OF THE DEPARTMENT

Department of Computer Science and Engineering
B.S. Abdur Rahman Crescent Institute of Science and Technology
Vandalur, Chennai

DECLARATION

I, **UMER BASITH G** (Register Number: **230171601192**), hereby declare that the mini project report entitled **"PNEUMONIA DETECTION USING RESNET"**, submitted for the course **CSDX530 – Deep Learning** (CAT-2 Mini Project) in the Department of Computer Science and Engineering, B.S. Abdur Rahman Crescent Institute of Science and Technology, Chennai, is a record of original work carried out by me under the guidance of **Mrs. Mahalakshmi R**, Assistant Professor / CSE.

I further declare that the work reported in this report has not been submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. All external sources of data, figures and text have been acknowledged and cited.

Place: Chennai
Date: 09.11.2026

UMER BASITH G
RRN: 230171601192

ACKNOWLEDGEMENT

I express my sincere gratitude to the Chancellor, the Vice-Chancellor and the Registrar of B.S. Abdur Rahman Crescent Institute of Science and Technology for providing an excellent academic environment and the facilities that made this work possible.

I am thankful to the Head of the Department of Computer Science and Engineering for the constant encouragement and support extended to the students of the department.

I owe my deepest thanks to my faculty in-charge, **Mrs. Mahalakshmi R**, Assistant Professor / CSE, for teaching the course CSDX530 – Deep Learning, for her valuable guidance and suggestions, and for giving me the opportunity to carry out this mini project on a meaningful healthcare problem.

I thank all the faculty members and staff of the Department of Computer Science and Engineering for their help during the course of this work. I also acknowledge Kermany and colleagues for making the paediatric chest X-ray dataset publicly available under an open licence, and the developers of the open-source software used in this project.

Finally, I am grateful to my family and friends for their encouragement, patience and support throughout this project.

UMER BASITH G

ABSTRACT

Pneumonia is an infection of the lungs and a leading cause of death in young children; the World Health Organization reports 740,180 deaths of children under five in 2019 [14]. Chest X-ray imaging is the most frequently used method for diagnosing pneumonia [12], but reading X-rays needs trained radiologists, who are scarce in India [16]. This project presents **PneumoScan AI**, a decision-support system that classifies frontal chest X-ray images as NORMAL or PNEUMONIA using deep residual networks (ResNet) [1].

Three ImageNet-pretrained ResNets (ResNet-50 at 384 × 384 pixels, ResNet-34 at 384 × 384 and ResNet-50 at 448 × 448) are fine-tuned end-to-end with transfer learning on the public paediatric chest X-ray dataset of Kermany et al. [2], [17], using class-weighted cross-entropy with label smoothing, realistic data augmentation, mixed-precision training and an exponential moving average of the weights. The members are combined into an ensemble with horizontal-flip test-time augmentation, the output is calibrated by temperature scaling, and the decision threshold is chosen by Youden's J statistic. All of these choices were made on a patient-level validation split carved out of the training data; the official independent test set of 624 images was scored exactly once.

On this test set the system achieves an accuracy of **94.55%**, a sensitivity of **99.74%** (only 1 of 390 pneumonia X-rays missed), a specificity of **85.90%**, a precision of **92.18%**, an F1-score of **95.81%** and a ROC-AUC of **0.9867**. The project target of 95% accuracy was missed by three images; the remaining errors are mainly normal X-rays flagged for review, which is the safer kind of error for a screening aid and matches the published behaviour of this test set. Grad-CAM heat maps [4] show the image regions behind each prediction, and an out-of-distribution gate turns away images that are not chest X-rays: it rejected all 33 non-X-ray check images (photographs, textures, documents and blank images) while accepting 620 of the 624 test X-rays.

The model is delivered through an offline web application built with FastAPI and React, offering single and batch (triage) scanning, patient PDF reports, local scan history, a CURB-65 severity calculator [5] and sourced patient education. The tool is intended for education and decision support only and does not replace a qualified doctor.

Keywords: pneumonia, chest X-ray, deep learning, ResNet, transfer learning, ensemble, Grad-CAM, explainable AI, medical decision support.

TABLE OF CONTENTS

BONAFIDE CERTIFICATE ..... **i**

DECLARATION ..... **ii**

ACKNOWLEDGEMENT ..... **iii**

ABSTRACT ..... **iv**

LIST OF FIGURES ..... **ix**

LIST OF TABLES ..... **x**

LIST OF ABBREVIATIONS ..... **xi**

1 INTRODUCTION **1**

1.1 Background 1

1.2 Role of the Chest X-ray 1

1.3 Deep Learning for Medical Images 2

1.4 Residual Networks (ResNet) 2

1.5 Problem Statement 2

1.6 Objectives 3

1.7 Scope 3

1.8 Motivation 3

1.9 Organisation of the Report 4

2 LITERATURE SURVEY **5**

2.1 Introduction 5

2.15 Summary of the Literature 8

2.16 Research Gap 9

3 SYSTEM ANALYSIS AND DESIGN **11**

3.1 Existing System 11

3.2 Proposed System 11

3.3 Feasibility Study 12

3.4 Requirement Specification 13

3.5 System Architecture 14

3.6 Use-Case Diagram 15

3.7 Data Flow 16

3.8 Module Description 17

4 DATASET DETAILS **18**

4.1 Source of the Dataset 18

4.2 Class Distribution and Split 18

4.3 Sample Images 19

4.4 Pre-processing 20
4.5 Data Augmentation 20
4.6 Handling Class Imbalance 21
4.7 Leakage Checks 21
5 MODEL ARCHITECTURE AND METHODOLOGY 22
5.1 Convolutional Neural Networks 22
5.2 Residual Learning 22
5.3 Basic and Bottleneck Blocks 22
5.4 ResNet-18, ResNet-34 and ResNet-50 23
5.5 Transfer Learning 24
5.6 Complete Pipeline 24
5.7 Training Setup 26
5.8 Ensemble and Test-Time Augmentation 27
5.9 Temperature Scaling 27
5.10 Threshold Selection and Risk Levels 28
5.11 Grad-CAM Explanations 28
5.12 X-ray Gate (Out-of-Distribution Detection) 29
5.13 Evaluation Metrics 29
6 IMPLEMENTATION 31
6.1 Development Environment 31
6.2 Project Structure 31
6.3 Data Preparation 31
6.4 Dataset and Augmentation 32
6.5 Building the Model 33
6.6 Training Loop 33
6.7 Evaluation, Calibration and Threshold 34
6.8 Grad-CAM 34
6.9 Inference API 35
6.10 X-ray Gate 36
6.11 Back-end API (FastAPI) 36
6.12 Front-end (React + TypeScript) 37
6.13 Testing 39
7 RESULTS AND DISCUSSION 40
7.1 Experimental Protocol 40
7.2 Training Behaviour 40
7.3 Validation Results 41
7.4 Official Test Results 42
7.5 Confusion Matrix 42

7.6 ROC and Precision–Recall Curves 43
7.7 Distribution of Predicted Probabilities 44
7.8 Grad-CAM Explanations 45
7.9 X-ray Gate Results 45
7.10 Inference Speed 46
7.11 Comparison with Published Work 46
7.12 Error Analysis 47
7.13 Discussion of the 95% Target 48
8 APPLICATION AND SCREENSHOTS 49
8.1 Overview 49
8.2 Scan Page 49
8.3 Result with Grad-CAM Heat Map 50
8.4 Patient PDF Report 52
8.5 Rejection of Non-X-ray Images 53
8.6 Triage Queue (Doctor Mode) 54
8.7 Scan History 55
8.8 CURB-65 Severity Calculator 56
8.9 Learn Page 57
8.10 Model Page 58
8.11 About Page 59
8.12 Dark Mode 60
9 CONCLUSION AND FUTURE ENHANCEMENTS 62
9.1 Conclusion 62
9.2 Limitations 62
9.3 Future Enhancements 63
9.4 Social Impact 63
REFERENCES 65
APPENDIX A: SOURCE CODE 68
A.1 ml/common.py 68
A.2 ml/prepare_data.py 69
A.3 ml/datasets.py 71
A.4 ml/train.py 72
A.5 ml/evaluate.py 75
A.6 ml/gradcam.py 78
A.7 ml/xray_gate.py 79
A.8 ml/inference.py 82
A.9 app/backend/model_service.py 85
A.10 app/backend/main.py 88

APPENDIX B: HOW TO RUN THE PROJECT 95
B.1 Running the Application (Classroom Demo) 95
B.2 First-Time Setup 95
B.3 Reproducing the Machine-Learning Pipeline 95
B.4 Analysing a Single Image from the Command Line 96
B.5 Running the Tests 96

LIST OF FIGURES

Figure 3.1: Overall system architecture of PneumoScan AI 15
Figure 3.2: Use-case diagram 16
Figure 3.3: Training, validation and deployment data flow 16
Figure 4.1: Class distribution in the training, validation and test splits 19
Figure 4.2: Sample training X-rays: four NORMAL (top) and four PNEUMONIA (bottom) 20
Figure 5.1: Bottleneck residual block with identity shortcut 23
Figure 5.2: Architecture of the PneumoScan AI pipeline (ResNet-50 member shown; rotated to fit the page) 25
Figure 7.1: Training curves of ResNet-50 at 384 px (r50_384_s1) 41
Figure 7.2: Training curves of ResNet-34 at 384 px (r34_384_s2) 41
Figure 7.3: Training curves of ResNet-50 at 448 px (r50_448_s3) 41
Figure 7.4: Confusion matrix of the final ensemble on the test set 43
Figure 7.5: ROC curves on the test set (ensemble and members) 43
Figure 7.6: Precision–recall curves on the test set 44
Figure 7.7: Ensemble probabilities on the test set by true class, with the decision threshold 44
Figure 7.8: Grad-CAM explanations on test X-rays (red = regions driving the prediction) 45
Figure 8.1: Scan page with the upload lightbox and optional patient details 49
Figure 8.2: Result screen for a pneumonia X-ray with the Grad-CAM heat map 50
Figure 8.3: Full result page with explanation, next steps and disclaimer (top part of the page) 51
Figure 8.4: Side-by-side view of the X-ray and the Grad-CAM heat map 52
Figure 8.5: First page of a patient PDF report generated by the application 53
Figure 8.6: A colour photograph rejected by the X-ray gate 54
Figure 8.7: Completed triage queue sorted by pneumonia probability 55
Figure 8.8: Scan history in grid view (top part of the page) 56
Figure 8.9: CURB-65 severity calculator (top part of the page) 57
Figure 8.10: Learn page with sourced health information (top part of the page) 58
Figure 8.11: Model page with test-set performance (top part of the page) 59
Figure 8.12: About page 60
Figure 8.13: Result screen in dark mode 61

LIST OF TABLES

Table 2.1: Summary of the literature survey 9
Table 3.1: Comparison of the existing and proposed systems 12
Table 3.2: Hardware used for development, training and testing 13
Table 3.3: Software used in the project 13
Table 3.4: Modules of the system 17
Table 4.1: Summary of the dataset 18
Table 4.2: Images and patients in each split 19
Table 4.3: Training augmentation (torchvision transforms v2) 21
Table 4.4: Leakage checks 21
Table 5.1: Comparison of the ResNet variants used in this work 23
Table 5.2: Training hyperparameters (identical for all members unless stated) 27
Table 5.3: Risk levels derived from the calibrated probability p (t = 0.0743) 28
Table 6.1: REST API endpoints 37
Table 7.1: Training runs on the RTX 3050 GPU 40
Table 7.2: Validation results (747 images, flip TTA, settings fitted on validation) 42
Table 7.3: Official test-set results (624 images; thresholds from validation) 42
Table 7.4: Validation of the X-ray gate 46
Table 7.5: Time per image (median, after warm-up, measured on official test images) 46
Table 7.6: Comparison with published results on the official Kermany test set 47
Table 7.7: Post-hoc threshold sensitivity on the test set (discussion only; NOT the reported result) 48
Table 8.1: CURB-65 risk groups used by the calculator (derivation cohort of Lim et al. [5]) 56

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
AMP	Automatic Mixed Precision
API	Application Programming Interface
AUC	Area Under the Curve
BN	Batch Normalisation
CAP	Community-Acquired Pneumonia
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CSV	Comma-Separated Values
CUDA	Compute Unified Device Architecture (NVIDIA)
CURB-65	Confusion, Urea, Respiratory rate, Blood pressure, age ≥ 65 (severity score)
DICOM	Digital Imaging and Communications in Medicine
EMA	Exponential Moving Average
FC	Fully Connected (layer)
FN / FP	False Negative / False Positive
GPU	Graphics Processing Unit
Grad-CAM	Gradient-weighted Class Activation Mapping
ICU	Intensive Care Unit
JSON	JavaScript Object Notation
NLL	Negative Log-Likelihood
OOD	Out-of-Distribution
PACS	Picture Archiving and Communication System
PDF	Portable Document Format
PNG / JPEG	Portable Network Graphics / Joint Photographic Experts Group (image formats)
PR	Precision–Recall
ReLU	Rectified Linear Unit
ResNet	Residual Network
REST	Representational State Transfer
ROC	Receiver Operating Characteristic
SQL	Structured Query Language
TN / TP	True Negative / True Positive
TTA	Test-Time Augmentation
UI	User Interface
WHO	World Health Organization

CHAPTER 1
INTRODUCTION

1.1 Background

Pneumonia is a form of acute respiratory infection in which the alveoli (the tiny air sacs of the lungs) fill with pus and fluid, making breathing painful and reducing the amount of oxygen the body can take in [14]. It can be caused by viruses, bacteria or fungi [14]. In children, *Streptococcus pneumoniae* is the most common bacterial cause and respiratory syncytial virus the most common viral cause [14]. Children younger than five years and adults aged 65 years or older are at increased risk, and people with a weakened immune system are at the greatest risk [19].

The disease burden is very large. Pneumonia caused 740,180 deaths of children under five in 2019, which is 14% of all deaths in that age group [14]. Across all ages, lower respiratory infections caused 2.7 million deaths in 2023, were the fourth leading cause of death worldwide and have remained the leading cause of death in low-income countries since around 2010 [15]. Pneumonia can be prevented by immunisation, adequate nutrition and cleaner indoor air, and pneumonia caused by bacteria can be treated with antibiotics [14], so early and correct diagnosis matters.

1.2 Role of the Chest X-ray

A doctor diagnoses pneumonia from the history, the physical examination (temperature, breathing rate, listening to the chest) and investigations. The chest X-ray (radiograph) is the most frequently used imaging test to confirm pneumonia [12]. On a frontal chest X-ray, pneumonia typically appears as **consolidation**: a white (opaque) area where the air in the alveoli has been replaced by fluid or pus. The World Health Organization standard for reading children's chest radiographs counts alveolar consolidation or pleural effusion as the primary end-point for pneumonia [20]. Other signs include air bronchograms inside the opacity and loss of the normal sharp borders of the heart or diaphragm.

Reading radiographs correctly needs training and experience, and different readers can disagree, especially for subtle or patchy changes in children [12], [20]. An X-ray also cannot tell which germ caused the infection, so it is always interpreted together with the clinical picture.

1.3 Deep Learning for Medical Images

Deep learning is a branch of machine learning in which a neural network with many layers learns useful features directly from raw data. Convolutional neural networks (CNNs) learn small filters that detect edges and textures in the first layers and progressively more complex patterns in deeper layers, which makes them very effective for image classification. CheXNet, a 121-layer CNN trained on more than 100,000 adult chest X-rays, exceeded the average performance of four radiologists (F1 metric) for pneumonia detection [3]. Kermany et al. showed that **transfer learning**, which starts from a network pretrained on a large collection of everyday photographs (ImageNet) and adapts it to medical images, can produce an accurate paediatric pneumonia classifier with a fraction of the data normally required [2].

1.4 Residual Networks (ResNet)

Very deep networks are hard to train: beyond a certain depth, adding more layers makes the training accuracy saturate and then degrade. He et al. [1] solved this **degradation problem** with residual learning. Instead of asking a stack of layers to learn a desired mapping $H(x)$ directly, a residual block learns the residual $F(x) = H(x) - x$ and adds the input back through an identity **shortcut connection**, so that the block outputs $F(x) + x$. The shortcut gives gradients a direct path through the network, which made it possible to train networks with up to 152 layers; an ensemble of residual networks won the ILSVRC 2015 classification task with a 3.57% top-5 error [1]. ResNets are now one of the most widely used backbones for transfer learning in medical imaging, and they are the backbone of this project. They are explained in detail in Chapter 5.

1.5 Problem Statement

Manual interpretation of chest X-rays for pneumonia is time-consuming, depends on the availability of trained radiologists, and is subject to variation between readers [12]. In India there are only about 20,500 registered radiologists, roughly one per 100,000 people [16]. There is a need for an accurate, explainable and easy-to-use automated system that can analyse a chest X-ray, indicate the likelihood of pneumonia, highlight the regions that influenced the decision and help to prioritise patients, while making clear that the final decision rests with a qualified doctor.

1.6 Objectives

- To build a deep-learning classifier based on the ResNet architecture that labels a frontal chest X-ray as NORMAL or PNEUMONIA.
- To reach a target accuracy of at least 95% on the official held-out test set of 624 images, and to report sensitivity, specificity, precision, F1-score and ROC-AUC honestly, whatever the outcome.
- To avoid any tuning on the test set by selecting every setting (epochs, ensemble, calibration and decision threshold) on a validation split drawn only from the training data.
- To provide a visual explanation of each prediction using Grad-CAM heat maps.
- To reject input images that are not chest X-rays.
- To develop an offline web application with single-image scanning, batch triage, patient PDF reports, scan history and a CURB-65 severity calculator.
- To present reliable, sourced health information about pneumonia for patients and families.

1.7 Scope

- **Input:** frontal chest X-ray images in JPG, PNG, BMP, WEBP, TIFF or DICOM format.
- **Output:** a two-class prediction (NORMAL / PNEUMONIA), a calibrated probability, a risk level (Low / Moderate / High), a Grad-CAM heat map and a downloadable PDF report.
- **Data:** the public paediatric chest X-ray dataset of Kermany et al. (children aged 1–5 years from one hospital in Guangzhou, China) [2], [17].
- **Users:** medical students (learning), doctors (second reader and triage aid) and patients (education).
- **Out of scope:** identifying the cause (bacterial or viral), detecting other lung diseases, validation on adults, and any use as a certified medical device.

1.8 Motivation

- **Shortage of radiologists in India:** about one radiologist per 100,000 population compared with one per 10,000 in the United States; most radiologists work in larger cities and towns, while about sixty percent of the population lives in rural areas [16].

- **High burden of disease:** pneumonia caused 740,180 deaths of children under five in 2019 [14], and lower respiratory infections were the fourth leading cause of death worldwide in 2023 [15].
- **Evidence that AI can help:** a deep CNN exceeded average radiologist performance (F1) for pneumonia detection [3], and a review of Indian radiology found that teleradiology combined with AI and mobile digital imaging can address radiologist shortages [16].
- **Need for honest and explainable tools:** many published results use re-split data, which may overstate accuracy on independent data (Chapter 2), and pneumonia models can perform worse at new hospitals [13]. A transparent tool with heat maps and honestly reported test results supports appropriate trust.

1.9 Organisation of the Report

The rest of this report is organised as follows. **Chapter 2** reviews related work on ResNets, pneumonia detection and explainability, and identifies the research gap. **Chapter 3** analyses the existing and proposed systems, lists the requirements and presents the system architecture, use-case diagram, data flow and modules. **Chapter 4** describes the dataset, the patient-level split, pre-processing, augmentation and leakage checks. **Chapter 5** explains the ResNet architecture and the complete methodology, including training, ensembling, calibration, threshold selection, Grad-CAM and the X-ray gate. **Chapter 6** presents the implementation with key code listings. **Chapter 7** reports and discusses the results. **Chapter 8** walks through the application with screenshots. **Chapter 9** concludes the report with limitations, future enhancements and social impact. The complete source code of the machine-learning pipeline and backend is given in **Appendix A**, and instructions to run the project are in **Appendix B**.

CHAPTER 2
LITERATURE SURVEY

2.1 Introduction

This chapter reviews the work that this project builds on: the ResNet architecture, transfer learning for chest X-rays, ensembles of CNNs, explainability with Grad-CAM, generalisation across hospitals and the CURB-65 clinical score. Every paper was checked against PubMed, Crossref, arXiv or the open-access full text. An important observation is that several papers report very high accuracy (97–99%) on the Kermany data, but they **re-split** the images or used cross-validation instead of the official 624-image test set. Such numbers are not directly comparable with results on the official test set, and this is pointed out for each paper.

2.2 Deep Residual Learning for Image Recognition (He et al., 2016) [1]

He et al. observed that very deep plain networks suffer from a degradation problem: accuracy saturates and then falls as layers are added, even on the training set. They reformulated layers to learn residual functions with reference to the layer input, using identity shortcut connections, batch normalisation and global average pooling, with two 3 × 3 convolutions per block in shallower models and 1 × 1–3 × 3–1 × 1 "bottleneck" blocks in deeper ones. Networks up to 152 layers (eight times deeper than VGG nets, but with lower complexity) were trained on ImageNet; an ensemble reached a **3.57% top-5 error** and won first place in ILSVRC 2015, with a 28% relative improvement on COCO object detection. **Relation:** ResNet is the backbone of PneumoScan AI, which fine-tunes ImageNet-pretrained ResNet-34 and ResNet-50 models.

2.3 Identifying Medical Diagnoses by Image-Based Deep Learning (Kermany et al., 2018) [2]

The authors built a transfer-learning diagnostic tool, first for retinal OCT images and then for paediatric chest X-rays, by fine-tuning an ImageNet-pretrained Inception-V3. Transfer learning allowed a network trained on general images to be adapted with a fraction of the usual data, and image regions used by the network were highlighted to improve interpretability. The chest X-ray data (5,232 training images: 1,349 normal and 3,883 pneumonia; and an independent 624-image test set) come from Guangzhou Women and Children's Medical Center. For pneumonia versus normal they reported **accuracy 92.8%,**

sensitivity 93.2%, specificity 90.1% and AUC 96.8% (as quoted in the review by Chumbita et al. [18]). **Relation:** this is the dataset used in this project and the main baseline on the same official test set.

2.4 CheXNet: Radiologist-Level Pneumonia Detection (Rajpurkar et al., 2017) [3]

CheXNet is a 121-layer DenseNet trained end-to-end on ChestX-ray14, a set of more than 100,000 frontal adult chest X-rays with 14 disease labels. Four practising academic radiologists labelled a test set so that the model could be compared with human readers, and CheXNet **exceeded the average radiologist on the F1 metric** for pneumonia; it was then extended to all 14 diseases. **Relation:** it shows that deep CNNs can approach expert reading of chest X-rays and motivates the CNN approach used here.

2.5 Grad-CAM: Visual Explanations from Deep Networks (Selvaraju et al., 2017) [4]

Grad-CAM produces class-discriminative visual explanations for CNN decisions. It uses the gradients of the target class score flowing into the final convolutional layer to weight its feature maps and build a coarse heat map of the regions that matter for the prediction. It works with many CNN families, including ResNets, without changing or retraining the model. Human studies showed that Grad-CAM helps users build appropriate trust and can reveal dataset bias. **Relation:** PneumoScan AI shows a Grad-CAM heat map for every X-ray so that a doctor can check whether the model looks at the lungs.

2.6 Defining Pneumonia Severity: the CURB-65 Score (Lim et al., 2003) [5]

The authors pooled three prospective studies from the UK, New Zealand and the Netherlands (1,068 adults with community-acquired pneumonia, 30-day mortality 9%) to build a simple severity score. One point each for Confusion, Urea > 7 mmol/L, Respiratory rate ≥ 30/min, low Blood pressure (systolic < 90 or diastolic ≤ 60 mmHg) and age ≥ 65 gives the six-point CURB-65 score. In the derivation cohort, 30-day mortality was **1.5%** for scores 0–1, **9.2%** for score 2 and **22%** for scores 3–5. A version without urea (CRB-65) was proposed for use in the community. **Relation:** the application includes a CURB-65 calculator so that a doctor can combine the X-ray result with a standard clinical severity score.

2.7 MedMNIST v2 (Yang et al., 2023) [6]

MedMNIST v2 is a collection of standardised 2D and 3D biomedical datasets for benchmarking. Its PneumoniaMNIST subset is built from the Kermany chest X-rays, **keeps the official 624 test images** and splits the original training set 9:1 into training and

validation. ResNet baselines on the official test set reached ACC 0.854 / AUC 0.944 (ResNet-18, 28 px), ACC 0.864 / AUC 0.956 (ResNet-18, 224 px) and **ACC 0.884 / AUC 0.962 (ResNet-50, 224 px)**. **Relation:** a clean, honest ResNet baseline on the same test images, and a model for carving the validation split from the training data.

2.8 An Efficient Deep Learning Approach to Pneumonia Classification (Stephen et al., 2019) [7]

The authors designed a CNN trained from scratch and, because the data are limited, used heavy augmentation (rotation up to 40°, shifts, shear and zoom) and studied input sizes from 100 × 100 to 300 × 300 pixels. The Kermany images were rearranged into 3,722 training and 2,134 validation images, giving a **validation accuracy of 93.73%**. **Relation:** a from-scratch baseline; its result is on a re-split validation set, not the official test set, so it is not directly comparable.

2.9 A Novel Transfer Learning Based Approach for Pneumonia Detection (Chouhan et al., 2020) [8]

Features were extracted with five ImageNet-pretrained networks and their outputs were combined by majority voting. The ensemble outperformed every single model and reached **accuracy 96.4% and recall 99.62%** on unseen test data from the Guangzhou dataset, after training on the 5,232 training images. **Relation:** the closest published evidence that an ensemble of pretrained CNNs can exceed 95% on held-out Guangzhou test data; it supports the ResNet-ensemble design of this project.

2.10 Transfer Learning with Deep CNN for Pneumonia Detection (Rahman et al., 2020) [9]

AlexNet, ResNet-18, DenseNet-201 and SqueezeNet were fine-tuned on 5,247 chest X-rays for three schemes: normal versus pneumonia, bacterial versus viral, and three classes. Reported accuracies were **98%**, 95% and 93.3% respectively, using the authors' own split rather than the official 624-image test set. **Relation:** it shows that ResNet-18 is a common, strong choice, but the numbers are not directly comparable with ours.

2.11 Efficient Pneumonia Detection using Deep Transfer Learning (Hashmi et al., 2020) [10]

Five pretrained networks (ResNet-18, Xception, Inception-V3, DenseNet-121 and MobileNetV2) were fine-tuned (SGD, learning rate 0.001, momentum 0.9, 25 epochs) and combined by a weighted classifier whose weights were found on 400 images held out from training. Only the minority (normal) class was augmented, and class activation maps

showed the regions behind each decision. The images were shuffled and re-split into 5,136 training and 700 test images, giving **98.43% accuracy and 99.76% AUC** (97.45% with equal weights) on a 6 GB GTX 1060 GPU. **Relation:** it inspired the idea of fitting ensemble settings on held-out training data and shows that a 6 GB GPU is sufficient; its test set is not the official one.

2.12 A Transfer Learning Method with Deep Residual Network for Pediatric Pneumonia (Liang & Zheng, 2020) [11]

This framework combines residual connections with dilated convolutions for childhood pneumonia. Residual connections counter over-fitting and the degradation problem, dilated convolutions keep spatial information that is otherwise lost with depth, and the model is initialised by transfer learning. On the test set it reached **recall 96.7% and F1-score 92.7%**. **Relation:** the closest prior work, a residual network for paediatric pneumonia on the same data.

2.13 Pneumonia Detection using an Ensemble of Deep Learning Models (Kundu et al., 2021) [12]

GoogLeNet, ResNet-18 and DenseNet-121 were combined with a weighted-average ensemble whose weights are computed from each model's precision, recall, F1-score and AUC. With five-fold cross-validation, it reached **98.81% accuracy** on the pooled Kermany data (ResNet-18 alone 97.29%) and 86.85% on the RSNA pneumonia dataset; Grad-CAM was used to analyse correct and wrong predictions. **Relation:** ensembles beat single models, but cross-validation numbers are not comparable with the official test split.

2.14 Variable Generalisation of Pneumonia Models (Zech et al., 2018) [13]

Zech et al. trained pneumonia-screening CNNs on chest X-rays from several hospital systems and showed that performance dropped on an external hospital (**internal AUC 0.931 versus external 0.815**). The CNNs could even recognise which hospital an image came from (for example 99.95% of NIH images), which means they can learn hospital-specific shortcuts. **Relation:** this motivates the limitations, the Grad-CAM inspection and the clear disclaimer in this project.

2.15 Summary of the Literature

Table 2.1 summarises the reviewed works. Results marked "official test" use the same 624 test images as this project; the others use different evaluation data and are listed for context only.

Table 2.1: Summary of the literature survey

Ref.	Authors (year)	Model / method	Data / evaluation	Main result
[1]	He et al. (2016)	ResNet, up to 152 layers	ImageNet	3.57% top-5 error (ensemble)
[2]	Kermany et al. (2018)	Inception-V3, transfer learning	Kermany, official test	Acc 92.8%, Sens 93.2%, Spec 90.1%, AUC 96.8%
[3]	Rajpurkar et al. (2017)	DenseNet-121 (CheXNet)	ChestX-ray14 (adults)	F1 above average radiologist
[4]	Selvaraju et al. (2017)	Grad-CAM	ImageNet, VQA, captioning	Class-discriminative heat maps
[5]	Lim et al. (2003)	CURB-65 score	1,068 adults with CAP	Mortality 1.5% / 9.2% / 22% by group
[6]	Yang et al. (2023)	ResNet-18 / ResNet-50	PneumoniaMNIST, official test	ResNet-50 (224): Acc 88.4%, AUC 0.962
[7]	Stephen et al. (2019)	CNN from scratch	Re-split, validation	Val acc 93.73%
[8]	Chouhan et al. (2020)	5-CNN ensemble (voting)	Guangzhou unseen test data	Acc 96.4%, recall 99.62%
[9]	Rahman et al. (2020)	AlexNet / ResNet-18 / DenseNet-201 / SqueezeNet	Own split, 5,247 images	Acc 98% (normal vs pneumonia)
[10]	Hashmi et al. (2020)	Weighted 5-CNN ensemble	Re-split (700 test)	Acc 98.43%, AUC 99.76%
[11]	Liang & Zheng (2020)	Residual + dilated convolution	Kermany test set	Recall 96.7%, F1 92.7%
[12]	Kundu et al. (2021)	GoogLeNet + ResNet-18 + DenseNet-121	5-fold cross-validation	Acc 98.81%
[13]	Zech et al. (2018)	CNN, multi-hospital	Internal vs external hospitals	AUC 0.931 internal vs 0.815 external

2.16 Research Gap

From the survey, the following gaps are identified:

- Most high-accuracy results use re-split data or cross-validation, so they do not show how a model behaves on the official, independent test set, which is known to be harder [2], [6].
- On the official test set, reported errors are dominated by normal X-rays called pneumonia (for example Kermany et al. report a specificity of 90.1% against a sensitivity of 93.2% [2]), which makes the choice of decision threshold critical; yet thresholds and model selection are rarely fixed on a separate validation split.
- Models can learn hospital-specific shortcuts and generalise poorly [13], so visual explanations and honest limitations are essential.

- Few works combine a ResNet ensemble, a validation-only threshold, Grad-CAM explanations, rejection of non-X-ray inputs and clinical tools (CURB-65, triage, patient reports) in one offline application for doctors and patients.

PneumoScan AI addresses these gaps with a patient-level validation split, a three-member ResNet ensemble whose every setting is chosen on validation, a single logged evaluation on the official test set, Grad-CAM explanations, an out-of-distribution X-ray gate and a complete offline application.

CHAPTER 3
SYSTEM ANALYSIS AND DESIGN

3.1 Existing System

In current practice a chest X-ray is read manually by a radiologist or by the treating doctor. This works well where specialists are available, but it has several drawbacks:

- Reporting depends on the availability of trained radiologists, who are few and concentrated in cities [16].
- Films are usually read in the order in which they arrive; a severe case may wait behind routine ones.
- Interpretation varies between readers, especially for subtle or patchy changes in children [12], [20].
- Most published deep-learning models are research prototypes: they are evaluated on re-split data, often tuned on the test set, give no explanation for their output, accept any image (even a photograph) and are not packaged as a tool that a doctor or student can actually use.

3.2 Proposed System

PneumoScan AI is an offline decision-support application that analyses a chest X-ray with an ensemble of ResNets and explains its output. Its main parts are:

1. **Pre-processing:** any image mode (8/16-bit, RGB, RGBA, DICOM) is converted to 8-bit grayscale, squashed to a square canonical view, resized to the network input size (384 or 448 pixels), replicated to three channels and normalised with ImageNet statistics.
2. **X-ray gate:** simple colour, blank and size checks plus an out-of-distribution distance on ResNet features reject images that are not chest X-rays.
3. **Classification:** three fine-tuned ResNets score the image and its mirror image; their logit margins are averaged, calibrated by temperature scaling and compared with a threshold chosen on validation data.
4. **Explainability:** Grad-CAM heat maps from the last residual stage of every member are averaged and overlaid on the X-ray.

5. **Application:** a FastAPI back-end and a React front-end provide scanning, batch triage, PDF reports, scan history, a CURB-65 calculator, a Learn page and a Model page, all running on the local computer.

Table 3.1 compares the existing approach with the proposed system.

Table 3.1: Comparison of the existing and proposed systems

Aspect	Existing system	Proposed system (PneumoScan AI)
Reading	Manual reading of every film by a specialist	Automatic second reading in about 0.4 s per image on the GPU
Availability	Depends on radiologist availability	Runs offline on an ordinary laptop (GPU or CPU)
Prioritisation	Usually first come, first served	Triage queue sorted by pneumonia probability, urgent cases flagged
Explanation	Not applicable / black-box models	Grad-CAM heat map for every prediction
Evaluation	Often re-split data or test-set tuning	All choices on validation; official test set scored once
Input check	Models accept any image	X-ray gate rejects photographs, documents and blanks
Reporting	Separate, manual	One-click patient PDF with heat map and disclaimer
Clinical context	Separate tools	Built-in CURB-65 calculator and sourced patient education

3.3 Feasibility Study

- **Technical feasibility:** the dataset is public, ImageNet-pretrained ResNets are available in torchvision, and fine-tuning three ResNets took 55.5 minutes in total on a 6 GB laptop GPU.
- **Economic feasibility:** only free and open-source software and an openly licensed dataset are used; no paid service or cloud resource is needed.
- **Operational feasibility:** the application starts with a double-click (start.bat), runs in a web browser, needs no internet connection and stores data only on the local computer.

3.4 Requirement Specification

3.4.1 Hardware Requirements

Table 3.2: Hardware used for development, training and testing

Component	Specification
GPU	NVIDIA GeForce RTX 3050 Laptop GPU, 6 GB (CUDA)
CPU	Intel Core i5-12450HX
Memory	16 GB RAM (15.7 GB usable)
Storage	About 1.2 GB for the dataset archive, plus the extracted images, cache and model weights (three fp16 weight files of 41–46 MB each)
Operating system	Windows 11
Minimum to run the app	Any Windows computer that can run Python 3.11; the GPU is optional (CPU-only inference measured at about 1.25 s per image)

3.4.2 Software Requirements

Table 3.3: Software used in the project

Software	Version	Purpose
Python	3.11.9	Programming language for the ML pipeline and back-end
PyTorch + CUDA	2.11.0 (CUDA 12.8)	Deep-learning framework, GPU training and inference [21]
torchvision	0.26.0	ResNet models, ImageNet weights, image transforms
timm	1.0.31	Installed in the environment; the final models use torchvision ResNets
scikit-learn	1.9.1	Metrics, patient-level split, Ledoit-Wolf covariance [22]
NumPy / SciPy / pandas	2.4 / 1.17 / 3.1	Numerical work, temperature fitting, split tables
Pillow / OpenCV / pydicom	13.0 / 5.0 / 3.0	Image reading, colour maps, DICOM support
matplotlib / scikit-image	3.11 / 0.26	Report figures; non-X-ray test images for the gate
FastAPI / Uvicorn	0.162 / 0.54	REST API back-end and web server
SQLite (sqlite3)	bundled with Python	Local scan history database
ReportLab	5.1	Patient PDF reports
Node.js	24	Building the front-end
React / TypeScript / Vite	19.1 / 5.8 / 7.3	Front-end user interface and build tool
Tailwind CSS / React Router / Motion	4.1 / 7.18 / 12.43	Styling, page routing and animation
pytest	–	Automated tests of the inference contract and the API

3.4.3 Functional Requirements

- Accept a chest X-ray by drag-and-drop, file browsing, paste or a built-in sample gallery (JPG, PNG, BMP, WEBP, TIFF or DICOM, up to 25 MB).
- Reject images that are not chest X-rays with a friendly message, without saving them.

- Return the label, calibrated probability, confidence, decision threshold, risk level, inference time and a Grad-CAM heat map with an opacity slider and side-by-side view.
- Store each scan with optional patient details and doctor's notes; allow search, edit and deletion.
- Generate a patient PDF report with the result, heat map and disclaimer.
- Analyse up to 50 X-rays at once and sort them by pneumonia probability, with CSV export.
- Compute the CURB-65 / CRB-65 score with the recommended care setting.
- Show the model's test performance, figures, limitations and sourced patient information.

3.4.4 Non-Functional Requirements

- **Offline and private:** no internet access at runtime; the server listens on 127.0.0.1 only and all data stay on the local computer.
- **Performance:** a full analysis (three ResNets, two views each, Grad-CAM) in well under one second on the GPU.
- **Reliability:** weights loaded once at start-up with a warm-up prediction; clear error messages for corrupt, empty or oversized files.
- **Usability:** light and dark themes, keyboard accessible, readable on a laptop screen and a projector.
- **Safety and honesty:** a disclaimer on every result screen and PDF; honest reporting of metrics.

3.5 System Architecture

Figure 3.1 shows the overall architecture. It has three layers. The **presentation layer** is a React single-page application that runs in the browser. The **application layer** is a FastAPI server that exposes a JSON REST API, decodes and validates uploaded images, manages the model and builds PDF reports. The **model and data layer** contains the X-ray gate, the ResNet ensemble with calibration and threshold, Grad-CAM and the SQLite database with saved images.

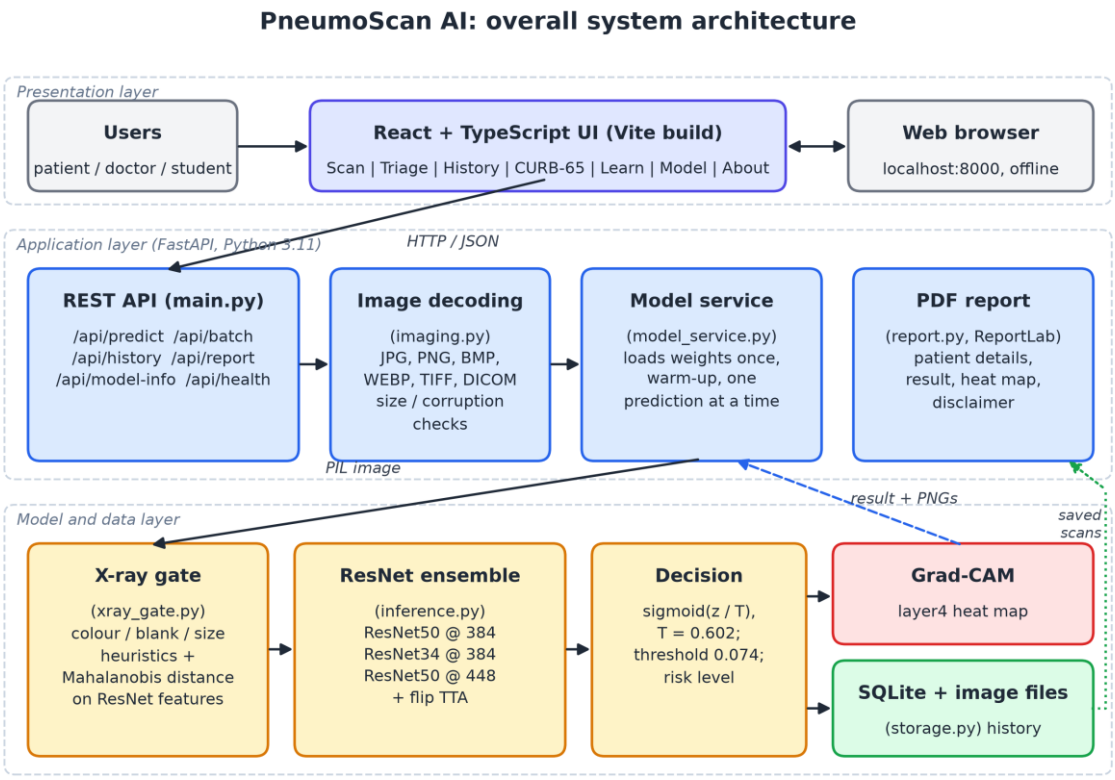


Figure 3.1: Overall system architecture of PneumoScan AI

3.6 Use-Case Diagram

Figure 3.2 shows the three kinds of users and what they can do. A **patient** uploads an X-ray, sees the result with its heat map, downloads a report and reads the Learn page. A **doctor** additionally uses the batch triage queue, manages the scan history and uses the CURB-65 calculator. A **student** mainly uses the scanner, the heat maps, the Learn page and the Model page to understand how the system works.

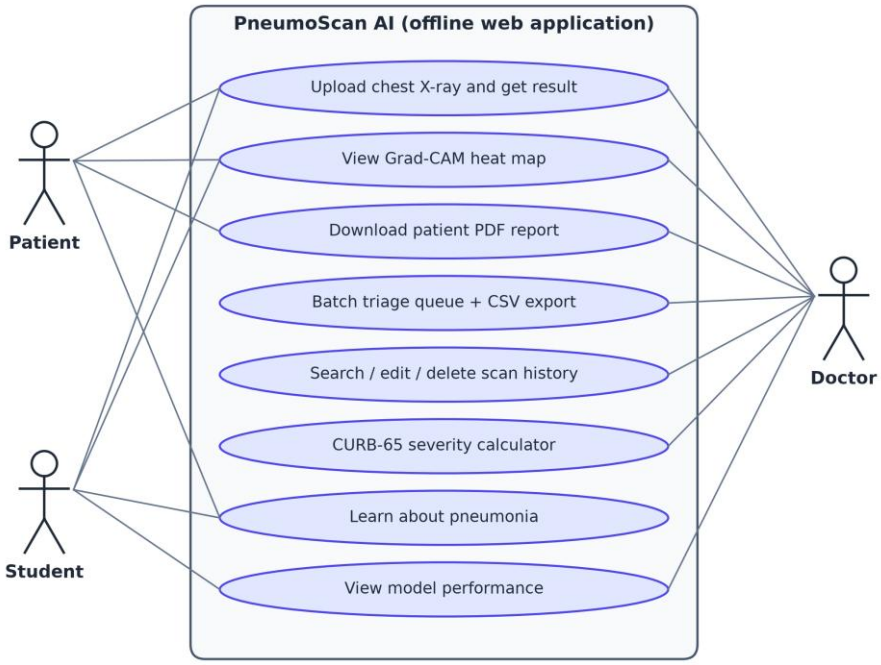


Figure 3.2: Use-case diagram

3.7 Data Flow

Figure 3.3 shows how data flows from the raw dataset to the deployed application. The prediction data flow for a single upload is:

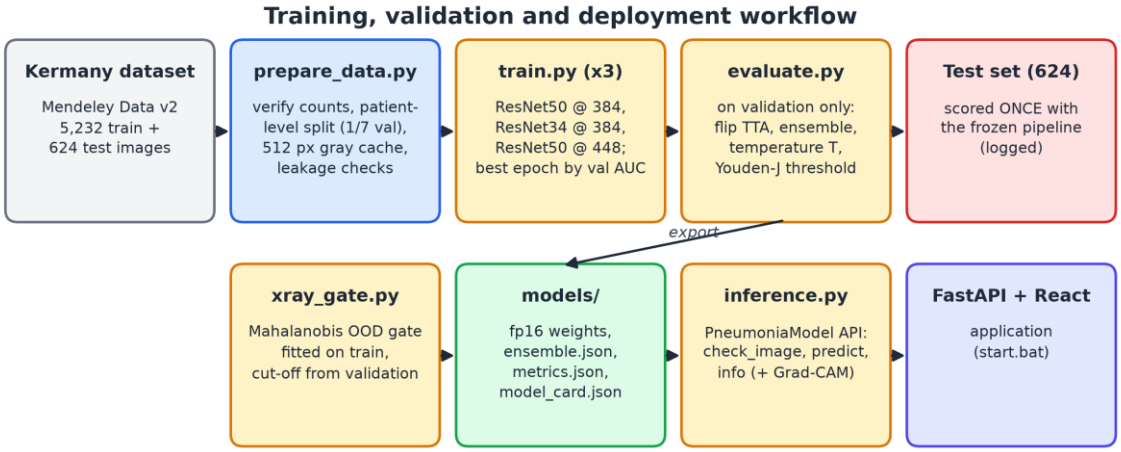


Figure 3.3: Training, validation and deployment data flow

1. The browser sends the image and optional patient details to POST /api/predict as multipart form data.
2. The server checks the size, decodes the image (including DICOM) and applies the EXIF orientation.

3. The X-ray gate runs; if the image is not a chest X-ray, a rejection with the reason is returned and nothing is stored.
4. Each of the three ResNets scores the image and its mirror image; the averaged logit margin is calibrated into a probability and compared with the threshold to give the label and risk level.
5. Grad-CAM maps for the final label are computed, averaged and turned into overlay and heat-map PNG images.
6. The scan record and images are saved in SQLite and on disk, and the full result is returned as JSON for display; the PDF report can then be downloaded from /api/report/{id}.pdf.

3.8 Module Description

Table 3.4: Modules of the system

Module	Source files	Responsibility
Data preparation	ml/prepare_data.py	Verify image counts, patient-level validation split, 512 × 512 grayscale cache, leakage checks
Dataset and augmentation	ml/datasets.py	PyTorch dataset, training augmentation, evaluation transform
Model building and training	ml/common.py, ml/train.py	ResNet with a 2-class head; fine-tuning with AdamW, cosine schedule, AMP, EMA, early stopping
Evaluation and calibration	ml/evaluate.py	Flip TTA, ensemble, temperature scaling, Youden-J threshold on validation, single test scoring, export
Explainability	ml/gradcam.py	Grad-CAM on layer4, colour map and overlay
X-ray gate	ml/xray_gate.py	Heuristics and Mahalanobis out-of-distribution detector
Inference API	ml/inference.py	PneumoniaModel: check_image, predict, info
Back-end API	app/backend/main.py, model_service.py, imaging.py	REST endpoints, model loading and warm-up, safe image decoding
Storage and reports	app/backend/storage.py, report.py	SQLite history and images; PDF reports
Front-end	app/frontend/src	Scan, Triage, History, CURB-65, Learn, Model and About pages

CHAPTER 4
DATASET DETAILS

4.1 Source of the Dataset

The project uses the paediatric chest X-ray dataset released by Kermany et al. together with their *Cell* paper [2], [17]. The images were collected at Guangzhou Women and Children's Medical Center from children aged one to five years, and the labels were assigned by expert physicians. The original release was downloaded from Mendeley Data without any login and its integrity was verified against the SHA-256 hash published by Mendeley. Table 4.1 summarises the dataset.

Table 4.1: Summary of the dataset

Item	Value
Name	Labeled Optical Coherence Tomography (OCT) and Chest X-Ray Images for Classification, Mendeley Data, version 2 (2018) [17]
DOI	10.17632/rscbjbr9sj.2
File used	ChestXRay2017.zip (1,235,512,464 bytes)
Licence	CC BY 4.0 (credit: Kermany et al., Cell, 2018 [2])
Population	Paediatric patients (1–5 years), Guangzhou Women and Children's Medical Center; frontal (anterior–posterior) chest X-rays
Official training set	5,232 images: 1,349 NORMAL and 3,883 PNEUMONIA (2,538 bacterial and 1,345 viral by file name)
Official test set	624 images: 234 NORMAL and 390 PNEUMONIA (matches the official release exactly)
Image format	JPEG; 5,573 grayscale and 283 RGB-encoded grayscale images
Image size	Median 1281 × 888 pixels; width from 384 to 2916 pixels

4.2 Class Distribution and Split

The official test set is kept untouched until the final evaluation. A validation set was carved out of the official training folder with stratified group K-fold (seven folds, shuffled, random seed 42), grouping the images **by patient** so that no child appears in both training and validation. Patient identifiers were parsed from the file names (person123_bacteria_456.jpeg → patient P123; IM-0115-0001.jpeg or NORMAL2-IM-0115-0001.jpeg → patient N0115). One fold (14.3% of the training images) became the validation set. Table 4.2 and Figure 4.1 show the resulting split.

Table 4.2: Images and patients in each split

Split	NORMAL	PNEUMONIA	Total	Patients	Pneumonia share
Train	1,157	3,328	4,485	2,272	74.2%
Validation	192	555	747	378	74.3%
Test (official)	234	390	624	404	62.5%
Total	**1,583**	**4,273**	**5,856**	–	**73.0%**

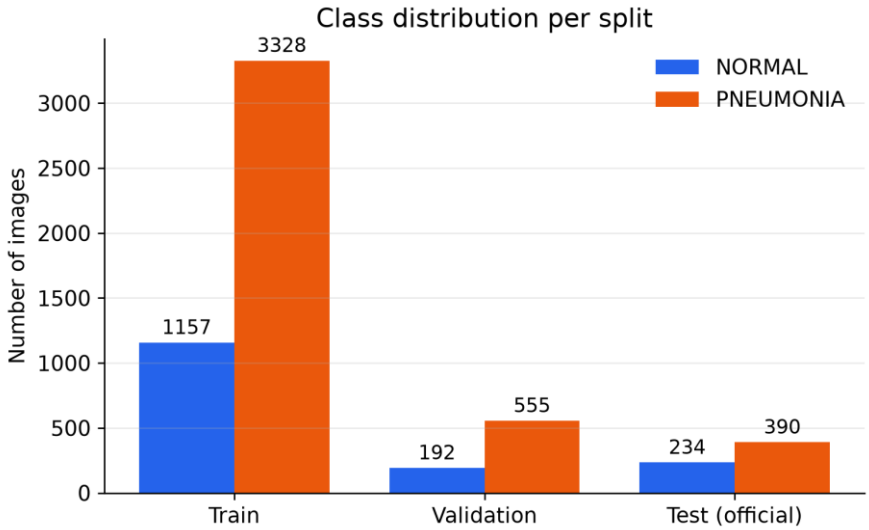


Figure 4.1: Class distribution in the training, validation and test splits

The training data are imbalanced (about three pneumonia images for every normal image), while the test set has a different balance (62.5% pneumonia). This difference matters for the choice of the decision threshold and is discussed in Chapter 7.

4.3 Sample Images

Figure 4.2 shows typical training images. Normal X-rays show clear, dark lung fields with sharp borders of the heart and diaphragm, while pneumonia X-rays show hazy or dense white areas. Many images also contain burned-in text such as the “R” side marker and exposure values.

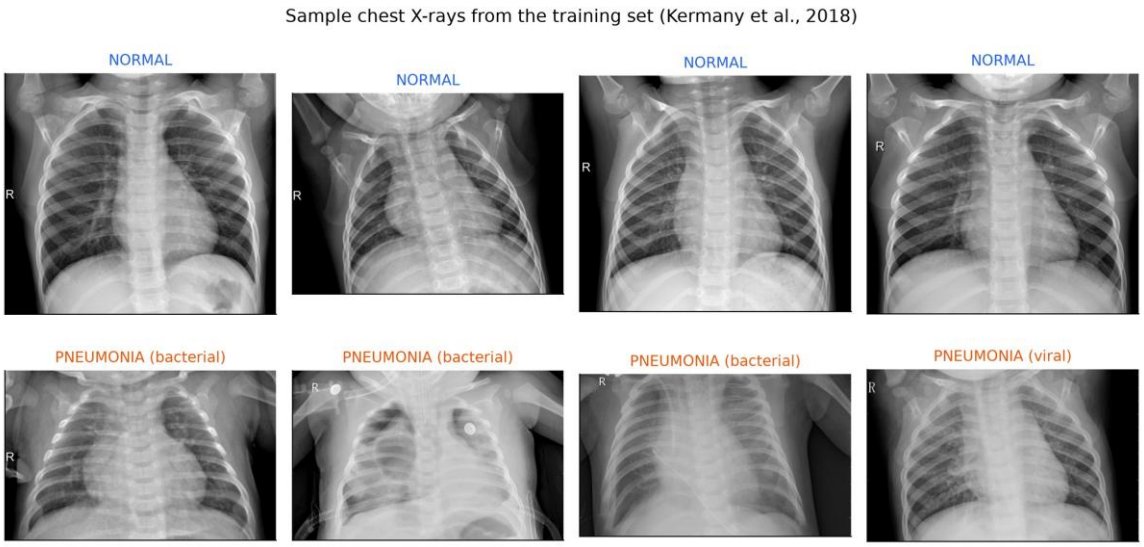


Figure 4.2: Sample training X-rays: four NORMAL (top) and four PNEUMONIA (bottom)

4.4 Pre-processing

The same pre-processing code (ml/common.py) is used for training, evaluation and the application, so the network always sees images in the same way:

1. Any PIL image mode is converted to 8-bit grayscale: 16- and 32-bit images (for example from DICOM) are min–max scaled, transparent pixels are composited on black, the EXIF orientation is applied and colour is converted with the ITU-R 601 luma formula.
2. The image is **squashed** (not cropped) to a 512 × 512 canonical view with Lanczos resampling, so the whole chest is always visible and the heat map maps back exactly onto the original aspect ratio. Training images are cached in this form to speed up loading.
3. The canonical view is resized to the member's input size (384 × 384 or 448 × 448), the gray channel is replicated to three channels, and the values are normalised with the ImageNet mean (0.485, 0.456, 0.406) and standard deviation (0.229, 0.224, 0.225), because the backbones were pretrained on colour ImageNet photographs [23].

4.5 Data Augmentation

To reduce over-fitting, every training image is randomly transformed each time it is used. The augmentations (Table 4.3) imitate realistic variation between X-ray acquisitions; vertical flips are not used because an upside-down chest X-ray is not realistic.

Table 4.3: Training augmentation (torchvision transforms v2)

Transform	Setting	Purpose
RandomResizedCrop	scale 0.75–1.0, aspect ratio 0.9–1.1	Small changes of framing and zoom
RandomHorizontalFlip	p = 0.5	Left–right variation
RandomAffine	rotation ±10°, translation ±5%, scale 0.95–1.05	Patient tilt and position
ColorJitter	brightness ±0.2, contrast ±0.2	Exposure differences
GaussianBlur	kernel 5, σ 0.1–1.2, p = 0.2	Slight motion or focus blur
Gaussian noise	σ = 0.02, p = 0.2	Sensor noise

4.6 Handling Class Imbalance

Instead of discarding images, the loss is weighted by inverse class frequency, $w_c = N / (2 N_c)$, where N is the number of training images and N_c the number in class c. With 1,157 normal and 3,328 pneumonia training images this gives weights of **1.94 for NORMAL** and **0.67 for PNEUMONIA**, so mistakes on the minority class cost about three times more.

4.7 Leakage Checks

Data leakage (the same image or patient in training and evaluation data) would make the results look better than they really are. The checks in Table 4.4 were run automatically by ml/prepare_data.py.

Table 4.4: Leakage checks

Check	Result
Patients in both train and validation	0
Exact duplicate files (SHA-1) between test and train/validation	0
Exact duplicate files between validation and train	0
Near duplicates between test and train/validation (correlation of 64 × 64 thumbnails ≥ 0.99)	0 (highest correlation 0.968)
Patient numbers appearing in both official train and test folders	264 numbers, but no shared images (see text)

The dataset authors number patients separately inside the training and test folders and state that the test set comes from independent patients. Because the exact-hash and near-duplicate checks found no shared images, the 264 matching numbers are a naming coincidence, not leakage.

CHAPTER 5
MODEL ARCHITECTURE AND METHODOLOGY

5.1 Convolutional Neural Networks

A convolutional neural network is built from a few kinds of layers. A **convolution layer** slides small learnable filters (kernels) over the input and computes, at every position, a weighted sum of the pixels under the filter (5.1). Early filters respond to edges and textures; deeper filters combine them into larger patterns such as rib outlines or opacities.

$$(I * K)(i,j) = \sum_m \sum_n I(i + m, j + n)\, K(m,n) \tag{5.1}$$

Batch normalisation standardises the activations of each channel during training, which stabilises and speeds up learning. The **ReLU** activation, ReLU(x) = max(0, x), adds non-linearity. **Pooling** layers reduce the spatial size, and at the end **global average pooling** turns each feature map into a single number. A **fully connected** layer maps these features to one score (logit) per class, and the softmax function turns the logits into probabilities.

5.2 Residual Learning

Simply stacking more layers does not keep improving a plain CNN: deeper plain networks show higher training error than shallower ones (the degradation problem) [1]. A residual block lets a few layers learn a residual function F and adds the input x back through an identity shortcut (5.2). When the number of channels or the spatial size changes, a 1 × 1 projection W_s is used on the shortcut (5.3) [1].

$$y = F(x, \{W_i\}) + x \tag{5.2}$$

$$y = F(x, \{W_i\}) + W_s x \tag{5.3}$$

If the best mapping is close to the identity, the block only needs to push F towards zero, which is easy. During back-propagation the gradient also flows directly through the shortcut, so very deep networks remain trainable.

5.3 Basic and Bottleneck Blocks

ResNet-18 and ResNet-34 use the **basic block**: two 3 × 3 convolutions, each followed by batch normalisation, with ReLU after the first and after the addition. ResNet-50 uses the **bottleneck block** shown in Figure 5.1: a 1 × 1 convolution reduces the number of

channels, a 3 × 3 convolution works on the reduced representation, and a second 1 × 1 convolution restores the channels (for example 256 → 64 → 64 → 256) before the shortcut is added. This keeps deep networks affordable.

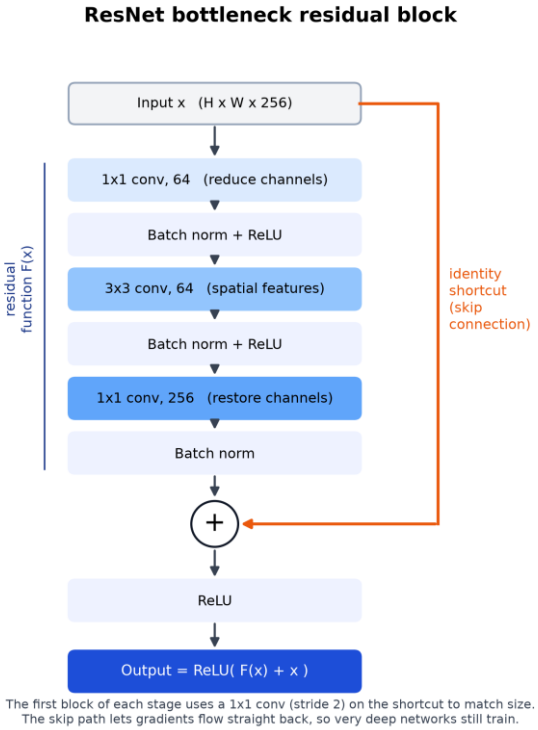


Figure 5.1: Bottleneck residual block with identity shortcut

5.4 ResNet-18, ResNet-34 and ResNet-50

All three depths share the same design: a 7 × 7 stride-2 convolution and a 3 × 3 max-pool, four residual stages (layer1 to layer4) with 64, 128, 256 and 512 base channels, global average pooling and a fully connected layer. They differ in the number and type of blocks (Table 5.1). The training code supports all three; the final ensemble uses ResNet-34 and ResNet-50.

Table 5.1: Comparison of the ResNet variants used in this work

Property	ResNet-18	ResNet-34	ResNet-50
Block type	Basic (2 × 3×3)	Basic (2 × 3×3)	Bottleneck (1×1, 3×3, 1×1)
Blocks in layer1–layer4	2, 2, 2, 2	3, 4, 6, 3	3, 4, 6, 3
Weighted layers	18	34	50
Channels after layer4	512	512	2048
Parameters with our 2-class head	11.18 M	21.29 M	23.51 M
FLOPs at 224 × 224 [1]	1.8×10^9	3.6×10^9	3.8×10^9
ImageNet weights (torchvision)	IMAGENET1K_V1	IMAGENET1K_V1	IMAGENET1K_V2
Used in final ensemble	No	Yes (384 px)	Yes (384 px and 448 px)

5.5 Transfer Learning

Training a deep network from random weights needs far more labelled images than the 4,485 training X-rays available. Transfer learning starts instead from weights learned on ImageNet [23], whose early filters (edges, textures, shapes) are useful for almost any image. The 1000-class ImageNet layer is replaced by a new head, Dropout(0.2) followed by Linear(features, 2), and **all layers are fine-tuned** end-to-end. The pretrained backbone uses a learning rate three times smaller than the new head, so the useful pretrained features are adapted gently while the head learns quickly.

5.6 Complete Pipeline

Figure 5.2 shows the full pipeline with the output size of every ResNet-50 stage at 384 × 384 input. The X-ray is pre-processed, passed through three fine-tuned ResNets (each on the original and the mirrored image), the logit margins are averaged and calibrated, the threshold gives the decision, and Grad-CAM uses the layer4 activations and gradients to produce the heat map.

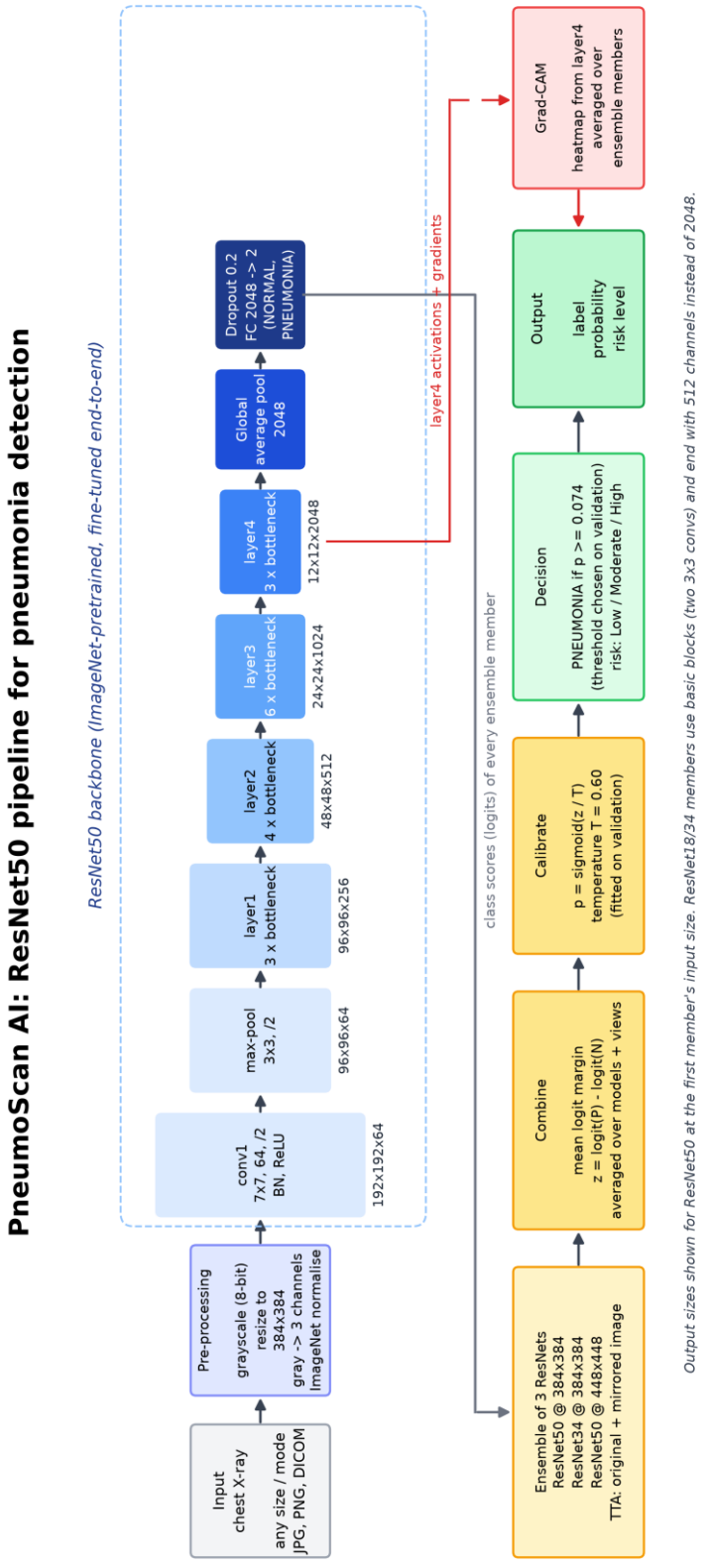


Figure 5.2: Architecture of the PneumoScan AI pipeline (ResNet-50 member shown; rotated to fit the page)

5.7 Training Setup

5.7.1 Loss Function

The loss is cross-entropy with class weights w_c (Section 4.6) and label smoothing ε = 0.1 [24]. Label smoothing replaces the one-hot target with q_c = (1 − ε)·1[c = y] + ε/K, with K = 2 classes, which discourages over-confident predictions:

$$L = -\sum_{c=1}^{K} w_c \, q_c \log p_c \tag{5.4}$$

5.7.2 Optimiser and Learning-Rate Schedule

AdamW [25] (Adam with decoupled weight decay 0.01) is used with a base learning rate of 3×10^{-4} for the head and 1×10^{-4} (0.33×) for the backbone. The learning rate rises linearly during one warm-up epoch and then follows a cosine decay that is updated every step (5.5), where s is the current step, s_w the number of warm-up steps and S the total number of steps. Gradients are clipped to a maximum norm of 2.

$$\eta_s = \eta_0 \cdot \frac{1}{2}\left(1 + \cos\left(\pi \, \frac{s - s_w}{S - s_w}\right)\right) \tag{5.5}$$

5.7.3 Mixed Precision, EMA and Early Stopping

Training uses automatic mixed precision (fp16 autocast with a gradient scaler) [26] to save memory and time on the 6 GB GPU. An exponential moving average (EMA) of the weights ((5.6), decay d = 0.998 with a short warm-up) gives a smoother model; the EMA weights are the ones evaluated and saved. After every epoch the validation ROC-AUC is computed; the checkpoint with the best validation AUC (ties broken by lower validation loss) is kept, and training stops early after 7 epochs without improvement.

$$\theta_{\text{EMA}} \leftarrow d \cdot \theta_{\text{EMA}} + (1 - d) \cdot \theta \tag{5.6}$$

Table 5.2 lists the complete training configuration.

Table 5.2: Training hyperparameters (identical for all members unless stated)

Parameter	Value
Members	ResNet-50 at 384 px (batch 24, seed 1); ResNet-34 at 384 px (batch 32, seed 2); ResNet-50 at 448 px (batch 16, seed 3)
Classification head	Dropout(0.2) → Linear(features, 2); all layers trainable
Loss	Weighted cross-entropy (NORMAL 1.94, PNEUMONIA 0.67), label smoothing 0.1
Optimiser	AdamW, weight decay 0.01
Learning rate	Head 3×10^{-4}, backbone 1×10^{-4}; 1 warm-up epoch, then per-step cosine decay
Gradient clipping	Maximum norm 2.0
Precision	Mixed precision (fp16 autocast + GradScaler)
Weight averaging	EMA, decay 0.998 (with warm-up); EMA weights evaluated and saved
Epochs	Up to 20; early stopping with patience 7 on validation ROC-AUC
Data loading	Shuffled, drop last incomplete batch, 3 worker processes
Shipped weights	fp16 state dictionaries (41–46 MB each); all metrics computed with these weights

5.8 Ensemble and Test-Time Augmentation

For each member m and view v (the original image and its horizontal mirror), the network outputs two logits; their difference is the **logit margin** (5.7). The margins of the M = 3 members and V = 2 views are averaged (5.8). Averaging several models that make different mistakes reduces the variance of the prediction, and the ensemble of all trained members was fixed as the final model before the test set was scored.

$$z_{m,v} = \ell_{\text{PNEUMONIA}} - \ell_{\text{NORMAL}} \tag{5.7}$$

$$\overline{z} = \frac{1}{M\,V} \sum_{m=1}^{M} \sum_{v=1}^{V} z_{m,v} \tag{5.8}$$

5.9 Temperature Scaling

Modern networks are often over-confident. Temperature scaling [27] divides the averaged margin by a single temperature T before the sigmoid (5.9). T is fitted by minimising the negative log-likelihood on the **validation** set (5.10); for the ensemble T = 0.602. Temperature scaling does not change the ranking of images, only how well the probability values match reality.

$$p = \sigma\left(\frac{\overline{z}}{T}\right) = \frac{1}{1 + e^{-\overline{z}/T}} \tag{5.9}$$

$$T^* = \arg\min_T -\frac{1}{N}\sum_{n=1}^{N}[y_n \log p_n + (1 - y_n)\log(1 - p_n)] \quad (5.10)$$

5.10 Threshold Selection and Risk Levels

The decision threshold t is chosen on the validation set to maximise Youden's J statistic [28] (5.11), which is equivalent to maximising balanced accuracy. When a whole interval of thresholds gives the same best J, the middle of the interval (in logit space) is taken. This rule was written into ml/evaluate.py before the test set was ever scored, and it gave **t = 0.0743**. An image is labelled PNEUMONIA when p ≥ t.

$$J(t) = \text{Sensitivity}(t) + \text{Specificity}(t) - 1 \quad (5.11)$$

Because the threshold is low, a PNEUMONIA label can come with a modest probability. The application therefore also reports a **risk level** whose bands are defined relative to t (Table 5.3).

Table 5.3: Risk levels derived from the calibrated probability p (t = 0.0743)

Risk level	Rule	Probability range	Meaning
Low	p < t / 2	p < 0.037	Normal side, well below the threshold
Moderate	t / 2 ≤ p < (1 + t) / 2	0.037 ≤ p < 0.537	Borderline or possible pneumonia; see a doctor
High	p ≥ (1 + t) / 2	p ≥ 0.537	Pneumonia likely; prompt medical review

5.11 Grad-CAM Explanations

Grad-CAM [4] explains a prediction by finding which regions of the last convolutional feature maps increased the class score. Let A^k be the k-th feature map of layer4 (2048 maps of 12 × 12 for ResNet-50 at 384 px) and y^c the score for class c. The importance of each map is the spatial average of the gradient (5.12), where Z is the number of positions, and the heat map is the ReLU of the weighted sum of the maps (5.13).

$$\alpha_k^c = \frac{1}{Z}\sum_i\sum_j \frac{\partial y^c}{\partial A_{ij}^k} \quad (5.12)$$

$$L^c_{\text{Grad-CAM}} = \text{ReLU}\left(\sum_k \alpha_k^c A^k\right) \quad (5.13)$$

In this project the score for PNEUMONIA is the logit margin z, and the score for NORMAL is −z, so one backward pass gives the map for either class. The map for the

ensemble's final label is computed for every member, resized to the display image, averaged over the members and normalised to [0, 1]. It is drawn with a "jet" colour map (blue = low, red = high) whose transparency grows with importance, and overlaid on the X-ray at 60% opacity.

5.12 X-ray Gate (Out-of-Distribution Detection)

A classifier trained only on chest X-rays will still output NORMAL or PNEUMONIA for a photograph of a cat. To prevent this, every image first passes two checks. (1) **Heuristics:** images smaller than 64 pixels, nearly uniform images (gray-level standard deviation below 8) and colourful images (mean saturation above 0.12 or more than 5% strongly coloured pixels) are rejected. (2) **Feature distance:** the 2048-dimensional pooled layer4 features f(x) of the first ResNet-50 member are compared with the class means μ_c of the training X-rays using a Mahalanobis distance with a shared Ledoit-Wolf covariance Σ [29], [30] (5.14). The cut-off τ is 1.15 times the 99.5th percentile of the validation distances (τ = 66.04), and the distance is mapped to a score between 0 and 1 (5.15), which equals 0.5 exactly at the cut-off.

$$d(x) = \min_c \sqrt{(f(x) - \mu_c)^T \Sigma^{-1} (f(x) - \mu_c)} \tag{5.14}$$

$$s(x) = \frac{1}{1 + \left(\frac{d(x)}{\tau}\right)^4} \tag{5.15}$$

5.13 Evaluation Metrics

With PNEUMONIA as the positive class, TP, TN, FP and FN denote true positives, true negatives, false positives and false negatives. The metrics reported in Chapter 7 are:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{Precision} = \frac{TP}{TP + FP} \tag{5.16}$$

$$\text{Sensitivity} = \frac{TP}{TP + FN}, \quad \text{Specificity} = \frac{TN}{TN + FP} \tag{5.17}$$

$$F_1 = \frac{2 \cdot Precision \cdot Sensitivity}{Precision + Sensitivity} \tag{5.18}$$

Sensitivity (recall) is the share of pneumonia X-rays that are detected, and specificity is the share of normal X-rays correctly called normal. The **ROC-AUC** is the area under the curve of sensitivity against (1 − specificity) over all thresholds; it measures how well the

model ranks pneumonia above normal images, independent of the threshold. The **PR-AUC** (average precision) summarises the precision–recall curve.

CHAPTER 6
IMPLEMENTATION

6.1 Development Environment

The project was developed on Windows 11 with Python 3.11.9 in a virtual environment (.venv) and Node.js 24 for the front-end. Training and evaluation ran on the NVIDIA RTX 3050 Laptop GPU with PyTorch 2.11.0 and CUDA 12.8 (Chapter 3, Table 3.2 and Table 3.3). Every step of the machine-learning pipeline is a separate script that is run from the project root, for example **.venv\Scripts\python.exe -m ml.train --name r50_384_s1 --arch resnet50 --res 384 --bs 24 --seed 1**. This chapter explains the most important parts of the code; the complete source of the pipeline and the back-end is given in Appendix A.

6.2 Project Structure

Listing 6.1: Project folder structure

```
pneumoscan-ai/
├── start.bat                double-click launcher (Windows)
├── data/                    raw images, 512 px cache, splits.csv, dataset summary
├── ml/                      machine-learning pipeline
│   ├── common.py            paths, image conversion, build_model, metrics
│   ├── prepare_data.py      counts, patient-level split, cache, leakage checks
│   ├── datasets.py          PyTorch Dataset + augmentation
│   ├── train.py             fine-tuning loop (AdamW, cosine, AMP, EMA)
│   ├── evaluate.py          TTA, ensemble, temperature, threshold, export
│   ├── gradcam.py           Grad-CAM on layer4
│   ├── xray_gate.py         "is this a chest X-ray?" detector
│   ├── inference.py         PneumoniaModel API used by the app
│   ├── make_figures.py      report figures
│   └── test_inference.py    pytest contract tests
├── models/                  fp16 weights (3 members), ensemble.json,
│                            metrics.json, model_card.json, xray_gate.npz
├── app/
│   ├── backend/             FastAPI server
│   │   ├── main.py          API routes + serves the built front-end
│   │   ├── model_service.py loads the model once, warm-up
│   │   ├── imaging.py       safe decoding (JPG/PNG/BMP/WEBP/TIFF/DICOM)
│   │   ├── storage.py       SQLite history + saved images
│   │   ├── report.py        patient PDF (ReportLab)
│   │   └── tests/           pytest suite for the API
│   └── frontend/            React + TypeScript + Vite + Tailwind CSS
│       └── src/             pages/, components/, lib/, content/
├── sample_xrays/            demo X-rays from the TEST split + non-X-rays
├── docs/                    plan, research notes, figures, screenshots
└── report/                  this report (generator scripts, .docx, .pdf)
```

6.3 Data Preparation

ml/prepare_data.py first checks that the extracted dataset has exactly the official number of images per split and class. Listing 6.2 shows how the patient identifier is parsed from

the file name and how a patient-level, stratified validation fold is carved out of the training images with scikit-learn's StratifiedGroupKFold.

Listing 6.2: Patient identifier and patient-level validation split (ml/prepare_data.py)

```
def patient_id(filename: str, label: str) -> str:
    """Patient identifier parsed from the file name.

    PNEUMONIA: person123_bacteria_456.jpeg -> P123
    NORMAL   : IM-0115-0001.jpeg / NORMAL2-IM-0115-0001.jpeg -> N0115
    (the NORMAL2- prefix is ignored on purpose: grouping more conservatively can only
    reduce leakage between train and validation, never add it).
    """
    if label == "PNEUMONIA":
        m = re.match(r"person(\d+)_", filename)
        return f"P{int(m.group(1))}"
    m = re.search(r"IM-(\d+)-", filename)
    return f"N{int(m.group(1))}"

# ... inside main():
# Patient-level, stratified validation split carved out of TRAIN only.
train_idx = df.index[df.split == "train"]
sgkf = StratifiedGroupKFold(n_splits=VAL_FOLDS, shuffle=True, random_state=SEED)
_, val_pos = next(sgkf.split(train_idx, df.loc[train_idx, "y"], df.loc[train_idx,
"patient"]))
df.loc[train_idx[val_pos], "split"] = "val"
```

6.4 Dataset and Augmentation

Listing 6.3 shows the training and evaluation transforms built with torchvision's transforms v2. Images are read from the 512 × 512 grayscale cache, augmented, replicated to three channels and normalised.

Listing 6.3: Training and evaluation transforms (ml/datasets.py)

```
def train_transform(res: int) -> T.Compose:
    """Realistic X-ray augmentations: small crops, tilts, exposure changes, blur,
noise."""
    return T.Compose([
        T.ToImage(), T.ToDtype(torch.float32, scale=True),
        T.RandomResizedCrop(res, scale=(0.75, 1.0), ratio=(0.9, 1.1), antialias=True),
        T.RandomHorizontalFlip(p=0.5),
        T.RandomAffine(degrees=10, translate=(0.05, 0.05), scale=(0.95, 1.05)),
        T.ColorJitter(brightness=0.2, contrast=0.2),
        T.RandomApply([T.GaussianBlur(kernel_size=5, sigma=(0.1, 1.2))], p=0.2),
        T.RandomApply([AddGaussianNoise(0.02)], p=0.2),
        *to_network_input(),
    ])


def eval_transform(res: int) -> T.Compose:
    """Deterministic resize used for validation, test and the app."""
    return T.Compose([
        T.ToImage(), T.ToDtype(torch.float32, scale=True),
        T.Resize((res, res), antialias=True),
        *to_network_input(),
    ])
```

6.5 Building the Model

Listing 6.4 loads a torchvision ResNet with its ImageNet weights and replaces the 1000-class layer with a dropout layer and a two-class linear layer.

Listing 6.4: Building a ResNet with a two-class head (ml/common.py)

```
def build_model(arch: str, pretrained: bool = True):
    """torchvision ResNet with a 2-class head (NORMAL, PNEUMONIA)."""
    import torch.nn as nn
    import torchvision.models as tvm

    weights = {
        "resnet18": tvm.ResNet18_Weights.IMAGENET1K_V1,
        "resnet34": tvm.ResNet34_Weights.IMAGENET1K_V1,
        "resnet50": tvm.ResNet50_Weights.IMAGENET1K_V2,
    }[arch] if pretrained else None
    model = getattr(tvm, arch)(weights=weights)
    model.fc = nn.Sequential(nn.Dropout(0.2), nn.Linear(model.fc.in_features,
len(CLASSES)))
    return model
```

6.6 Training Loop

Listing 6.5 is the core of ml/train.py. For every step the learning rate is set from the warm-up and cosine schedule, the forward pass runs under fp16 autocast, the scaled loss is back-propagated, gradients are clipped, the optimiser steps and the EMA copy of the weights is updated. After each epoch the EMA model is evaluated on validation, and the best checkpoint by validation ROC-AUC is saved.

Listing 6.5: One training epoch (ml/train.py, excerpt)

```
for x, y in train_dl:
    # Cosine schedule with linear warm-up, updated every step.
    f = (step + 1) / max(warm, 1) if step < warm else \
        0.5 * (1 + math.cos(math.pi * (step - warm) / max(total - warm, 1)))
    for g, lr0 in zip(opt.param_groups, base_lrs):
        g["lr"] = lr0 * f
    x = x.to(device, non_blocking=True)
    y = y.to(device, non_blocking=True)
    with torch.autocast("cuda", dtype=torch.float16, enabled=device.type == "cuda"):
        logits = model(x)
        loss = criterion(logits.float(), y)
    opt.zero_grad(set_to_none=True)
    scaler.scale(loss).backward()
    scaler.unscale_(opt)
    nn.utils.clip_grad_norm_(model.parameters(), 2.0)
    scaler.step(opt)
    scaler.update()
    if ema:
        ema.update(model)
    step += 1
    run_loss += loss.item() * len(y)
    run_correct += (logits.argmax(1) == y).sum().item()
    seen += len(y)
```

6.7 Evaluation, Calibration and Threshold

ml/evaluate.py computes the flip-TTA logit margins of every member, averages them, and then fits the temperature and the decision threshold **on the validation split only** (Listing 6.6). The test split is scored only with the --export option, and every test scoring is appended to logs/test_evaluations.jsonl so that the number of test evaluations can be audited (it is exactly one).

Listing 6.6: Temperature scaling and Youden-J threshold (ml/evaluate.py)

```
def fit_temperature(z: np.ndarray, y: np.ndarray) -> float:
    """Temperature T minimising validation negative log-likelihood of sigmoid(z / T)."""
    def nll(log_t):
        p = 1 / (1 + np.exp(-z / np.exp(log_t)))
        p = np.clip(p, 1e-7, 1 - 1e-7)
        return -np.mean(y * np.log(p) + (1 - y) * np.log(1 - p))
    return float(np.exp(minimize_scalar(nll, bounds=(-3, 3), method="bounded").x))


def pick_threshold(prob: np.ndarray, y: np.ndarray) -> float:
    """Threshold maximising Youden's J (= balanced accuracy) on validation.

    Ties are common (a whole interval of thresholds gives the same J), so we take
    the middle of the best interval, measured in logit space.
    """
    cands = np.unique(np.concatenate([prob, [0.0, 1.0]]))
    mids = (cands[:-1] + cands[1:]) / 2
    j = np.array([np.mean(prob[y == 1] >= t) + np.mean(prob[y == 0] < t) - 1 for t in
mids])
    best = mids[np.isclose(j, j.max())]
    logit = np.log(np.clip(best, 1e-6, 1 - 1e-6) / (1 - np.clip(best, 1e-6, 1 - 1e-6)))
    return float(1 / (1 + np.exp(-(logit.min() + logit.max()) / 2)))
```

6.8 Grad-CAM

Listing 6.7 registers a forward hook on layer4 to keep its activations, runs one forward and one backward pass of the logit margin, and builds the class map from the gradient-weighted activations.

Listing 6.7: Grad-CAM for torchvision ResNets (ml/gradcam.py)

```
class GradCAM:
    def __init__(self, model: torch.nn.Module, layer: torch.nn.Module | None = None):
        self.model = model
        self._acts = None
        (layer if layer is not None else model.layer4).register_forward_hook(self._save)

    def _save(self, _module, _inp, out):
        self._acts = out

    def explain(self, x: torch.Tensor):
        """Forward + backward once. Returns (activations, d z / d activations, logits)."""
        self.model.eval()
        with torch.enable_grad():
            x = x.detach().requires_grad_(True)  # lets gradients flow even if weights are
frozen
            logits = self.model(x)
            z = logits[:, 1] - logits[:, 0]
            grads = torch.autograd.grad(z.sum(), self._acts)[0]
        return self._acts.detach(), grads.detach(), logits.detach()

    @staticmethod
    def cam(acts: torch.Tensor, grads: torch.Tensor, class_idx: int, size: tuple[int,
int]) -> np.ndarray:
        """Grad-CAM map in [0, 1] for class_idx (0 NORMAL, 1 PNEUMONIA) resized to
size=(H, W)."""
        sign = 1.0 if class_idx == 1 else -1.0
        weights = sign * grads.float().mean(dim=(2, 3), keepdim=True)
        m = F.relu((weights * acts.float()).sum(1, keepdim=True))
        m = F.interpolate(m, size=size, mode="bilinear", align_corners=False)[0,
0].cpu().numpy()
        return m / m.max() if m.max() > 0 else m
```

6.9 Inference API

The application uses a single class, PneumoniaModel (ml/inference.py), which loads the three members once, checks the image with the X-ray gate and returns the label, calibrated probability, risk level and three aligned PNG images (input, heat map and overlay). Listing 6.8 shows how the ensemble decision is made inside predict().

Listing 6.8: Ensemble decision inside PneumoniaModel.predict (ml/inference.py, excerpt)

```
margins, explained = [], []
for m in self.members:
    x = self._tensor(m, view)
    with torch.autocast("cuda", dtype=torch.float16, enabled=self.amp):
        if with_gradcam:
            acts, grads, lg = m["cam"].explain(x)
            explained.append((acts, grads, x.shape[-2:]))
        else:
            with torch.no_grad():
                lg = m["net"](x)
        views = [lg]
        if self.cfg.get("tta_hflip", True):
            with torch.no_grad():
                views.append(m["net"](torch.flip(x, dims=[3])))
    margins.append(float(np.mean([float(v[0, 1] - v[0, 0]) for v in views])))
z = float(np.mean(margins))
prob = float(1 / (1 + np.exp(-z / self.cfg["temperature"])))
thr = float(self.cfg["threshold"])
label = CLASSES[int(prob >= thr)]
```

6.10 X-ray Gate

The feature-distance part of the gate is shown in Listing 6.9; the parameters (class means, precision matrix and cut-off) are stored in models/xray_gate.npz.

Listing 6.9: Mahalanobis distance and decision of the X-ray gate (ml/xray_gate.py)

```
def distance(self, feats: np.ndarray) -> np.ndarray:
    """Min over classes of sqrt(Mahalanobis distance); feats: (N, D)."""
    feats = np.atleast_2d(feats).astype(np.float64)
    d = [np.einsum("nd,de,ne->n", feats - m, self.precision, feats - m) for m in
self.means]
    return np.sqrt(np.maximum(np.min(d, axis=0), 0))

def score(self, dist: float) -> float:
    """Map distance to 0..1 (1 = typical chest X-ray, 0.5 exactly at the cut-off)."""
    return float(1 / (1 + (dist / self.threshold) ** 4))

def decide(self, img: Image.Image, feats: np.ndarray | None) -> dict:
    ok, reason, stats = heuristics(img)
    if not ok:
        return {"is_xray": False, "score": 0.0, "reason": reason}
    dist = float(self.distance(feats)[0])
    s = self.score(dist)
    if dist > self.threshold:
        return {"is_xray": False, "score": s,
                "reason": "This does not look like a chest X-ray (it is unlike the X-rays
the model was trained on)."}
    return {"is_xray": True, "score": s, "reason": "Looks like a chest X-ray."}
```

6.11 Back-end API (FastAPI)

The back-end (app/backend/main.py) exposes the JSON API listed in Table 6.1 and also serves the built React front-end, so the whole application runs from one local server on port 8000. Errors are always returned in the same shape, {"error": {"code", "message"}}, with a friendly message.

Table 6.1: REST API endpoints

Method	Path	Purpose
GET	/api/health	Status, model mode, device and model version
GET	/api/model-info	metrics.json, model_card.json and figure URLs
POST	/api/predict	One X-ray (+ optional patient details) → result and saved scan
POST	/api/batch	Up to 50 X-rays → results sorted by pneumonia probability
GET	/api/history	Search and page through saved scans
GET / PATCH / DELETE	/api/history/{id}	Read, edit patient details or notes, delete a scan
DELETE	/api/history	Clear all history
GET	/api/history/{id}/image/{kind}	Saved input, overlay, heat-map or thumbnail image
GET	/api/report/{id}.pdf	Patient PDF report
GET	/api/samples	Demo X-rays from the test split

Listing 6.10 shows the prediction endpoint. The upload is size-checked, the patient fields are validated, and the CPU/GPU-heavy analysis runs in a worker thread so that the server stays responsive.

Listing 6.10: Prediction endpoint (app/backend/main.py, excerpt)

```
@app.post("/api/predict")
async def predict(
    file: UploadFile = File(...),
    name: str = Form(""),
    age: str = Form(""),
    sex: str = Form(""),
    patient_id: str = Form(""),
    symptoms: str = Form(""),
    notes: str = Form(""),
    source: str = Form("scan"),
    ground_truth: str = Form(""),
):
    model.require_ready()
    patient = _patient_from_form(name, age, sex, patient_id, symptoms)
    notes_clean = _clean_text(notes, "doctor_notes")
    src = source if source in SOURCES else "scan"
    gt = ground_truth.strip().upper() or None
    if gt not in (None, "NORMAL", "PNEUMONIA"):
        gt = None
    data = await _read_upload(file)
    fname = Path(file.filename or "upload").name
    return await run_in_threadpool(_analyse, data, fname, source=src, patient=patient,
                                   notes=notes_clean, ground_truth=gt)
```

6.12 Front-end (React + TypeScript)

The front-end is a React 19 single-page application written in TypeScript, styled with Tailwind CSS and built with Vite into static files. All fonts are bundled, so nothing is loaded from the internet. A typed client (src/lib/api.ts) wraps every endpoint. Listing 6.11

shows the component that draws the pneumonia probability on a 0–100 ruler with the decision threshold marked, which helps users see how close a result is to the boundary.

Listing 6.11: ProbabilityRuler component (app/frontend/src/components/ProbabilityRuler.tsx)

```
import { motion, useReducedMotion } from 'motion/react'

/**
 * Probability of pneumonia on a 0-100 ruler with the decision threshold marked.
 * The threshold tick shows where the model switches from "Normal" to "Pneumonia".
 */
export function ProbabilityRuler({ prob, threshold }: { prob: number; threshold: number }) {
  const reduce = useReducedMotion()
  const p = Math.min(1, Math.max(0, prob))
  const t = Math.min(1, Math.max(0, threshold))
  const pneu = p >= t
  const ticks = Array.from({ length: 11 }, (_, i) => i / 10)
  const tText = `${(t * 100).toFixed(t < 0.1 ? 1 : 0)}%`
  const thrLeft = t > 0.82 ? 'right' : t < 0.18 ? 'left' : 'center'

  return (
    <div>
      <div className="flex items-end justify-between gap-4">
        <div>
          <p className="eyebrow">Pneumonia probability</p>
          <p className="mt-1 font-display text-[3.4rem] leading-none font-[640] tracking-[-
0.04em] tabular sm:text-[4rem]">
            {(p * 100).toFixed(1)}
            <span className="ml-0.5 text-[0.5em] font-[560] text-ink-3">%</span>
          </p>
        </div>
      </div>

      <div
        className="relative mt-7 mb-2"
        role="meter"
        aria-valuemin={0}
        aria-valuemax={100}
        aria-valuenow={Math.round(p * 1000) / 10}
        aria-label={`Pneumonia probability ${(p * 100).toFixed(1)} percent; decision threshold
${tText}`}
      >
        {/* threshold label */}
        <div
          className="absolute -top-6 font-mono text-[0.68rem] tracking-[0.06em] whitespace-
nowrap text-ink-3 uppercase"
          style={{
            left: `${t * 100}%`,
            transform: thrLeft === 'center' ? 'translateX(-50%)' : thrLeft === 'right' ?
'translateX(-100%)' : 'none',
          }}
        >
          Threshold {tText}
        </div>
        {/* track */}
        <div className="relative h-3 overflow-hidden rounded-full bg-sunken ring-1 ring-line
ring-inset">
          <div className="absolute inset-y-0 right-0 bg-flag/10" style={{ left: `${t * 100}%` }}
aria-hidden />
          <motion.div
            className={`absolute inset-y-0 left-0 rounded-full ${pneu ? 'bg-flag' : 'bg-
clear'}`}
            initial={reduce ? false : { width: '0%' }}
            animate={{ width: `${p * 100}%` }}
            transition={{ duration: 0.9, ease: [0.2, 0.8, 0.2, 1], delay: 0.1 }}
          />
        </div>
        {/* threshold tick */}
        <div className="absolute -top-1.5 -bottom-1.5 w-[2px] rounded bg-ink" style={{ left:
`calc(${t * 100}% - 1px)` }} aria-hidden />
```

```
        {/* marker */}
        <motion.div
          className="absolute top-1/2 size-5 -translate-x-1/2 -translate-y-1/2 rounded-full
border-[3px] border-surface shadow-2"
          style={{ background: pneu ? 'var(--flag)' : 'var(--clear)' }}
          initial={reduce ? false : { left: '0%' }}
          animate={{ left: `${p * 100}%` }}
          transition={{ duration: 0.9, ease: [0.2, 0.8, 0.2, 1], delay: 0.1 }}
          aria-hidden
        />
      </div>

      {/* scale */}
      <div className="relative mt-3 h-5" aria-hidden>
        {ticks.map((v) => (
          <span key={v} className="absolute top-0 flex -translate-x-1/2 flex-col items-center"
style={{ left: `${v * 100}%` }}>
            <span className={`w-px bg-line-strong ${v === 0 || v === 0.5 || v === 1 ? 'h-2' :
'h-1'}`} />
            {(v === 0 || v === 0.5 || v === 1) && (
              <span className="mt-0.5 font-mono text-[0.65rem] text-ink-3">{v * 100}</span>
            )}
          </span>
        ))}
      </div>
      <div className="mt-1 flex justify-between text-[0.78rem] text-ink-3">
        <span>Normal side</span>
        <span>Pneumonia side</span>
      </div>
    </div>
  )
}
```

6.13 Testing

Automated tests were written for both parts of the system. **ml/test_inference.py** checks the inference contract: the keys and types of the result, the PNG sizes and aspect ratio, all PIL image modes (L, RGB, RGBA, LA, P, I;16, I, F, CMYK, 1), tiny and very large images, 16-bit DICOM input, rejection of photographs, blank and noise images, the sample answer key and info(); all **24 tests passed**. In addition, running predict() one image at a time on all 624 test files reproduced the batch evaluation (maximum probability difference 0.00047, no label changes). The back-end test suite (app/backend/tests) covers every endpoint, bad and huge files, synthetic DICOM files, mixed batches, history create/read/update/delete, PDF generation with long and Unicode text, and the switch between the real model and the mock model used during development.

CHAPTER 7
RESULTS AND DISCUSSION

7.1 Experimental Protocol

All decisions (number of epochs, ensemble members, test-time augmentation, temperature and threshold) were made on the 747-image validation split. The final pipeline, a pre-committed ensemble of every trained member, was then applied **once** to the official 624-image test set on 7 November 2026; in the same run each member was also scored with its own validation-chosen settings. Nothing was re-trained, re-thresholded or re-selected afterwards. All numbers below are copied from models/metrics.json.

7.2 Training Behaviour

Table 7.1 summarises the three training runs and Figure 7.1 to Figure 7.3 show the training and validation curves. All runs converged within a few epochs: validation ROC-AUC exceeded 0.99 after the first or second epoch and peaked at epoch 7–10. Afterwards training accuracy kept rising towards 99% while validation AUC flattened or fell slightly, so early stopping ended each run between epochs 14 and 17. (Loss values stay around 0.3 because label smoothing prevents the loss from approaching zero.)

Table 7.1: Training runs on the RTX 3050 GPU

Run	Model	Input	Batch	Seed	Epochs run	Best epoch	Time (min)	Best val AUC*
r50_384_s1	ResNet-50	384	24	1	14	7	17.7	0.9965
r34_384_s2	ResNet-34	384	32	2	17	10	13.9	0.9971
r50_448_s3	ResNet-50	448	16	3	14	7	23.9	0.9974

** Validation ROC-AUC of the EMA weights at the best epoch, single view, during training. Total training time 55.5 minutes.*

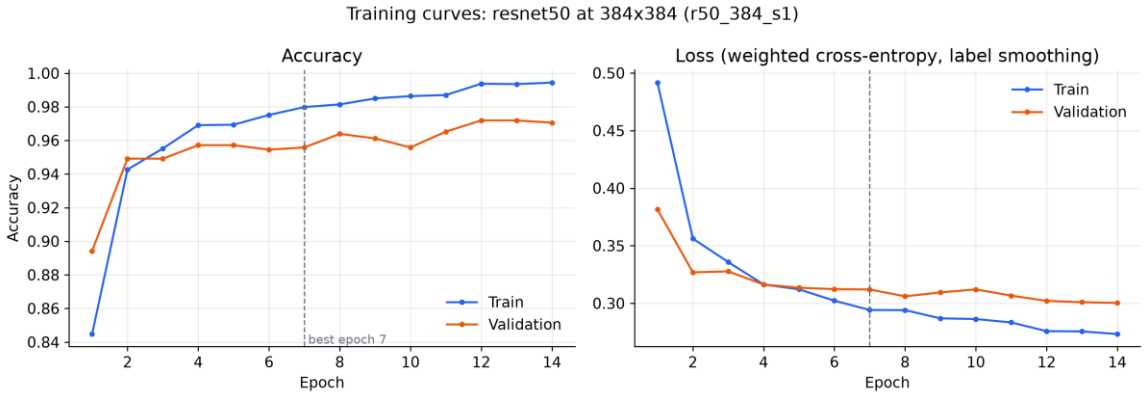


Figure 7.1: Training curves of ResNet-50 at 384 px (r50_384_s1)

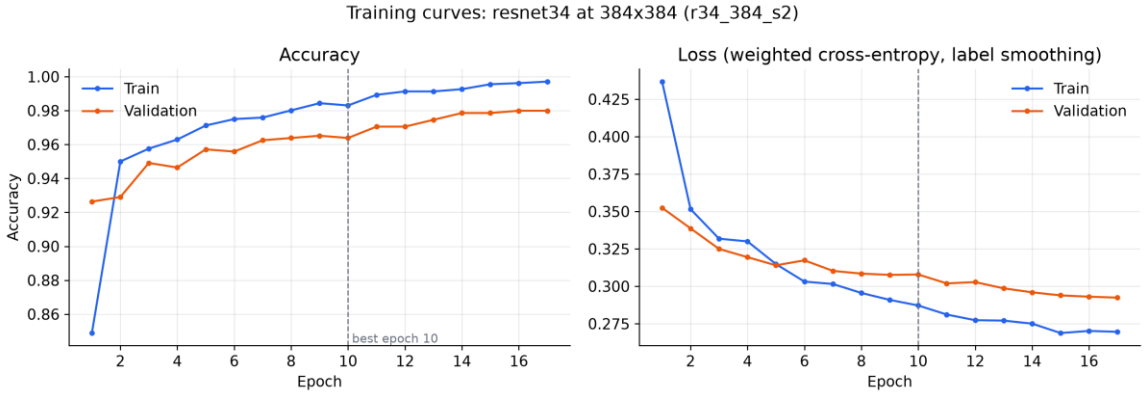


Figure 7.2: Training curves of ResNet-34 at 384 px (r34_384_s2)

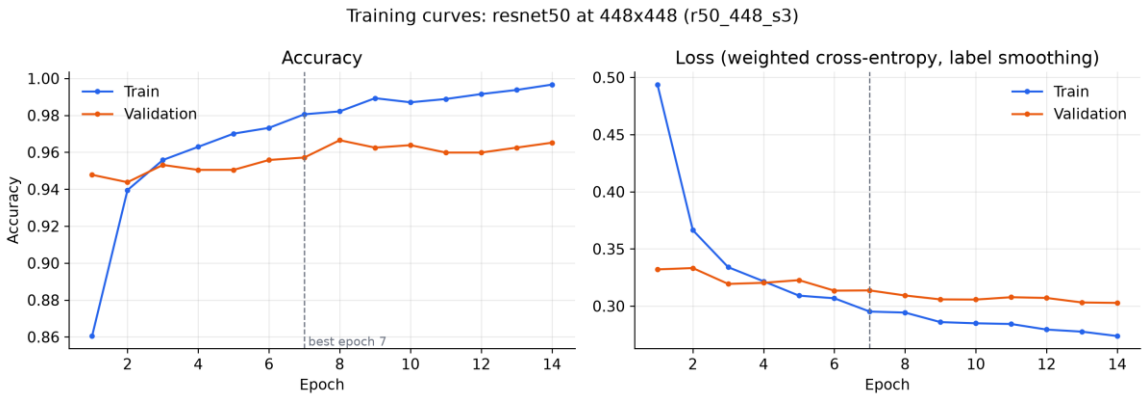


Figure 7.3: Training curves of ResNet-50 at 448 px (r50_448_s3)

7.3 Validation Results

Table 7.2 gives the validation results with flip TTA and each model's own temperature and threshold. The ensemble has the highest validation ROC-AUC (0.9980) and ties for the best balanced accuracy, which confirms the pre-committed choice of the ensemble as the final model.

Table 7.2: Validation results (747 images, flip TTA, settings fitted on validation)

Model	T	Threshold	Accuracy	Bal. acc.	Sensitivity	Specificity	ROC-AUC
ResNet-50 @ 384	0.609	0.038	98.39%	98.58%	98.20%	98.96%	0.9974
ResNet-34 @ 384	0.612	0.067	98.13%	98.57%	97.66%	99.48%	0.9977
ResNet-50 @ 448	0.618	0.044	98.13%	97.89%	98.38%	97.40%	0.9972
Ensemble (final)	**0.602**	**0.074**	**98.39%**	**98.58%**	**98.20%**	**98.96%**	**0.9980**

7.4 Official Test Results

Table 7.3 reports the single evaluation on the official test set. The final ensemble reached an **accuracy of 94.55% (590 of 624 images)**, a **sensitivity of 99.74%** (389 of 390 pneumonia X-rays detected), a specificity of 85.90%, a precision of 92.18%, an F1-score of 95.81%, a ROC-AUC of 0.9867 and a PR-AUC of 0.9898. By pneumonia type, the sensitivity was 99.6% for bacterial pneumonia (241 of 242) and 100% for viral pneumonia (148 of 148).

Table 7.3: Official test-set results (624 images; thresholds from validation)

Model	Accuracy	Precision	Sensitivity	Specificity	F1	ROC-AUC	TN / FP / FN / TP
ResNet-50 @ 384	91.99%	88.81%	99.74%	79.06%	93.96%	0.9811	185 / 49 / 1 / 389
ResNet-34 @ 384	94.87%	92.82%	99.49%	87.18%	96.04%	0.9884	204 / 30 / 2 / 388
ResNet-50 @ 448	91.67%	88.41%	99.74%	78.21%	93.73%	0.9828	183 / 51 / 1 / 389
Ensemble (final)	**94.55%**	**92.18%**	**99.74%**	**85.90%**	**95.81%**	**0.9867**	**201 / 33 / 1 / 389**

The ensemble improved clearly on the two ResNet-50 members (91.99% and 91.67%). The ResNet-34 member alone scored slightly higher on the test set (94.87%), but validation did not prefer it (balanced accuracy 98.57% versus 98.58%, AUC 0.9977 versus 0.9980). Switching to it after seeing the test results would be test-set tuning, so the pre-committed ensemble remains the reported model.

7.5 Confusion Matrix

Figure 7.4 shows the confusion matrix of the final ensemble. Of the 34 errors, **33 are false positives** (normal X-rays labelled PNEUMONIA) and only **one is a false negative** (a missed pneumonia).

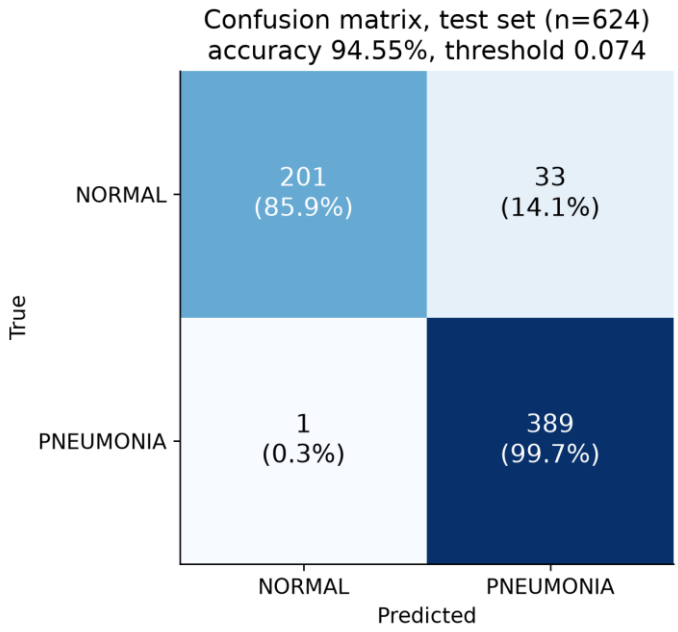


Figure 7.4: Confusion matrix of the final ensemble on the test set

7.6 ROC and Precision–Recall Curves

The ROC curves (Figure 7.5) of all models rise steeply towards the top-left corner, with AUC values between 0.981 and 0.988, and the precision–recall curves (Figure 7.6) stay high across almost the whole recall range. The marked operating point of the ensemble sits at very high sensitivity, which reflects the low validation-chosen threshold.

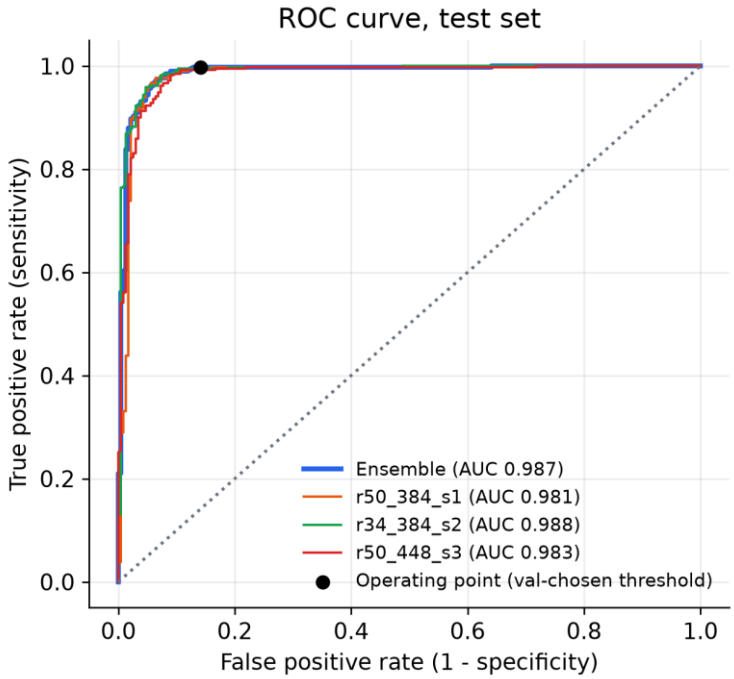


Figure 7.5: ROC curves on the test set (ensemble and members)

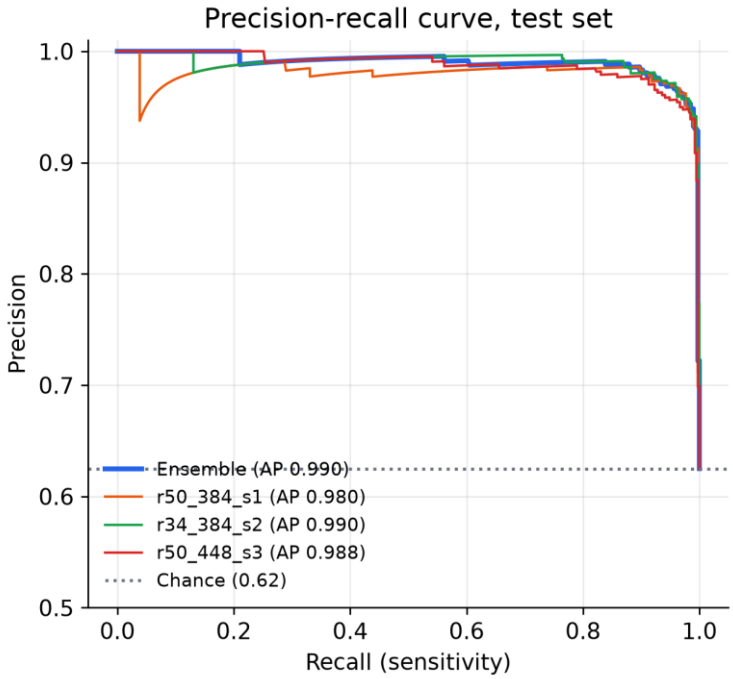


Figure 7.6: Precision–recall curves on the test set

7.7 Distribution of Predicted Probabilities

Figure 7.7 shows the calibrated probabilities of the ensemble for each true class. Almost all pneumonia X-rays receive very high probabilities, while normal X-rays are concentrated near zero with a tail that crosses the low threshold of 0.074. This tail produces the false positives.

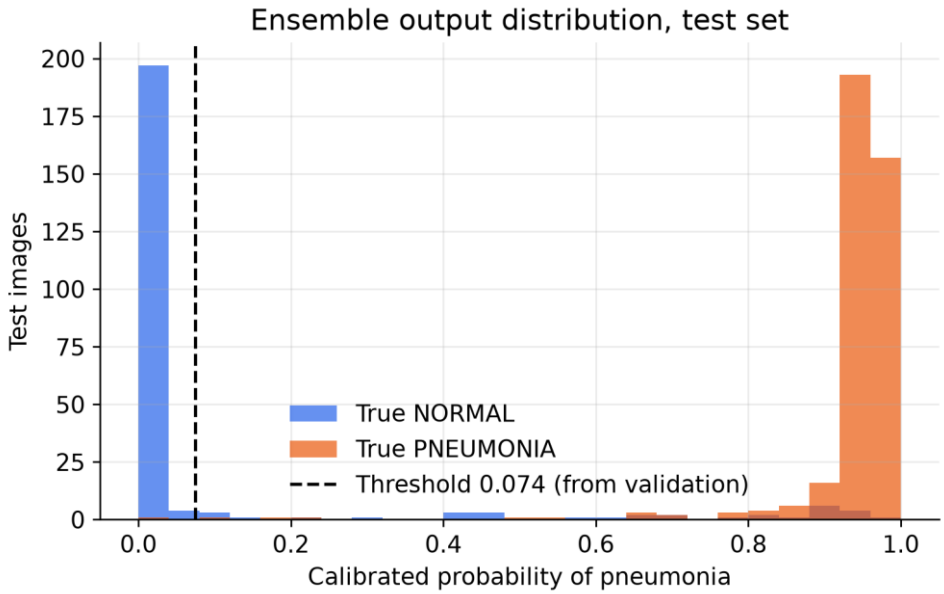


Figure 7.7: Ensemble probabilities on the test set by true class, with the decision threshold

7.8 Grad-CAM Explanations

Figure 7.8 shows Grad-CAM maps for two correctly detected pneumonia cases, two correctly classified normal cases, one false positive and the single false negative. For the correct predictions the heat maps mostly cover the lung fields: dense regions in the pneumonia cases, and broad areas of both lungs for the normal cases. However, some maps also highlight **burned-in text**, especially the "R" side marker in the upper corner and exposure values or time stamps. This indicates partial shortcut learning of hospital-specific image features, the same risk described by Zech et al. [13]. It is a likely contributor to errors on images acquired in a slightly different way and is listed as a limitation.

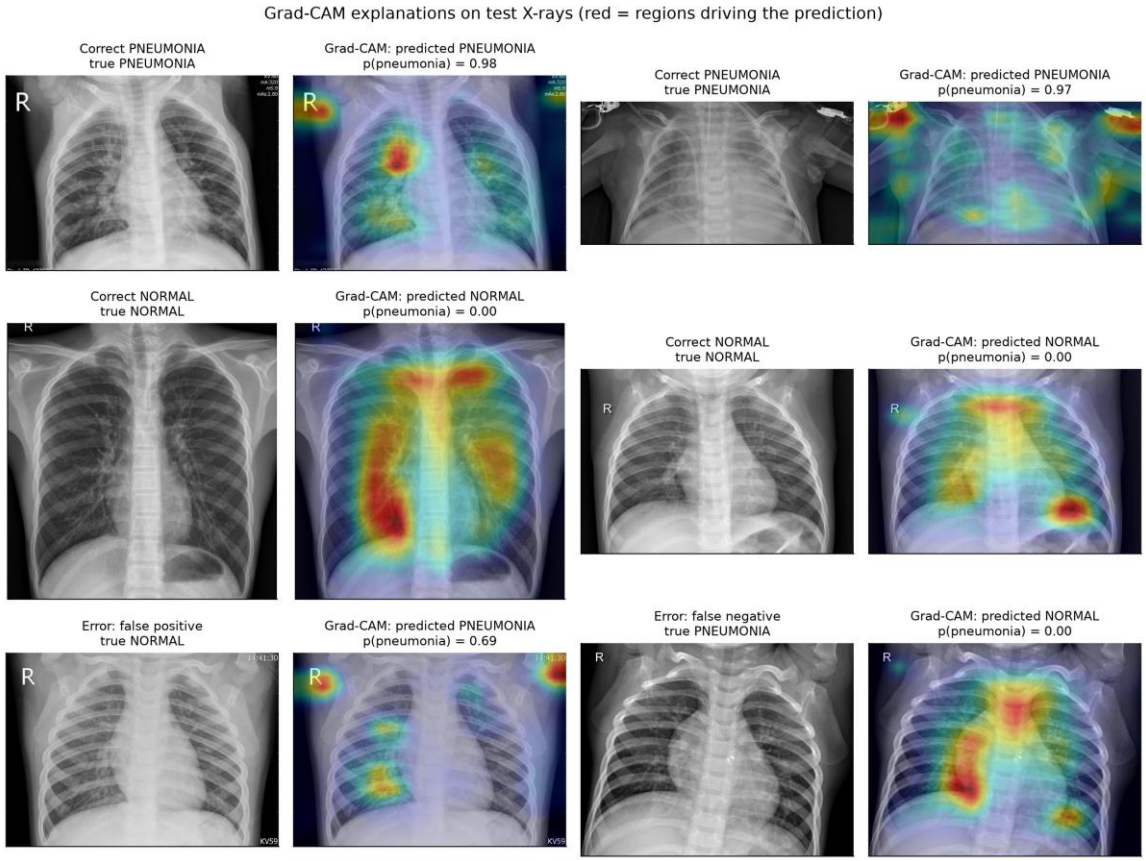


Figure 7.8: Grad-CAM explanations on test X-rays (red = regions driving the prediction)

7.9 X-ray Gate Results

The gate was validated end-to-end with check_image() on real files (Table 7.4). It accepted 620 of the 624 test chest X-rays; the four refused images are all normal X-rays that the classifier would have labelled correctly. It rejected every one of the 33 non-X-ray images (photographs, textures, documents, noise, plain colours and a gradient) and also

all 16 grayscale copies of the colour images, which only the feature-distance test can catch.

Table 7.4: Validation of the X-ray gate

Check	Result
Official test chest X-rays accepted	620 / 624 (99.36%)
Non-X-ray images rejected (22 scikit-image photographs and textures, gray and RGB noise, 5 plain colours, a gradient, 3 synthetic documents)	33 / 33
Grayscale copies of the 16 colour images rejected (highest score 0.165; cut-off 0.5)	16 / 16
Mahalanobis cut-off τ (1.15 × 99.5th percentile of validation distances)	66.04 (validation median 41.0, maximum 64.7)

The test-set metrics in Table 7.3 include all 624 images, including the four that the gate would refuse. The gate learns what the training X-rays look like, so other radiographs (for example a hand X-ray) or adult chest films may or may not be accepted; it is a convenience, not a safety guarantee.

7.10 Inference Speed

Table 7.5: Time per image (median, after warm-up, measured on official test images)

Setting	Time per image
GPU (RTX 3050): 3 ResNets × 2 views + Grad-CAM, including pre-processing and PNG encoding	376 ms (95th percentile 457 ms)
GPU without Grad-CAM	233 ms
CPU only (Intel i5-12450HX), with Grad-CAM	1,248 ms

The first prediction after start-up takes about one second longer because of CUDA warm-up; the application performs a warm-up prediction while loading so that the first real scan is fast. Even without a GPU, an X-ray is analysed in little more than a second, which is acceptable for a decision-support tool.

7.11 Comparison with Published Work

Table 7.6 compares PneumoScan AI with published results that use the **same official test set** (Chapter 2). Results on re-split data or cross-validation (97–99%) are excluded because they are not comparable.

Table 7.6: Comparison with published results on the official Kermany test set

Work	Model	Accuracy	Sensitivity	Specificity	Other
Kermany et al. [2]	Inception-V3, transfer learning	92.8%	93.2%	90.1%	AUC 0.968
Yang et al. [6]	ResNet-18, 28 px	85.4%	–	–	AUC 0.944
Yang et al. [6]	ResNet-18, 224 px	86.4%	–	–	AUC 0.956
Yang et al. [6]	ResNet-50, 224 px	88.4%	–	–	AUC 0.962
Liang & Zheng [11]	Residual + dilated convolution	–	96.7%	–	F1 92.7%
Chouhan et al. [8]	Ensemble of 5 CNNs (voting)	96.4%	99.62%	–	–
This work	**Ensemble of 3 ResNets**	**94.55%**	**99.74%**	**85.90%**	**AUC 0.9867, F1 95.81%**

Sensitivity = recall for the PNEUMONIA class. "–" means not reported in the cited source. Chouhan et al. report results on "unseen data from the Guangzhou dataset" after training on the 5,232 training images.

PneumoScan AI is well above the published ResNet baselines on the same images (85.4–88.4%) and the dataset authors' Inception-V3 (92.8%), and its sensitivity of 99.74% is the highest in the table. Only the five-CNN ensemble of Chouhan et al. reports a higher accuracy.

7.12 Error Analysis

- **False negative (1 image):** person154_bacteria_728.jpeg, a bacterial pneumonia case, received a very low probability (p = 0.0008). Its Grad-CAM map for the NORMAL decision (Figure 7.8, bottom right) highlights broad areas of the central chest and lungs, the pattern the model associates with normal X-rays, rather than a focal opacity.
- **False positives (33 images):** normal X-rays whose probability exceeded the low threshold of 0.074. Several of their heat maps highlight corner markers and text as well as the lungs. In the application, 12 of these 33 images fall in the Moderate risk band (possible pneumonia, see a doctor) and 21 in the High band, so the risk level alone does not remove them; the doctor's review remains essential.
- **Why validation looked better than test:** on validation the normal images were very clean (only 2 false positives out of 192), while a tail of pneumonia images had low scores. Maximising Youden's J therefore produced a low threshold. The official test set is known to be harder and differently distributed from the training pool (published ResNet baselines reach only 85–88% on it [6]), and on the test set the low threshold favours sensitivity (99.7%) over specificity (85.9%).

- **Ranking is strong:** a test ROC-AUC of 0.987 shows that the model ranks pneumonia above normal images very well; the remaining gap is about where the threshold sits, not about poor features.

7.13 Discussion of the 95% Target

The objective was at least 95% accuracy on the official test set. The honest result is **94.55%, which is three images short** of the target (95% of 624 is 593 correct images; the ensemble got 590). The result is nevertheless strong and clinically sensible:

- Only **1 of 390** pneumonia X-rays was missed (sensitivity 99.74%). For a screening and triage aid, missing pneumonia is the more dangerous error, and the validation-chosen threshold prioritises not missing it.
- Almost all errors are normal X-rays flagged for review, which a doctor can quickly dismiss. This matches the known behaviour of this test set in published work: Kermany et al. report lower specificity than sensitivity [2], the ResNet baselines of MedMNIST v2 have high AUC (0.944–0.962) but much lower accuracy (85.4–88.4%) [6], and the best published ensemble combines 96.4% accuracy with 99.62% recall, so its errors are also mostly false alarms [8].
- The evaluation is honest: the test set was scored once, with a threshold that was fixed on validation before the test set was touched.

For discussion only, a post-hoc analysis in the ML report shows how the same ensemble would have scored with other thresholds (Table 7.7). With a threshold of 0.5 the test accuracy would have been 95.8%. **This threshold was not adopted:** choosing it after seeing the test results would be tuning on the test set, which would make the reported number optimistic and untrustworthy. The analysis is shown only to explain that the gap comes from where the threshold sits under the shift between the validation and test distributions. A future validation set that resembles the deployment data (for example from a different hospital) would allow a better threshold to be chosen honestly.

Table 7.7: Post-hoc threshold sensitivity on the test set (discussion only; NOT the reported result)

Threshold	Test accuracy	Status
0.074 (chosen on validation)	**94.55%**	**Official, reported result**
0.3	94.9%	Post-hoc, not used
0.5	95.8% (sensitivity 98.7%, specificity 91.0%)	Post-hoc, not used
0.9	94.1%	Post-hoc, not used

CHAPTER 8
APPLICATION AND SCREENSHOTS

8.1 Overview

The trained ensemble is delivered through an offline web application. A double-click on start.bat starts the FastAPI server on the local computer and opens the browser; the model is loaded once in the background and the status pill in the top bar changes to **Model ready**. All screenshots in this chapter were taken from the running application with the real trained model (not the mock model used during development), in a browser window 1280 pixels wide; tall pages were captured in full and are shown from the top. The patient details in the screenshots are fictitious demo data. The navigation bar gives access to seven pages: Scan, Triage, History, CURB-65, Learn, Model and About. The sample X-rays used in the screenshots come from the held-out test split.

8.2 Scan Page

Figure 8.1 shows the home page. The user drops a chest X-ray on the "lightbox", browses for a file or pastes an image (JPG, PNG, BMP, WEBP, TIFF or DICOM, up to 25 MB). Patient details are optional and are printed on the PDF report. A gallery of sample X-rays from the test set, with their true diagnosis, lets a teacher or student try the model immediately.

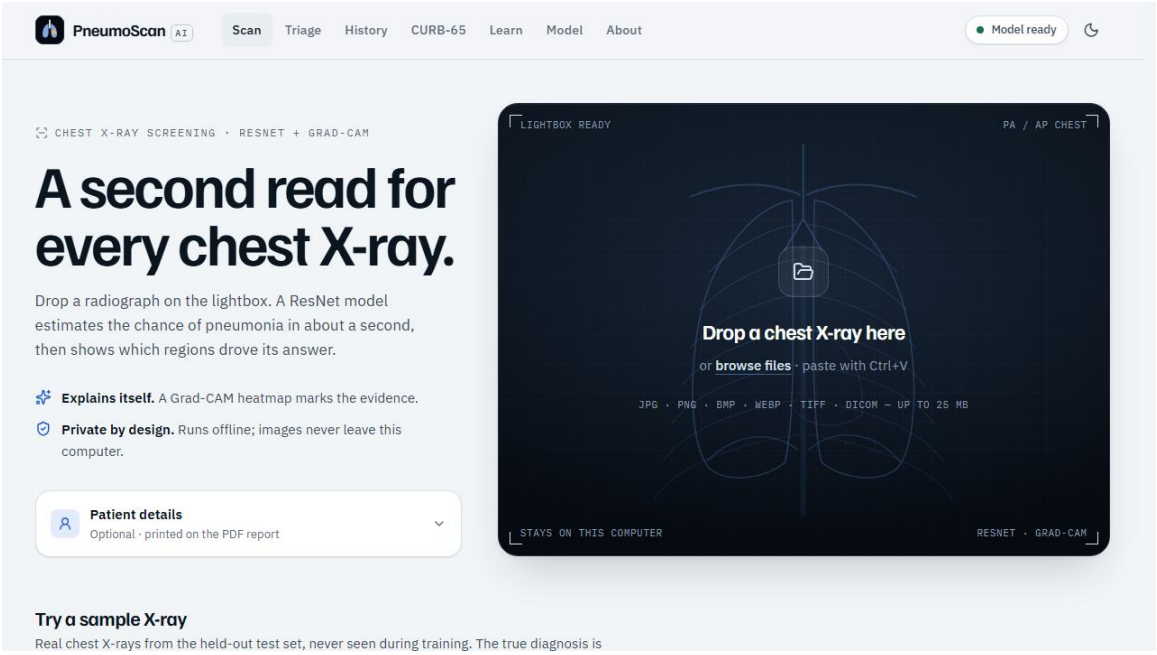


Figure 8.1: Scan page with the upload lightbox and optional patient details

8.3 Result with Grad-CAM Heat Map

The result screen (Figure 8.2) shows the verdict ("Pneumonia likely" or "Looks normal"), the calibrated pneumonia probability on a 0–100 ruler with the decision threshold marked, the confidence, the risk level, the decision threshold and the inference time. The X-ray viewer shows the Grad-CAM heat map over the X-ray with an opacity slider, zoom and pan.



Figure 8.2: Result screen for a pneumonia X-ray with the Grad-CAM heat map

Below the verdict the page gives a plain-English explanation, recommended next steps and the responsible-AI disclaimer (Figure 8.3).

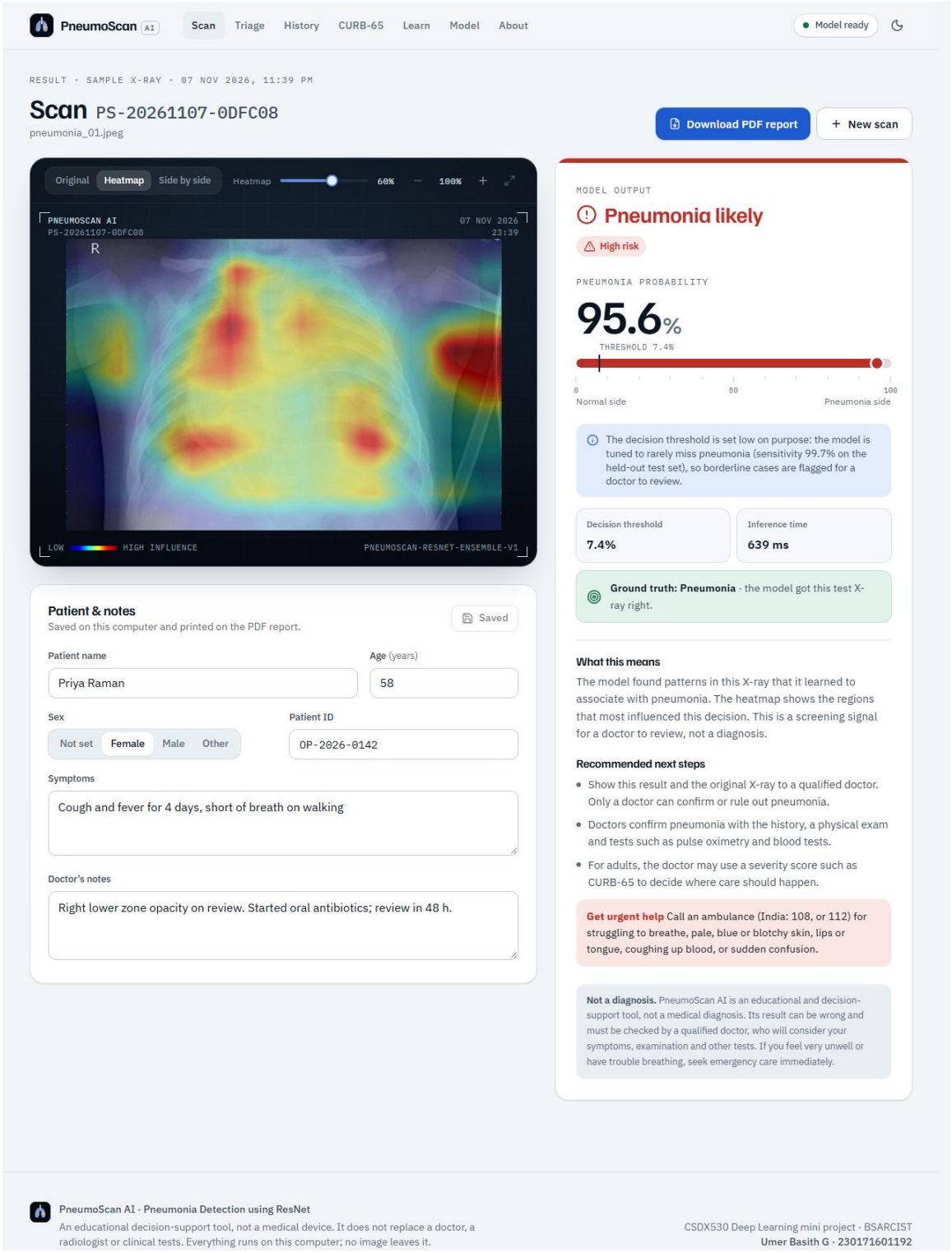


Figure 8.3: Full result page with explanation, next steps and disclaimer (top part of the page)

The viewer can also show the original X-ray and the heat map side by side (Figure 8.4), so a doctor can compare the highlighted regions with the underlying image.



Figure 8.4: Side-by-side view of the X-ray and the Grad-CAM heat map

8.4 Patient PDF Report

A one-click PDF report (Figure 8.5) contains the patient details, the result with probability and risk level, the X-ray with its heat map, the explanation, next steps and the disclaimer, and the model version and date. It is generated on the server with ReportLab using a bundled font, so it works offline.

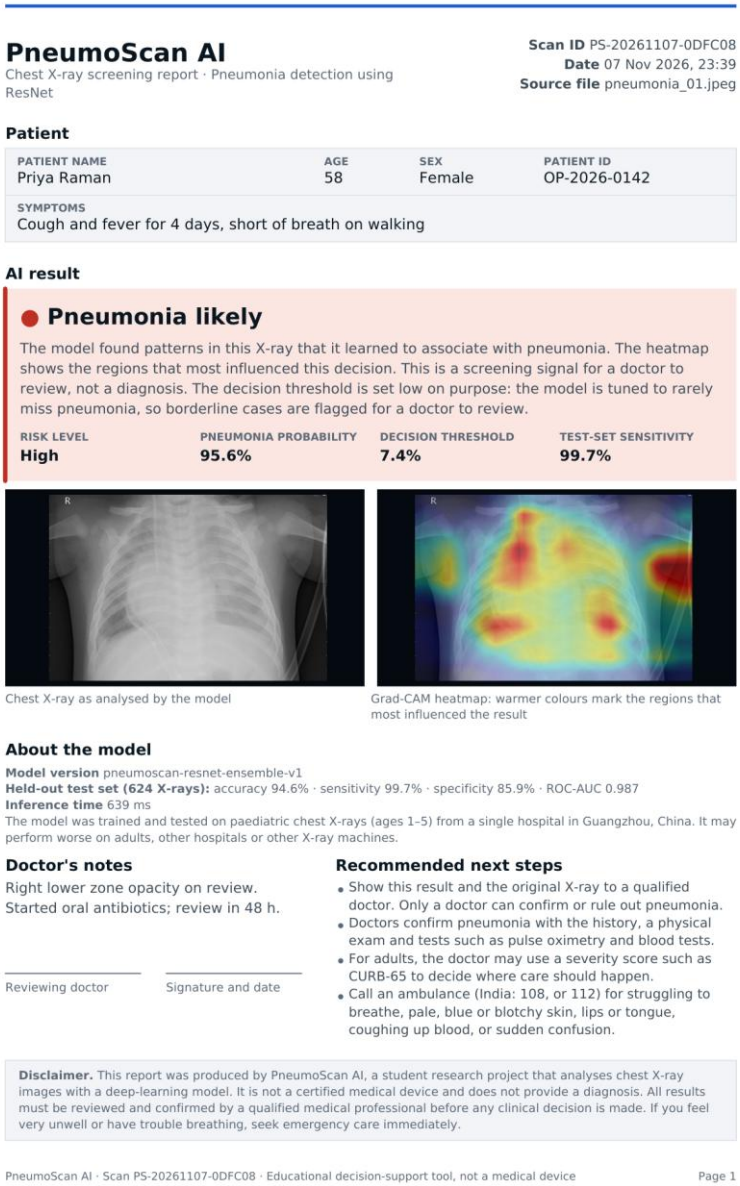


Figure 8.5: First page of a patient PDF report generated by the application

8.5 Rejection of Non-X-ray Images

If the uploaded image is not a chest X-ray, the X-ray gate turns it away with a friendly reason and a chest X-ray likeness score; no prediction is made and nothing is saved (Figure 8.6).

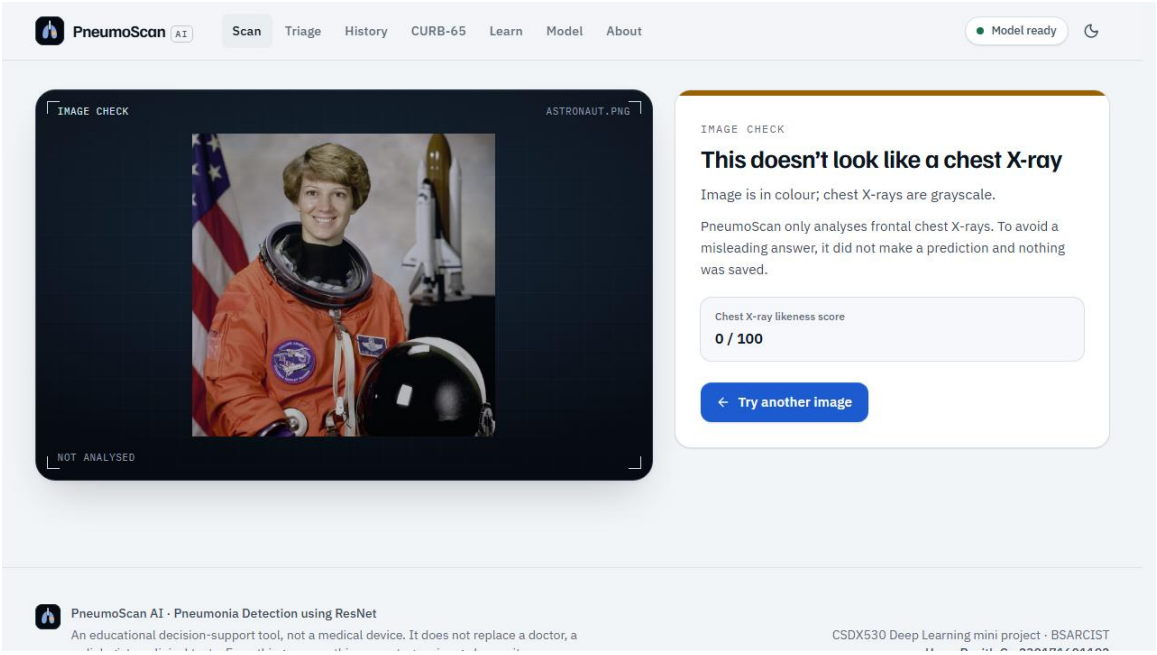


Figure 8.6: A colour photograph rejected by the X-ray gate

8.6 Triage Queue (Doctor Mode)

The Triage page accepts up to 50 X-rays at once and shows the progress of each file. As results arrive, the queue re-orders itself so that the most likely pneumonia cases rise to the top, high-risk results are flagged as urgent, images that are not X-rays are listed as issues, and the queue can be filtered and exported as CSV (Figure 8.7).

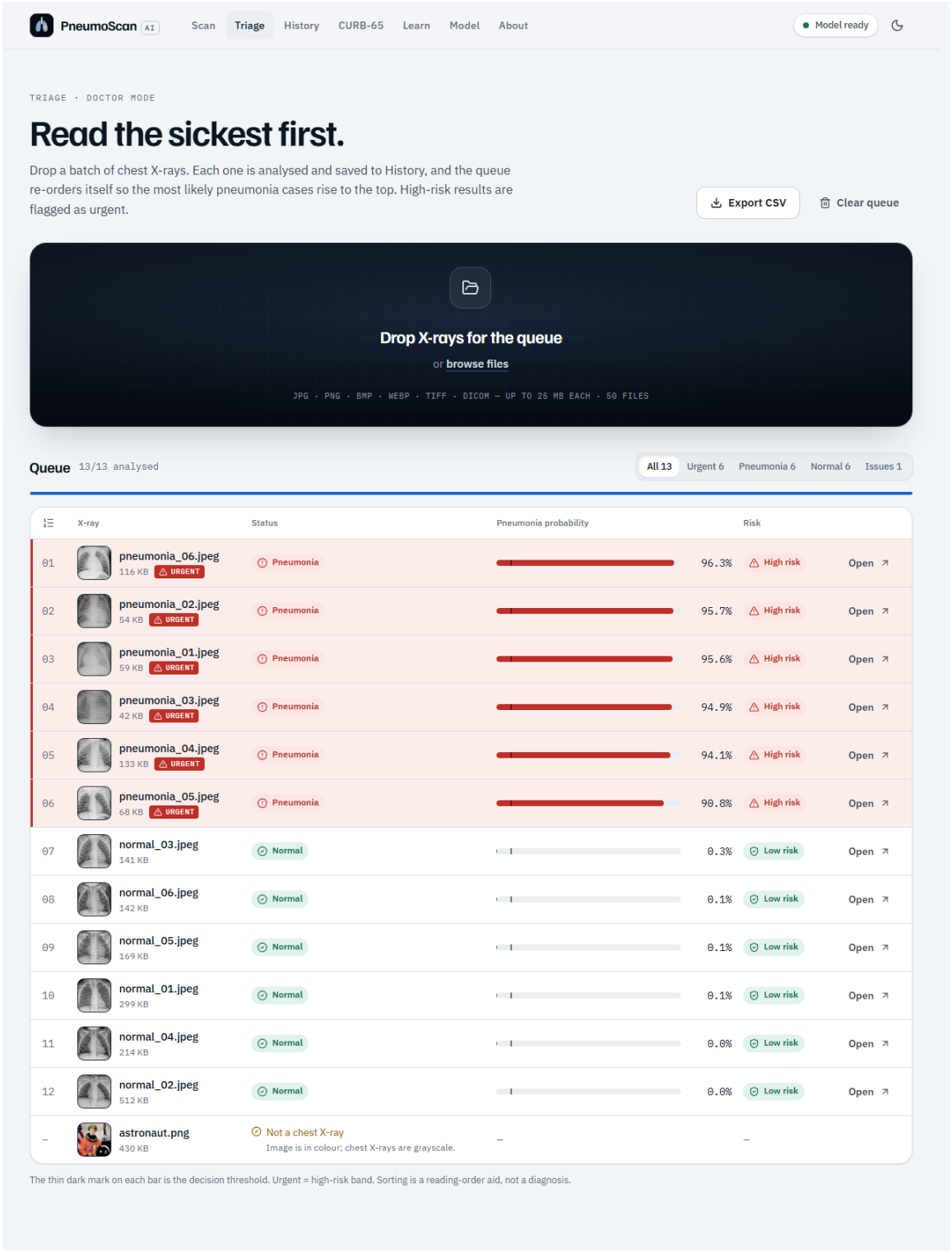


Figure 8.7: Completed triage queue sorted by pneumonia probability

8.7 Scan History

Every analysed scan is stored locally in SQLite. The History page can be shown as a grid of thumbnails (Figure 8.8) or as a list; it supports search and filters, and each scan can be opened, edited (patient details and doctor's notes), re-downloaded as PDF or deleted.

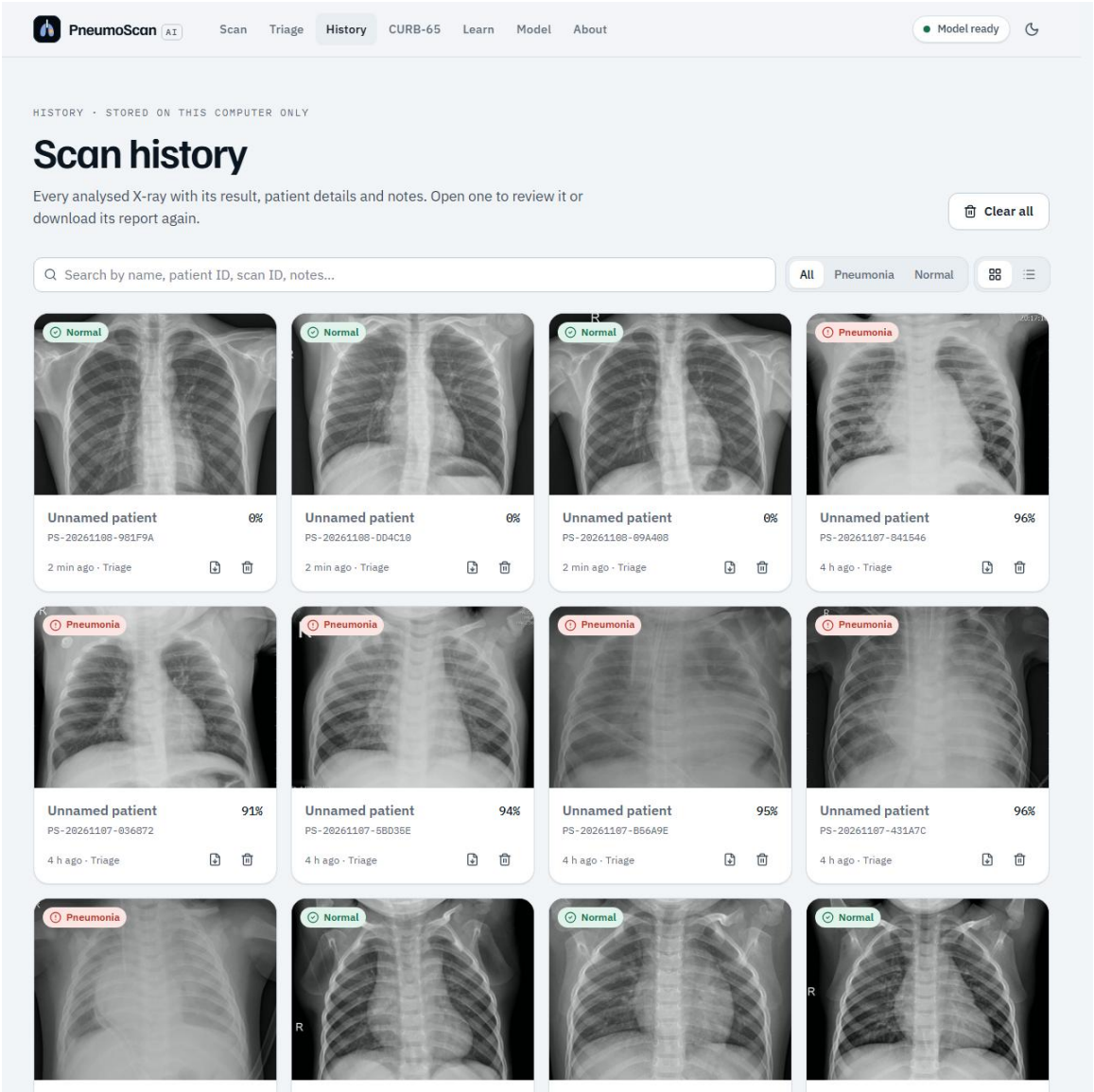


Figure 8.8: Scan history in grid view (top part of the page)

8.8 CURB-65 Severity Calculator

For adults with pneumonia, the CURB-65 page (Figure 8.9) computes the severity score from Lim et al. [5]. Urea can be entered in mmol/L or as BUN in mg/dL; if urea is not available the page switches to CRB-65. The live score, risk group, 30-day mortality and recommended care setting are shown (Table 8.1). The page states that the score is for adults; for children the WHO danger signs on the Learn page apply.

Table 8.1: CURB-65 risk groups used by the calculator (derivation cohort of Lim et al. [5])

Score	Risk group	30-day mortality	Suggested care
0–1	Low	1.5%	Likely suitable for home treatment
2	Intermediate	9.2%	Consider hospital-supervised treatment
3–5	High	22%	Manage in hospital as severe pneumonia; assess for ICU, especially at 4–5

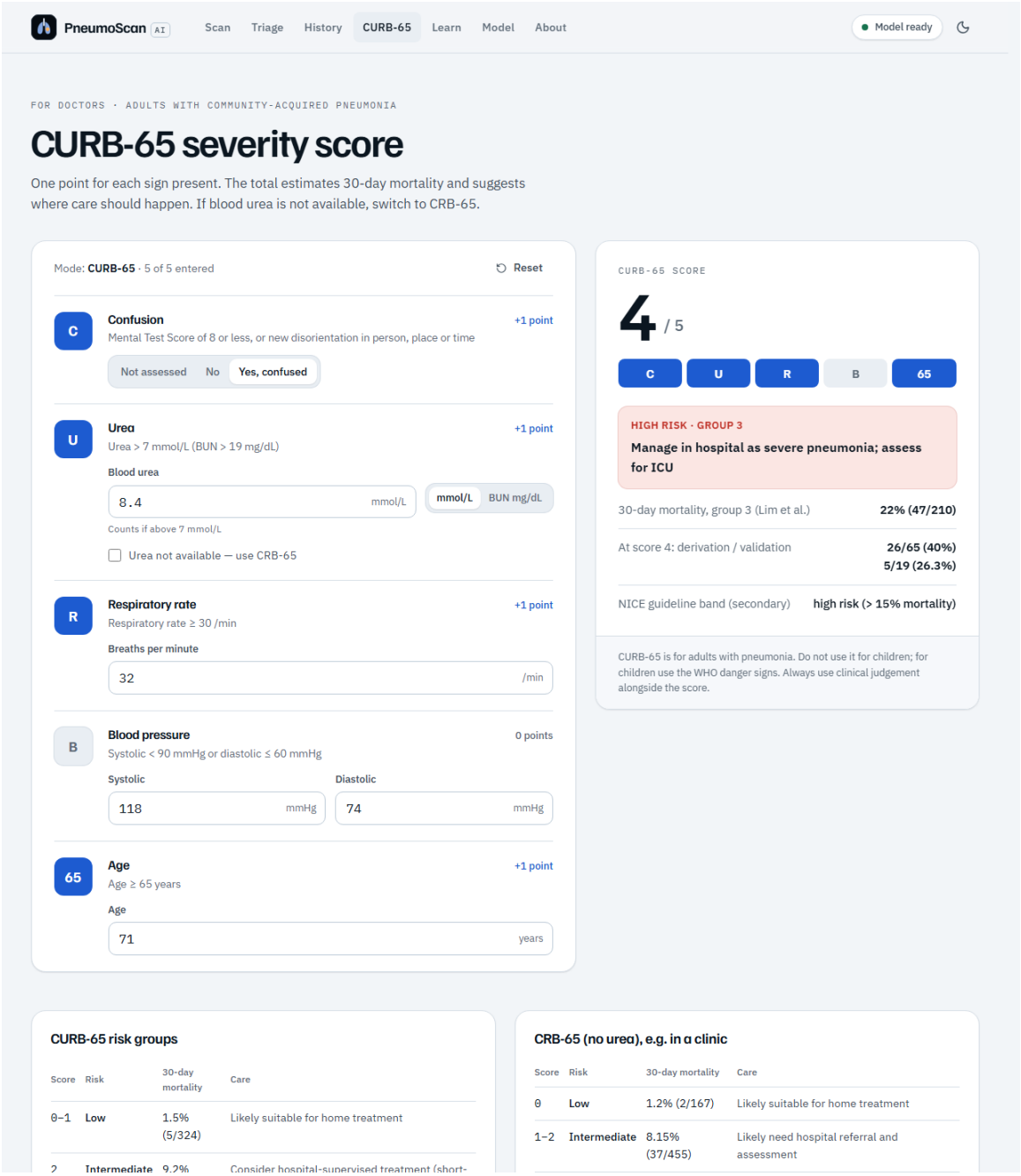


Figure 8.9: CURB-65 severity calculator (top part of the page)

8.9 Learn Page

The Learn page (Figure 8.10) explains pneumonia in simple English for patients and families: emergency warning signs, symptoms in adults, danger signs in children (WHO), causes, who is at risk, prevention, how doctors diagnose pneumonia, care at home and how pneumonia looks on an X-ray. Every card lists its sources (WHO, NHS, CDC, NIH, Radiopaedia).

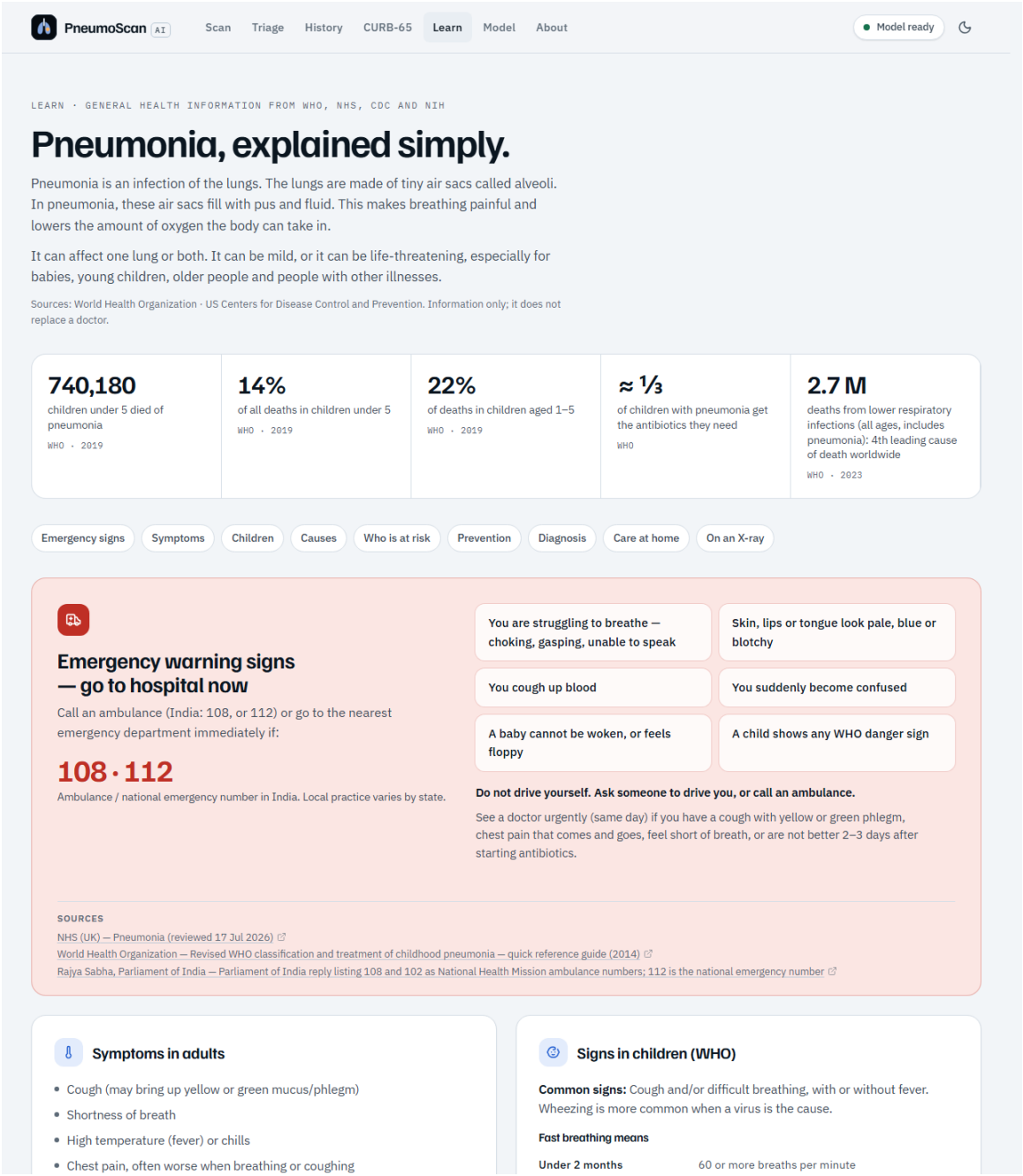


Figure 8.10: Learn page with sourced health information (top part of the page)

8.10 Model Page

The Model page (Figure 8.11) is designed for the classroom: it shows the official test accuracy, sensitivity, specificity and ROC-AUC, the confusion matrix, the report figures, how ResNet's skip connection works, the ensemble members, dataset facts, intended use and limitations.

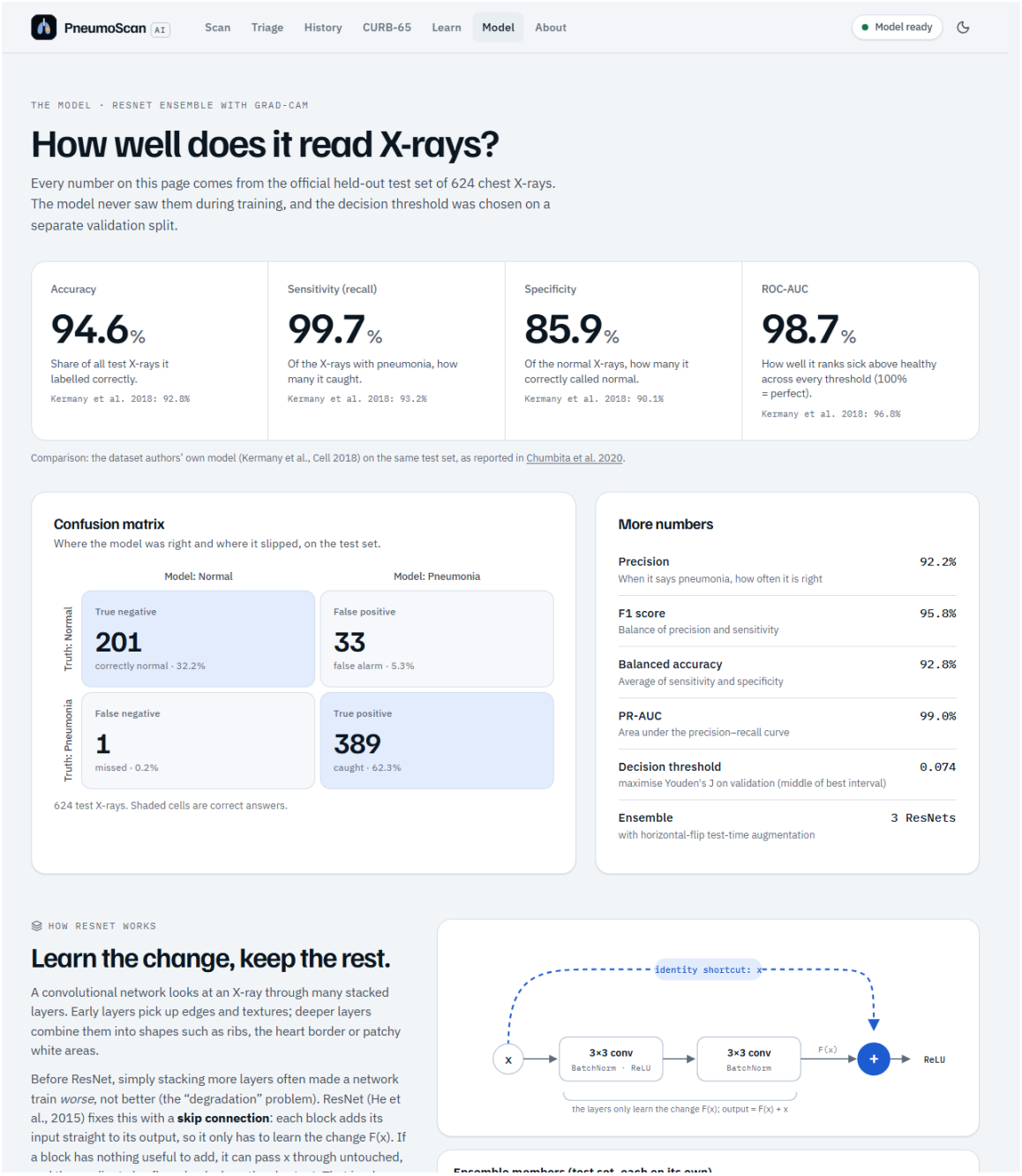


Figure 8.11: Model page with test-set performance (top part of the page)

8.11 About Page

The About page (Figure 8.12) gives the project, student, course and faculty details, the technology stack, the dataset credit and the disclaimer.

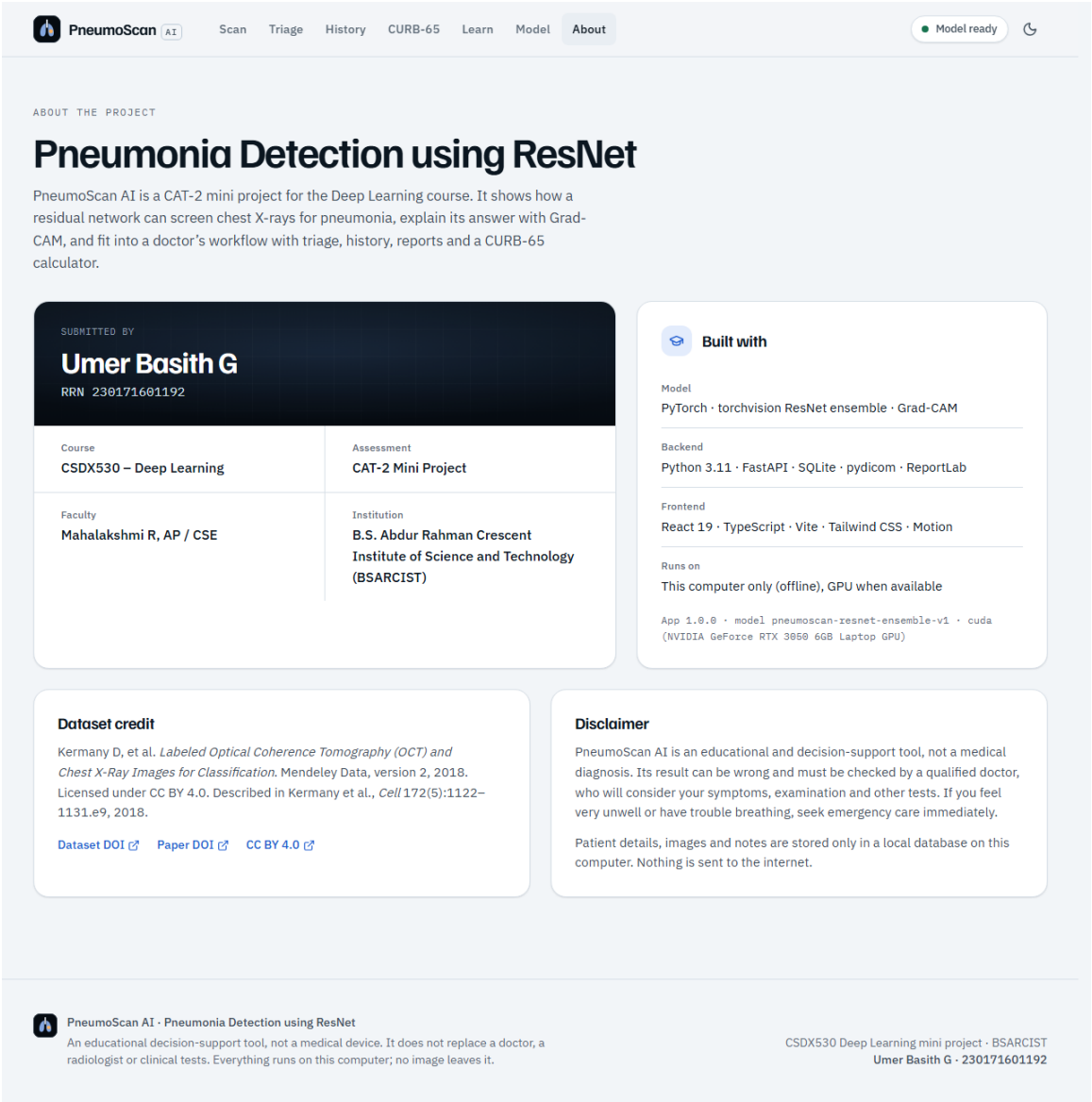


Figure 8.12: About page

8.12 Dark Mode

Light and dark themes follow the system setting and can be switched with the moon/sun button; the choice is remembered. Figure 8.13 shows the result screen in dark mode, which is comfortable on a projector in a darkened classroom.

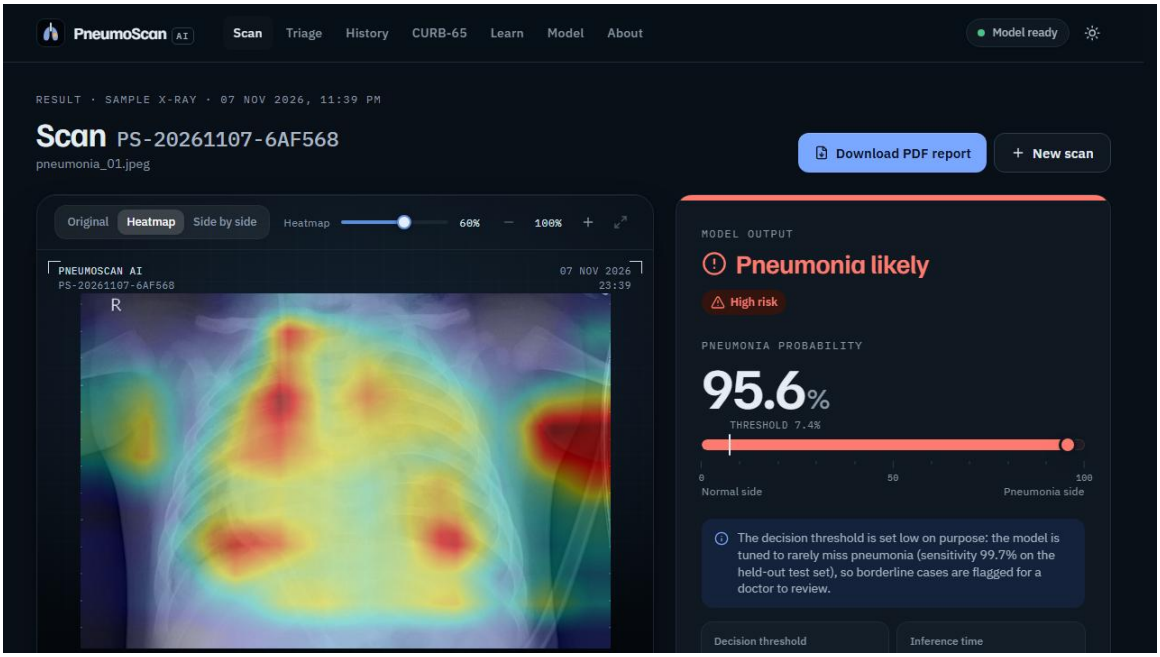


Figure 8.13: Result screen in dark mode

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

This project developed **PneumoScan AI**, a complete and explainable system for detecting pneumonia in paediatric chest X-rays using deep residual networks. ImageNet-pretrained ResNet-34 and ResNet-50 models were fine-tuned with transfer learning, class-weighted and label-smoothed cross-entropy, realistic augmentation, mixed precision and weight averaging, and combined into a three-member ensemble with test-time augmentation and temperature scaling. Every choice was made on a patient-level validation split, and the official 624-image test set was evaluated only once.

The final ensemble achieved an accuracy of **94.55%**, a sensitivity of **99.74%** (one missed pneumonia out of 390), a specificity of **85.90%**, a precision of **92.18%**, an F1-score of **95.81%** and a ROC-AUC of **0.9867**. These results are well above the published ResNet baselines on the same test set and above the dataset authors' own result, although the 95% accuracy target was missed by three images. The errors are almost entirely normal X-rays flagged for review, which is the safer kind of error for a screening aid. Grad-CAM heat maps make each decision transparent, and an out-of-distribution gate rejected all non-X-ray check images. The model is packaged in an offline web application with single and batch scanning, patient PDF reports, scan history, a CURB-65 calculator and sourced patient education, which makes it useful for teaching and as a second-reader aid. The project shows that a carefully and honestly evaluated ResNet system can deliver reliable, explainable support for pneumonia screening on ordinary hardware.

9.2 Limitations

- The model was trained and tested on paediatric X-rays (ages 1–5) from a single hospital; it may perform worse on adults, other hospitals or other X-ray machines. Pneumonia-screening CNNs have been shown to perform worse on data from external hospitals (AUC 0.931 internal versus 0.815 external) [13].
- It detects only two classes and cannot identify the cause of pneumonia (bacterial or viral) or other lung diseases.
- Specificity on the official test set is 85.9%: false alarms on normal X-rays are the main type of error.

- Grad-CAM heat maps are coarse (computed on 12 × 12 or 14 × 14 feature maps); they show where the model looked, not proof of disease [4]. Some maps also highlight burned-in text such as the "R" side marker, exposure values or time stamps, a sign of partial shortcut learning from hospital-specific image features.
- The validation set comes from the same distribution as the training data, so it overestimated test performance and produced a threshold that is lower than ideal for the test set.
- The X-ray gate recognises images that resemble the training X-rays; other radiographs or adult chest films may or may not be accepted.
- A chest X-ray alone is not enough for diagnosis; symptoms, examination and other tests are needed. Dataset labels come from clinical grading by physicians and may contain some errors.
- The system is a student project and is not a certified medical device.

9.3 Future Enhancements

- Train and validate on larger, multi-centre and adult datasets, including data from Indian hospitals, and choose the threshold on a validation set that resembles the deployment data.
- Mask or crop burned-in text and markers, or add lung segmentation, so the model focuses only on the lung fields and shortcut learning is reduced.
- Extend to multi-class classification (bacterial versus viral pneumonia and other chest findings).
- Evaluate the system prospectively alongside radiologists.
- Integrate with hospital PACS and national teleconsultation services.
- Build a lightweight mobile version for primary health centres.
- Support regional Indian languages in the patient interface.

9.4 Social Impact

PneumoScan AI shows how artificial intelligence can support, not replace, doctors where radiologists are few. A quick, explainable second opinion on a chest X-ray could help health workers decide which patients need urgent referral and help busy hospitals prioritise their reporting queues. Because the system runs offline on a normal computer, it can be used where internet access is unreliable, and patient data never leave the computer.

The Learn page and the patient report give families clear, trustworthy information on danger signs and prevention, including immunisation against Hib, pneumococcus, measles and whooping cough, cleaner indoor air and exclusive breastfeeding for the first six months, which the World Health Organization identifies as key ways to protect children from pneumonia [14]. By reporting results honestly and showing a clear disclaimer, the project also promotes responsible use of AI in healthcare.

REFERENCES

[1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, 2016, pp. 770–778, doi: 10.1109/CVPR.2016.90.

[2] D. S. Kermany *et al.*, "Identifying medical diagnoses and treatable diseases by image-based deep learning," *Cell*, vol. 172, no. 5, pp. 1122–1131.e9, Feb. 2018, doi: 10.1016/j.cell.2018.02.010.

[3] P. Rajpurkar *et al.*, "CheXNet: Radiologist-level pneumonia detection on chest X-rays with deep learning," arXiv:1711.05225, 2017. [Online]. Available: https://arxiv.org/abs/1711.05225

[4] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Venice, Italy, 2017, pp. 618–626, doi: 10.1109/ICCV.2017.74.

[5] W. S. Lim *et al.*, "Defining community acquired pneumonia severity on presentation to hospital: An international derivation and validation study," *Thorax*, vol. 58, no. 5, pp. 377–382, May 2003, doi: 10.1136/thorax.58.5.377.

[6] J. Yang *et al.*, "MedMNIST v2 – A large-scale lightweight benchmark for 2D and 3D biomedical image classification," *Sci. Data*, vol. 10, no. 1, Art. no. 41, Jan. 2023, doi: 10.1038/s41597-022-01721-8.

[7] O. Stephen, M. Sain, U. J. Maduh, and D.-U. Jeong, "An efficient deep learning approach to pneumonia classification in healthcare," *J. Healthc. Eng.*, vol. 2019, Art. no. 4180949, Mar. 2019, doi: 10.1155/2019/4180949.

[8] V. Chouhan *et al.*, "A novel transfer learning based approach for pneumonia detection in chest X-ray images," *Appl. Sci.*, vol. 10, no. 2, Art. no. 559, Jan. 2020, doi: 10.3390/app10020559.

[9] T. Rahman *et al.*, "Transfer learning with deep convolutional neural network (CNN) for pneumonia detection using chest X-ray," *Appl. Sci.*, vol. 10, no. 9, Art. no. 3233, May 2020, doi: 10.3390/app10093233.

[10] M. F. Hashmi, S. Katiyar, A. G. Keskar, N. D. Bokde, and Z. W. Geem, "Efficient pneumonia detection in chest X-ray images using deep transfer learning," *Diagnostics*, vol. 10, no. 6, Art. no. 417, Jun. 2020, doi: 10.3390/diagnostics10060417.

[11] G. Liang and L. Zheng, "A transfer learning method with deep residual network for pediatric pneumonia diagnosis," *Comput. Methods Programs Biomed.*, vol. 187, Art. no. 104964, Apr. 2020, doi: 10.1016/j.cmpb.2019.06.023.

[12] R. Kundu, R. Das, Z. W. Geem, G.-T. Han, and R. Sarkar, "Pneumonia detection in chest X-ray images using an ensemble of deep learning models," *PLoS ONE*, vol. 16, no. 9, Art. no. e0256630, Sep. 2021, doi: 10.1371/journal.pone.0256630.

[13] J. R. Zech, M. A. Badgeley, M. Liu, A. B. Costa, J. J. Titano, and E. K. Oermann, "Variable generalization performance of a deep learning model to detect pneumonia in

chest radiographs: A cross-sectional study," *PLoS Med.*, vol. 15, no. 11, Art. no. e1002683, Nov. 2018, doi: 10.1371/journal.pmed.1002683.

[14] World Health Organization, "Pneumonia," Fact sheet, Nov. 11, 2022. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/pneumonia

[15] World Health Organization, "The top 10 causes of death," Fact sheet, Oct. 2, 2026. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/the-top-10-causes-of-death

[16] A. Chandramohan *et al.*, "Teleradiology and technology innovations in radiology: status in India and its role in increasing access to primary health care," *Lancet Reg. Health Southeast Asia*, vol. 23, Art. no. 100195, Apr. 2024, doi: 10.1016/j.lansea.2023.100195.

[17] D. Kermany, "Labeled optical coherence tomography (OCT) and chest X-ray images for classification," Mendeley Data, v2, 2018, doi: 10.17632/rscbjbr9sj.2.

[18] M. Chumbita *et al.*, "Can artificial intelligence improve the management of pneumonia," *J. Clin. Med.*, vol. 9, no. 1, Jan. 2020. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC7019351/

[19] Centers for Disease Control and Prevention, "Risk factors for pneumonia," Feb. 25, 2026. [Online]. Available: https://www.cdc.gov/pneumonia/risk-factors/index.html

[20] T. Cherian *et al.*, "Standardized interpretation of paediatric chest radiographs for the diagnosis of pneumonia in epidemiological studies," *Bull. World Health Organ.*, vol. 83, no. 5, pp. 353–359, 2005. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC2626240/

[21] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019, pp. 8024–8035.

[22] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[23] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2009, pp. 248–255.

[24] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the Inception architecture for computer vision," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 2818–2826.

[25] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019.

[26] P. Micikevicius *et al.*, "Mixed precision training," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2018.

[27] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. 34th Int. Conf. Mach. Learn. (ICML)*, 2017, pp. 1321–1330.

[28] W. J. Youden, "Index for rating diagnostic tests," *Cancer*, vol. 3, no. 1, pp. 32–35, 1950.

[29] K. Lee, K. Lee, H. Lee, and J. Shin, "A simple unified framework for detecting out-of-distribution samples and adversarial attacks," in *Advances in Neural Information Processing Systems 31 (NeurIPS)*, 2018.

[30] O. Ledoit and M. Wolf, "A well-conditioned estimator for large-dimensional covariance matrices," *J. Multivariate Anal.*, vol. 88, no. 2, pp. 365–411, 2004.

APPENDIX A: SOURCE CODE

This appendix lists the complete source code of the machine-learning pipeline (training, evaluation, explainability, X-ray gate and inference) and the main back-end files, exactly as used to produce the results in this report. The remaining files (figure and sample generation, tests, the other back-end modules and the React front-end) are included in the project folder submitted with this report.

A.1 ml/common.py

Shared constants, image conversion, model builder and metrics.

```
"""Shared constants and helpers used by every ML script.

Keeping image preprocessing and model construction in ONE place guarantees
that training, evaluation and the app's inference all see images the same way.
"""
from __future__ import annotations

from pathlib import Path

import numpy as np
from PIL import Image, ImageOps

ROOT = Path(__file__).resolve().parents[1]
DATA_DIR = ROOT / "data"
RAW_DIR = DATA_DIR / "raw" / "chest_xray"      # extracted Kermany et al. 2018 dataset
CACHE_DIR = DATA_DIR / "processed"             # 512x512 grayscale PNG cache
SPLIT_CSV = DATA_DIR / "splits.csv"
MODELS_DIR = ROOT / "models"
CKPT_DIR = MODELS_DIR / "checkpoints"
LOGS_DIR = ROOT / "logs"
FIG_DIR = ROOT / "docs" / "figures"

CLASSES = ["NORMAL", "PNEUMONIA"]              # index 1 = positive class
CANON_SIZE = 512                               # canonical square view (cache + inference)
IMAGENET_MEAN = (0.485, 0.456, 0.406)
IMAGENET_STD = (0.229, 0.224, 0.225)


def to_gray_uint8(img: Image.Image) -> np.ndarray:
    """Convert a PIL image of ANY mode (L, RGB, RGBA, P, I;16, I, F, CMYK...) to uint8
grayscale.

    16-bit / 32-bit images (e.g. PNG16 exported from DICOM) are min-max scaled to 0..255.
    Transparent pixels are composited onto black (X-ray backgrounds are dark).
    """
    img = ImageOps.exif_transpose(img)  # respect phone-camera orientation tags
    mode = img.mode
    if mode in ("I;16", "I;16B", "I;16L", "I;16N", "I", "F"):
        arr = np.asarray(img, dtype=np.float32)
        lo, hi = float(arr.min()), float(arr.max())
        if hi - lo < 1e-6:
            return np.zeros(arr.shape, dtype=np.uint8)
        return ((arr - lo) / (hi - lo) * 255.0).round().astype(np.uint8)
    if mode in ("RGBA", "LA", "PA") or (mode == "P" and "transparency" in img.info):
        rgba = img.convert("RGBA")
        black = Image.new("RGBA", rgba.size, (0, 0, 0, 255))
        img = Image.alpha_composite(black, rgba)
    if mode == "1":
        img = img.convert("L")
    return np.asarray(img.convert("RGB").convert("L"), dtype=np.uint8)
```

```
def canonical_view(img: Image.Image) -> Image.Image:
    """Grayscale image squashed to CANON_SIZE x CANON_SIZE (the network's square view).

    Squashing (not cropping) keeps the whole chest visible, so a heatmap computed on the
    square view can be stretched back onto the original aspect ratio pixel-exactly.
    """
    gray = Image.fromarray(to_gray_uint8(img))
    return gray.resize((CANON_SIZE, CANON_SIZE), Image.Resampling.LANCZOS)


def build_model(arch: str, pretrained: bool = True):
    """torchvision ResNet with a 2-class head (NORMAL, PNEUMONIA)."""
    import torch.nn as nn
    import torchvision.models as tvm

    weights = {
        "resnet18": tvm.ResNet18_Weights.IMAGENET1K_V1,
        "resnet34": tvm.ResNet34_Weights.IMAGENET1K_V1,
        "resnet50": tvm.ResNet50_Weights.IMAGENET1K_V2,
    }[arch] if pretrained else None
    model = getattr(tvm, arch)(weights=weights)
    model.fc = nn.Sequential(nn.Dropout(0.2), nn.Linear(model.fc.in_features, len(CLASSES)))
    return model


def binary_metrics(y_true, prob, threshold: float) -> dict:
    """All headline metrics for a binary classifier at a given threshold."""
    from sklearn.metrics import average_precision_score, confusion_matrix, roc_auc_score

    y_true = np.asarray(y_true).astype(int)
    pred = (np.asarray(prob) >= threshold).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_true, pred, labels=[0, 1]).ravel()
    sens = tp / max(tp + fn, 1)
    spec = tn / max(tn + fp, 1)
    prec = tp / max(tp + fp, 1)
    return {
        "n": int(len(y_true)),
        "accuracy": float((tp + tn) / len(y_true)),
        "balanced_accuracy": float((sens + spec) / 2),
        "precision": float(prec),
        "recall_sensitivity": float(sens),
        "specificity": float(spec),
        "f1": float(2 * prec * sens / max(prec + sens, 1e-12)),
        "roc_auc": float(roc_auc_score(y_true, prob)),
        "pr_auc": float(average_precision_score(y_true, prob)),
        "confusion_matrix": {"tn": int(tn), "fp": int(fp), "fn": int(fn), "tp": int(tp)},
        "threshold": float(threshold),
    }
```

A.2 ml/prepare_data.py

Dataset verification, patient-level split, cache and leakage checks.

```
"""Step 1: verify the dataset, make a patient-level validation split, and cache images.

Input : data/raw/chest_xray/{train,test}/{NORMAL,PNEUMONIA}/*.jpeg  (Kermany et al. 2018)
Output: data/splits.csv            one row per image: path, label, patient, split, sha1
        data/processed/...png      512x512 grayscale copies (fast to load during training)
        data/dataset_summary.json  counts + leakage checks (used by the report)

Run:  python -m ml.prepare_data
"""
from __future__ import annotations

import hashlib
import json
import re
from concurrent.futures import ThreadPoolExecutor
```

```
import numpy as np
import pandas as pd
from PIL import Image
from sklearn.model_selection import StratifiedGroupKFold

from ml.common import CACHE_DIR, CLASSES, DATA_DIR, RAW_DIR, SPLIT_CSV, canonical_view

EXPECTED = {("train", "NORMAL"): 1349, ("train", "PNEUMONIA"): 3883,
            ("test", "NORMAL"): 234, ("test", "PNEUMONIA"): 390}
VAL_FOLDS = 7          # 1 of 7 folds -> ~14% of TRAIN becomes validation
SEED = 42


def patient_id(filename: str, label: str) -> str:
    """Patient identifier parsed from the file name.

    PNEUMONIA: person123_bacteria_456.jpeg -> P123
    NORMAL   : IM-0115-0001.jpeg / NORMAL2-IM-0115-0001.jpeg -> N0115
    (the NORMAL2- prefix is ignored on purpose: grouping more conservatively can only
    reduce leakage between train and validation, never add it).
    """
    if label == "PNEUMONIA":
        m = re.match(r"person(\d+)_", filename)
        return f"P{int(m.group(1))}"
    m = re.search(r"IM-(\d+)-", filename)
    return f"N{int(m.group(1))}"


def near_duplicate_corr(test_paths, other_paths) -> np.ndarray:
    """For each test image: highest Pearson correlation with any train/val image (64x64
thumbnails).
    Values >= 0.99 would indicate the same radiograph re-saved; independent chest X-rays stay
below."""
    def thumbs(paths):
        a = np.stack([np.asarray(Image.open(CACHE_DIR / p).resize((64, 64),
Image.Resampling.BILINEAR),
                                 dtype=np.float32).ravel() for p in paths])
        return (a - a.mean(1, keepdims=True)) / (a.std(1, keepdims=True) + 1e-6)
    a, b = thumbs(other_paths), thumbs(test_paths)
    return (b @ a.T / a.shape[1]).max(1)


def process(row: dict) -> dict:
    src = RAW_DIR / row["raw_path"]
    data = src.read_bytes()
    row["sha1"] = hashlib.sha1(data).hexdigest()
    with Image.open(src) as im:
        row["orig_w"], row["orig_h"], row["orig_mode"] = im.width, im.height, im.mode
        view = canonical_view(im)
    dst = CACHE_DIR / row["path"]
    dst.parent.mkdir(parents=True, exist_ok=True)
    view.save(dst, optimize=False)
    return row


def main() -> None:
    rows = []
    for split in ("train", "test"):
        for label in CLASSES:
            files = sorted((RAW_DIR / split / label).glob("*.jpeg"))
            assert len(files) == EXPECTED[(split, label)], (split, label, len(files))
            for f in files:
                rows.append({"raw_path": f"{split}/{label}/{f.name}",
                             "path": f"{split}/{label}/{f.stem}.png",
                             "label": label, "y": CLASSES.index(label),
                             "patient": patient_id(f.name, label), "split": split})
    print(f"Found {len(rows)} images; counts match the official release.")

    with ThreadPoolExecutor(max_workers=12) as pool:
        rows = list(pool.map(process, rows))
    df = pd.DataFrame(rows)

    # Patient-level, stratified validation split carved out of TRAIN only.
```

```
    train_idx = df.index[df.split == "train"]
    sgkf = StratifiedGroupKFold(n_splits=VAL_FOLDS, shuffle=True, random_state=SEED)
    _, val_pos = next(sgkf.split(train_idx, df.loc[train_idx, "y"], df.loc[train_idx,
"patient"]))
    df.loc[train_idx[val_pos], "split"] = "val"

    # Leakage checks.
    tr, va, te = (df[df.split == s] for s in ("train", "val", "test"))
    checks = {
        "patients_train_and_val": len(set(tr.patient) & set(va.patient)),
        "patients_train_or_val_and_test": len(set(df[df.split != "test"].patient) &
set(te.patient)),
        "exact_duplicate_files_test_vs_trainval": int(te.sha1.isin(df[df.split !=
"test"].sha1).sum()),
        "exact_duplicate_files_val_vs_train": int(va.sha1.isin(tr.sha1).sum()),
    }
    corr = near_duplicate_corr(list(te.path), list(df[df.split != "test"].path))
    checks["near_duplicates_test_vs_trainval_corr_ge_0.99"] = int((corr >= 0.99).sum())
    checks["max_thumbnail_corr_test_vs_trainval"] = round(float(corr.max()), 4)
    checks["note"] = ("patient numbers are assigned separately inside the official train and
test folders, "
                      "so equal numbers across them are NOT the same patient (dataset authors:
independent "
                      "patients); exact-hash and near-duplicate checks confirm no shared
images")
    assert checks["patients_train_and_val"] == 0
    assert checks["exact_duplicate_files_test_vs_trainval"] == 0, "test image found in train!"

    counts = {s: {c: int(((df.split == s) & (df.label == c)).sum()) for c in CLASSES}
              for s in ("train", "val", "test")}
    patients = {s: int(df[df.split == s].patient.nunique()) for s in ("train", "val", "test")}
    summary = {"counts": counts, "patients": patients, "leakage_checks": checks,
               "val_fraction_of_official_train": round(len(va) / (len(tr) + len(va)), 4),
               "original_size_px": {"median_w": int(df.orig_w.median()), "median_h":
int(df.orig_h.median()),
                                    "min_w": int(df.orig_w.min()), "max_w":
int(df.orig_w.max())},
               "original_modes": df.orig_mode.value_counts().to_dict()}
    df.drop(columns=["y"]).to_csv(SPLIT_CSV, index=False)
    (DATA_DIR / "dataset_summary.json").write_text(json.dumps(summary, indent=2))
    print(json.dumps(summary, indent=2))


if __name__ == "__main__":
    main()
```

A.3 ml/datasets.py

PyTorch dataset and augmentation pipelines.

```
"""PyTorch dataset + augmentation pipelines for chest X-rays.

Images come from the 512x512 grayscale cache made by prepare_data.py. The network
gets the gray image replicated to 3 channels with ImageNet normalisation, because
the ResNet backbones were pretrained on colour ImageNet photos.
"""
from __future__ import annotations

import pandas as pd
import torch
from PIL import Image
from torch.utils.data import Dataset
from torchvision.transforms import v2 as T

from ml.common import CACHE_DIR, CLASSES, IMAGENET_MEAN, IMAGENET_STD, SPLIT_CSV


class AddGaussianNoise(torch.nn.Module):
    """Light sensor-like noise on a [0,1] tensor."""
```

```
    def __init__(self, sigma: float = 0.02):
        super().__init__()
        self.sigma = sigma

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return (x + torch.randn_like(x) * self.sigma).clamp(0, 1)


class GrayTo3(torch.nn.Module):
    """Replicate the single gray channel to RGB (a class, not a lambda, so Windows
    DataLoader workers can pickle it)."""

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return x.expand(3, -1, -1).contiguous()


def to_network_input() -> list:
    """Shared tail of every pipeline: [0,1] gray tensor -> normalised 3-channel tensor."""
    return [GrayTo3(), T.Normalize(IMAGENET_MEAN, IMAGENET_STD)]


def train_transform(res: int) -> T.Compose:
    """Realistic X-ray augmentations: small crops, tilts, exposure changes, blur, noise."""
    return T.Compose([
        T.ToImage(), T.ToDtype(torch.float32, scale=True),
        T.RandomResizedCrop(res, scale=(0.75, 1.0), ratio=(0.9, 1.1), antialias=True),
        T.RandomHorizontalFlip(p=0.5),
        T.RandomAffine(degrees=10, translate=(0.05, 0.05), scale=(0.95, 1.05)),
        T.ColorJitter(brightness=0.2, contrast=0.2),
        T.RandomApply([T.GaussianBlur(kernel_size=5, sigma=(0.1, 1.2))], p=0.2),
        T.RandomApply([AddGaussianNoise(0.02)], p=0.2),
        *to_network_input(),
    ])


def eval_transform(res: int) -> T.Compose:
    """Deterministic resize used for validation, test and the app."""
    return T.Compose([
        T.ToImage(), T.ToDtype(torch.float32, scale=True),
        T.Resize((res, res), antialias=True),
        *to_network_input(),
    ])


class XrayDataset(Dataset):
    """Rows of data/splits.csv for one split ('train', 'val' or 'test')."""

    def __init__(self, split: str, transform, csv=SPLIT_CSV):
        df = pd.read_csv(csv)
        self.df = df[df.split == split].reset_index(drop=True)
        self.labels = [CLASSES.index(lbl) for lbl in self.df.label]
        self.transform = transform

    def __len__(self) -> int:
        return len(self.df)

    def __getitem__(self, i: int):
        with Image.open(CACHE_DIR / self.df.path[i]) as im:
            img = im.convert("L")
        return self.transform(img), self.labels[i]
```

A.4 ml/train.py

Fine-tuning of one ResNet.

```
"""Step 2: fine-tune an ImageNet-pretrained ResNet on the training split.

Model selection uses ONLY the validation split (val ROC-AUC, ties -> lower val loss).
The test split is never touched here.
```

```
Example:
  python -m ml.train --name r50_384_s1 --arch resnet50 --res 384 --epochs 20 --bs 24
Outputs (models/checkpoints/<name>/): best.pt, history.json, val_preds.npz, config.json
Log: logs/train_<name>.log
"""
from __future__ import annotations

import argparse
import copy
import json
import math
import time

import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import DataLoader

from ml.common import CKPT_DIR, LOGS_DIR, binary_metrics, build_model
from ml.datasets import XrayDataset, eval_transform, train_transform


def parse_args():
    p = argparse.ArgumentParser(description=__doc__,
formatter_class=argparse.RawDescriptionHelpFormatter)
    p.add_argument("--name", required=True)
    p.add_argument("--arch", default="resnet50", choices=["resnet18", "resnet34", "resnet50"])
    p.add_argument("--res", type=int, default=384)
    p.add_argument("--epochs", type=int, default=20)
    p.add_argument("--bs", type=int, default=24)
    p.add_argument("--lr", type=float, default=3e-4, help="head learning rate")
    p.add_argument("--backbone-lr-mult", type=float, default=0.33, help="backbone lr = lr *
mult")
    p.add_argument("--wd", type=float, default=1e-2)
    p.add_argument("--warmup", type=float, default=1.0, help="warm-up epochs")
    p.add_argument("--label-smoothing", type=float, default=0.1)
    p.add_argument("--ema", type=float, default=0.998, help="EMA decay of weights (0 = off)")
    p.add_argument("--patience", type=int, default=7, help="early-stopping patience (epochs)")
    p.add_argument("--workers", type=int, default=3)
    p.add_argument("--seed", type=int, default=42)
    return p.parse_args()


class EMA:
    """Exponential moving average of model weights (a smoother model for evaluation)."""

    def __init__(self, model: nn.Module, decay: float):
        self.decay, self.steps, self.model = decay, 0, copy.deepcopy(model).eval()
        for p in self.model.parameters():
            p.requires_grad_(False)

    @torch.no_grad()
    def update(self, model: nn.Module) -> None:
        self.steps += 1
        d = min(self.decay, (1 + self.steps) / (10 + self.steps))  # warm-up: follow closely at
first
        for e, m in zip(self.model.state_dict().values(), model.state_dict().values()):
            if e.dtype.is_floating_point:
                e.mul_(d).add_(m.detach(), alpha=1 - d)
            else:
                e.copy_(m)


@torch.no_grad()
def predict_logits(model, loader, device) -> tuple[np.ndarray, np.ndarray]:
    model.eval()
    out, ys = [], []
    for x, y in loader:
        x = x.to(device, non_blocking=True)
        with torch.autocast("cuda", dtype=torch.float16, enabled=device.type == "cuda"):
            out.append(model(x).float().cpu())
        ys.append(y)
    return torch.cat(out).numpy(), torch.cat(ys).numpy()
```

```
def main() -> None:
    a = parse_args()
    torch.manual_seed(a.seed)
    np.random.seed(a.seed)
    # NOTE: channels_last memory format was measured 8x SLOWER on this RTX 3050 + torch 2.11
    # (66 vs 516 img/s for ResNet18), so the default NCHW layout is used everywhere.
    torch.backends.cudnn.benchmark = False  # benchmark mode gave no speed-up here and spikes
memory
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    out_dir = CKPT_DIR / a.name
    out_dir.mkdir(parents=True, exist_ok=True)
    LOGS_DIR.mkdir(exist_ok=True)
    log_f = open(LOGS_DIR / f"train_{a.name}.log", "a", encoding="utf-8")

    def log(msg: str) -> None:
        line = f"[{time.strftime('%H:%M:%S')}] {msg}"
        print(line, flush=True)
        log_f.write(line + "\n")
        log_f.flush()

    log(f"config {vars(a)} device={device}")
    (out_dir / "config.json").write_text(json.dumps(vars(a), indent=2))

    train_ds = XrayDataset("train", train_transform(a.res))
    val_ds = XrayDataset("val", eval_transform(a.res))
    # Each Windows worker process costs ~0.6 GB RAM (it re-imports torch), so keep them few;
    # validation runs in the main process (747 images, a few seconds).
    train_dl = DataLoader(train_ds, batch_size=a.bs, shuffle=True, drop_last=True,
num_workers=a.workers,
                          pin_memory=True, persistent_workers=a.workers > 0)
    val_dl = DataLoader(val_ds, batch_size=a.bs * 2, shuffle=False, num_workers=0)
    log(f"train={len(train_ds)} val={len(val_ds)}")

    # Class-weighted loss: NORMAL is the minority class (~26% of train).
    counts = np.bincount(train_ds.labels, minlength=2)
    class_w = torch.tensor(counts.sum() / (2 * counts), dtype=torch.float32, device=device)
    criterion = nn.CrossEntropyLoss(weight=class_w, label_smoothing=a.label_smoothing)
    log(f"class counts={counts.tolist()} weights={class_w.tolist()}")

    model = build_model(a.arch).to(device)
    head = list(model.fc.parameters())
    head_ids = {id(p) for p in head}
    backbone = [p for p in model.parameters() if id(p) not in head_ids]
    opt = torch.optim.AdamW([{"params": backbone, "lr": a.lr * a.backbone_lr_mult},
                             {"params": head, "lr": a.lr}], weight_decay=a.wd)
    base_lrs = [g["lr"] for g in opt.param_groups]
    steps_per_epoch = len(train_dl)
    total, warm = a.epochs * steps_per_epoch, int(a.warmup * steps_per_epoch)
    scaler = torch.amp.GradScaler("cuda", enabled=device.type == "cuda")
    ema = EMA(model, a.ema) if a.ema > 0 else None

    history, best, bad_epochs, step = [], (-1.0, 1e9), 0, 0
    t_start = time.time()
    for epoch in range(1, a.epochs + 1):
        model.train()
        t0, run_loss, run_correct, seen = time.time(), 0.0, 0, 0
        for x, y in train_dl:
            # Cosine schedule with linear warm-up, updated every step.
            f = (step + 1) / max(warm, 1) if step < warm else \
                0.5 * (1 + math.cos(math.pi * (step - warm) / max(total - warm, 1)))
            for g, lr0 in zip(opt.param_groups, base_lrs):
                g["lr"] = lr0 * f
            x = x.to(device, non_blocking=True)
            y = y.to(device, non_blocking=True)
            with torch.autocast("cuda", dtype=torch.float16, enabled=device.type == "cuda"):
                logits = model(x)
                loss = criterion(logits.float(), y)
            opt.zero_grad(set_to_none=True)
            scaler.scale(loss).backward()
            scaler.unscale_(opt)
            nn.utils.clip_grad_norm_(model.parameters(), 2.0)
```

```
            scaler.step(opt)
            scaler.update()
            if ema:
                ema.update(model)
            step += 1
            run_loss += loss.item() * len(y)
            run_correct += (logits.argmax(1) == y).sum().item()
            seen += len(y)

        eval_model = ema.model if ema else model
        logits, yv = predict_logits(eval_model, val_dl, device)
        val_loss = float(criterion(torch.from_numpy(logits).to(device),
torch.from_numpy(yv).to(device)))
        prob = torch.softmax(torch.from_numpy(logits), 1)[:, 1].numpy()
        m = binary_metrics(yv, prob, 0.5)
        rec = {"epoch": epoch, "train_loss": run_loss / seen, "train_acc": run_correct / seen,
               "val_loss": val_loss, "val_acc": m["accuracy"], "val_bal_acc":
m["balanced_accuracy"],
               "val_auc": m["roc_auc"], "lr": opt.param_groups[1]["lr"], "sec": time.time() -
t0}
        history.append(rec)
        improved = (m["roc_auc"], -val_loss) > (best[0], -best[1])
        if improved:
            best, bad_epochs = (m["roc_auc"], val_loss), 0
            torch.save({"arch": a.arch, "res": a.res, "epoch": epoch, "state_dict":
eval_model.state_dict()},
                       out_dir / "best.pt")
            np.savez(out_dir / "val_preds.npz", logits=logits, y=yv)
        else:
            bad_epochs += 1
        log(f"ep {epoch:02d} train_loss {rec['train_loss']:.4f} acc {rec['train_acc']:.4f} |
val_loss {val_loss:.4f} "
            f"acc {m['accuracy']:.4f} bal {m['balanced_accuracy']:.4f} auc {m['roc_auc']:.5f} "
            f"| {rec['sec']:.0f}s {'*best*' if improved else ''}")
        (out_dir / "history.json").write_text(json.dumps(history, indent=2))
        if bad_epochs >= a.patience:
            log(f"early stop at epoch {epoch}")
            break

    best_ep = max(history, key=lambda r: (r["val_auc"], -r["val_loss"]))["epoch"]
    summary = {"best_epoch": best_ep, "best_val_auc": best[0], "best_val_loss": best[1],
               "epochs_run": len(history), "train_minutes": (time.time() - t_start) / 60}
    (out_dir / "summary.json").write_text(json.dumps(summary, indent=2))
    log(f"DONE {summary}")


if __name__ == "__main__":
    main()
```

A.5 ml/evaluate.py

Ensemble, calibration, threshold selection, test scoring and export.

```
"""Step 3: evaluate trained ResNets, build the ensemble, calibrate and pick the threshold.

Everything that is *chosen* (temperature, decision threshold) is fitted on the
VALIDATION split. The test split is only scored, and every test scoring is appended
to logs/test_evaluations.jsonl so the report can list all of them honestly.

  python -m ml.evaluate --members r50_384_s1 r34_320_s1 --val-only   # validation only
  python -m ml.evaluate --members r50_384_s1 r34_320_s1 --export     # final: writes models/
"""
from __future__ import annotations

import argparse
import json
import shutil
import time

import numpy as np
```

```
import torch
from scipy.optimize import minimize_scalar
from torch.utils.data import DataLoader

from ml.common import CKPT_DIR, LOGS_DIR, MODELS_DIR, SPLIT_CSV, binary_metrics, build_model
from ml.datasets import XrayDataset, eval_transform


def load_member(name: str, device) -> tuple[torch.nn.Module, dict]:
    """Load a checkpoint, rounding weights through fp16 exactly as they are shipped."""
    ckpt = torch.load(CKPT_DIR / name / "best.pt", map_location="cpu", weights_only=False)
    model = build_model(ckpt["arch"], pretrained=False)
    sd = {k: v.half().float() if v.is_floating_point() else v for k, v in
ckpt["state_dict"].items()}
    model.load_state_dict(sd)
    return model.to(device).eval(), ckpt


@torch.no_grad()
def margins(model, split: str, res: int, device, tta: bool) -> tuple[np.ndarray, np.ndarray]:
    """Logit margin z = logit(PNEUMONIA) - logit(NORMAL); TTA averages original + mirrored."""
    dl = DataLoader(XrayDataset(split, eval_transform(res)), batch_size=32, num_workers=4)
    zs, ys = [], []
    for x, y in dl:
        x = x.to(device)
        with torch.autocast("cuda", dtype=torch.float16, enabled=device.type == "cuda"):
            views = [x, torch.flip(x, dims=[3])] if tta else [x]
            z = torch.stack([(lg[:, 1] - lg[:, 0]).float() for lg in map(model, views)]).mean(0)
        zs.append(z.cpu())
        ys.append(y)
    return torch.cat(zs).numpy(), torch.cat(ys).numpy()


def fit_temperature(z: np.ndarray, y: np.ndarray) -> float:
    """Temperature T minimising validation negative log-likelihood of sigmoid(z / T)."""
    def nll(log_t):
        p = 1 / (1 + np.exp(-z / np.exp(log_t)))
        p = np.clip(p, 1e-7, 1 - 1e-7)
        return -np.mean(y * np.log(p) + (1 - y) * np.log(1 - p))
    return float(np.exp(minimize_scalar(nll, bounds=(-3, 3), method="bounded").x))


def pick_threshold(prob: np.ndarray, y: np.ndarray) -> float:
    """Threshold maximising Youden's J (= balanced accuracy) on validation.

    Ties are common (a whole interval of thresholds gives the same J), so we take
    the middle of the best interval, measured in logit space.
    """
    cands = np.unique(np.concatenate([prob, [0.0, 1.0]]))
    mids = (cands[:-1] + cands[1:]) / 2
    j = np.array([np.mean(prob[y == 1] >= t) + np.mean(prob[y == 0] < t) - 1 for t in mids])
    best = mids[np.isclose(j, j.max())]
    logit = np.log(np.clip(best, 1e-6, 1 - 1e-6) / (1 - np.clip(best, 1e-6, 1 - 1e-6)))
    return float(1 / (1 + np.exp(-(logit.min() + logit.max()) / 2)))


def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--members", nargs="+", required=True)
    ap.add_argument("--no-tta", action="store_true")
    ap.add_argument("--val-only", action="store_true", help="never touch the test split")
    ap.add_argument("--export", action="store_true", help="write final models/ files")
    ap.add_argument("--note", default="", help="free-text note stored with the test log entry")
    a = ap.parse_args()
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    tta = not a.no_tta
    splits = ["val"] if a.val_only else ["val", "test"]

    z, y, ckpts, t0 = {}, {}, {}, time.time()
    for name in a.members:
        model, ckpt = load_member(name, device)
        ckpts[name] = ckpt
        for s in splits:
```

```
            z[(name, s)], y[s] = margins(model, s, ckpt["res"], device, tta)
            if s == "val":  # cached for threshold analysis / figures (validation only)
                np.savez(CKPT_DIR / name / f"val_margins{'_tta' if tta else ''}.npz", z=z[(name,
s)], y=y[s])
        del model
        torch.cuda.empty_cache()

    results = {}
    for name in a.members + (["ensemble"] if len(a.members) > 1 else []):
        pick = a.members if name == "ensemble" else [name]
        zv = np.mean([z[(m, "val")] for m in pick], axis=0)
        t = fit_temperature(zv, y["val"])
        pv = 1 / (1 + np.exp(-zv / t))
        thr = pick_threshold(pv, y["val"])
        r = {"temperature": t, "threshold": thr, "val": binary_metrics(y["val"], pv, thr)}
        if not a.val_only:
            zt = np.mean([z[(m, "test")] for m in pick], axis=0)
            pt = 1 / (1 + np.exp(-zt / t))
            r["test"] = binary_metrics(y["test"], pt, thr)
            r["_test_prob"] = pt
        r["_val_prob"] = pv
        results[name] = r
        line = f"{name:>14}  T={t:.3f} thr={thr:.3f}  VAL acc {r['val']['accuracy']:.4f} " \
               f"bal {r['val']['balanced_accuracy']:.4f} auc {r['val']['roc_auc']:.4f}"
        if "test" in r:
            m = r["test"]
            line += f" | TEST acc {m['accuracy']:.4f} sens {m['recall_sensitivity']:.4f} " \
                    f"spec {m['specificity']:.4f} auc {m['roc_auc']:.4f}"
        print(line)

    if not a.val_only:  # honesty log: every time the test split is scored
        LOGS_DIR.mkdir(exist_ok=True)
        with open(LOGS_DIR / "test_evaluations.jsonl", "a", encoding="utf-8") as f:
            f.write(json.dumps({"time": time.strftime("%Y-%m-%d %H:%M:%S"), "members":
a.members,
                                "tta": tta, "note": a.note,
                                "results": {k: {"threshold": v["threshold"], "test": v["test"]}
                                            for k, v in results.items()}}) + "\n")

    if a.export:
        export(a.members, ckpts, results, y, tta)
    print(f"done in {time.time() - t0:.0f}s")


def export(members, ckpts, results, y, tta) -> None:
    """Ship fp16 weights + ensemble.json + metrics.json + predictions for the figures."""
    final = results["ensemble" if len(members) > 1 else members[0]]
    thr = final["threshold"]
    ens = {"members": [], "temperature": final["temperature"], "threshold": thr, "tta_hflip":
tta,
           "combine": "mean of member logit margins (PNEUMONIA - NORMAL), then sigmoid(z /
temperature)",
           "risk_bands": {"Low": [0.0, thr / 2], "Moderate": [thr / 2, (1 + thr) / 2],
                          "High": [(1 + thr) / 2, 1.0]}}
    per_model, pred_dump = {}, {"y_val": y["val"], "y_test": y["test"]}
    for name in members:
        ck = ckpts[name]
        sd = {k: v.half() if v.is_floating_point() else v for k, v in ck["state_dict"].items()}
        torch.save({"arch": ck["arch"], "res": ck["res"], "state_dict": sd}, MODELS_DIR /
f"{name}.pt")
        cfg = json.loads((CKPT_DIR / name / "config.json").read_text())
        summ = json.loads((CKPT_DIR / name / "summary.json").read_text())
        ens["members"].append({"name": name, "file": f"{name}.pt", "arch": ck["arch"], "res":
ck["res"]})
        per_model[name] = {k: results[name][k] for k in ("temperature", "threshold", "val",
"test")}
        per_model[name].update(hyperparameters=cfg, training=summ)
        pred_dump[f"{name}_val"], pred_dump[f"{name}_test"] = results[name]["_val_prob"],
results[name]["_test_prob"]
        shutil.copy(CKPT_DIR / name / "history.json", MODELS_DIR / f"{name}_history.json")
    pred_dump["ensemble_val"], pred_dump["ensemble_test"] = final["_val_prob"],
final["_test_prob"]
    np.savez(CKPT_DIR / "final_preds.npz", **pred_dump)
    (MODELS_DIR / "ensemble.json").write_text(json.dumps(ens, indent=2))
```

```
    summary = json.loads((SPLIT_CSV.parent / "dataset_summary.json").read_text())
    metrics = {
        "model_version": "pneumoscan-resnet-ensemble-v1",
        "task": "binary classification of pediatric chest X-rays: NORMAL vs PNEUMONIA",
        "final": {"members": members, "tta_hflip": tta, "temperature": final["temperature"],
                  "threshold": thr, "threshold_rule": "maximise Youden's J on validation (middle
of best interval)",
                  "risk_bands": ens["risk_bands"], "val": final["val"], "test": final["test"]},
        "per_model": per_model,
        "dataset": {"name": "Kermany et al. 2018, Chest X-Ray Images (Pneumonia),
ChestXRay2017.zip",
                    "source": "https://data.mendeley.com/datasets/rscbjbr9sj/2",
                    "doi": "10.17632/rscbjbr9sj.2", "license": "CC BY 4.0",
                    "counts": summary["counts"], "patients": summary["patients"],
                    "split_method": "official test set held out; validation = patient-level
stratified "
                                    "1/7 of official train (StratifiedGroupKFold, seed 42)",
                    "leakage_checks": summary["leakage_checks"]},
    }
    gate_file = MODELS_DIR / "xray_gate_metrics.json"
    if gate_file.exists():
        metrics["xray_gate"] = json.loads(gate_file.read_text())
    (MODELS_DIR / "metrics.json").write_text(json.dumps(metrics, indent=2))
    print("exported to", MODELS_DIR)


if __name__ == "__main__":
    main()
```

A.6 ml/gradcam.py

Grad-CAM.

```
"""Grad-CAM (Selvaraju et al., 2017) for torchvision ResNets.

The heatmap is taken from the last convolutional block (layer4): each of its 2048
(ResNet50) feature maps is weighted by the average gradient of the class score with
respect to it, the weighted maps are summed, and ReLU keeps only positive evidence.

For this 2-class model the score is the logit margin z = logit(PNEUMONIA) - logit(NORMAL).
The gradient of the NORMAL margin is simply -dz, so ONE backward pass serves both classes.
"""
from __future__ import annotations

import numpy as np
import torch
import torch.nn.functional as F


class GradCAM:
    def __init__(self, model: torch.nn.Module, layer: torch.nn.Module | None = None):
        self.model = model
        self._acts = None
        (layer if layer is not None else model.layer4).register_forward_hook(self._save)

    def _save(self, _module, _inp, out):
        self._acts = out

    def explain(self, x: torch.Tensor):
        """Forward + backward once. Returns (activations, d z / d activations, logits)."""
        self.model.eval()
        with torch.enable_grad():
            x = x.detach().requires_grad_(True)  # lets gradients flow even if weights are
frozen
            logits = self.model(x)
            z = logits[:, 1] - logits[:, 0]
            grads = torch.autograd.grad(z.sum(), self._acts)[0]
        return self._acts.detach(), grads.detach(), logits.detach()
```

```
    @staticmethod
    def cam(acts: torch.Tensor, grads: torch.Tensor, class_idx: int, size: tuple[int, int]) ->
np.ndarray:
        """Grad-CAM map in [0, 1] for class_idx (0 NORMAL, 1 PNEUMONIA) resized to size=(H,
W)."""
        sign = 1.0 if class_idx == 1 else -1.0
        weights = sign * grads.float().mean(dim=(2, 3), keepdim=True)
        m = F.relu((weights * acts.float()).sum(1, keepdim=True))
        m = F.interpolate(m, size=size, mode="bilinear", align_corners=False)[0,
0].cpu().numpy()
        return m / m.max() if m.max() > 0 else m


def colorize(cam: np.ndarray) -> np.ndarray:
    """[0,1] map -> uint8 RGBA 'jet' heatmap (blue = low, red = high).

    Alpha grows with the map value (0.2 .. 1.0), so weak regions only lightly tint the
    X-ray while strong regions stand out. Drawn over the X-ray at 60 % opacity this
    PNG reproduces the overlay image exactly, so the app can offer an opacity slider.
    """
    import cv2

    c = np.clip(cam, 0, 1)
    rgb = cv2.cvtColor(cv2.applyColorMap(np.uint8(c * 255), cv2.COLORMAP_JET),
cv2.COLOR_BGR2RGB)
    alpha = np.uint8(np.round((0.2 + 0.8 * c) * 255))
    return np.dstack([rgb, alpha])


def overlay(gray: np.ndarray, heat_rgba: np.ndarray, opacity: float = 0.6) -> np.ndarray:
    """Alpha-blend an RGBA heatmap (at the given overall opacity) onto a uint8 grayscale
image."""
    a = heat_rgba[..., 3:4].astype(np.float32) / 255.0 * opacity
    base = np.repeat(gray[..., None], 3, axis=2).astype(np.float32)
    return np.uint8(np.clip((1 - a) * base + a * heat_rgba[..., :3].astype(np.float32), 0, 255))
```

A.7 ml/xray_gate.py

X-ray gate (heuristics and Mahalanobis out-of-distribution detector).

```
"""X-ray gate: refuse images that are not chest X-rays (selfies, pets, documents, blanks...).

Two layers of checks:
  1. Cheap image heuristics: too small, blank/uniform, clearly colourful.
  2. Out-of-distribution score: the 2048-d (ResNet50) pooled layer4 features of the
     trained network are compared with the training X-rays using a class-conditional
     Mahalanobis distance with a shared (Ledoit-Wolf) covariance (Lee et al., 2018).
     The cut-off is set from VALIDATION X-rays only.

Fit + validate:  python -m ml.xray_gate
Outputs: models/xray_gate.npz (means, precision, threshold), models/xray_gate_metrics.json
"""
from __future__ import annotations

import json

import numpy as np
from PIL import Image

from ml.common import MODELS_DIR, to_gray_uint8

MIN_SIDE = 64          # px; anything smaller cannot be a usable radiograph
MIN_STD = 8.0          # gray-level std; below this the image is essentially blank
MAX_MEAN_SAT = 0.12    # mean HSV saturation of non-dark pixels; X-rays are ~0
MAX_SAT_FRAC = 0.05    # fraction of strongly coloured pixels (saturation > 0.25)
THRESH_QUANTILE = 0.995
THRESH_MARGIN = 1.15   # threshold = margin x 99.5th percentile of validation distances


def heuristics(img: Image.Image) -> tuple[bool, str, dict]:
```

```
    """Fast checks that need no neural network."""
    w, h = img.size
    if min(w, h) < MIN_SIDE:
        return False, f"Image is too small ({w}x{h} px). Please upload the full X-ray.", {}
    gray = to_gray_uint8(img)
    std = float(gray.std())
    if std < MIN_STD:
        return False, "Image looks blank or uniform, not an X-ray.", {"gray_std": std}
    stats = {"gray_std": std, "mean_saturation": 0.0, "colour_fraction": 0.0}
    if img.mode not in ("L", "LA", "1", "I", "I;16", "I;16B", "I;16L", "I;16N", "F"):
        rgb = img.convert("RGB")
        rgb.thumbnail((256, 256))
        a = np.asarray(rgb, dtype=np.float32)
        mx, mn = a.max(2), a.min(2)
        lit = mx > 25                                    # ignore near-black pixels
        sat = np.where(lit, (mx - mn) / np.maximum(mx, 1), 0)
        stats["mean_saturation"] = float(sat[lit].mean()) if lit.any() else 0.0
        stats["colour_fraction"] = float((sat > 0.25).mean())
        if stats["mean_saturation"] > MAX_MEAN_SAT or stats["colour_fraction"] > MAX_SAT_FRAC:
            return False, "Image is in colour; chest X-rays are grayscale.", stats
    return True, "ok", stats


class XrayGate:
    """Mahalanobis OOD detector on network features (parameters from models/xray_gate.npz)."""

    def __init__(self, path=MODELS_DIR / "xray_gate.npz"):
        p = np.load(path)
        self.means, self.precision, self.threshold = p["means"], p["precision"],
float(p["threshold"])
        self.member = str(p["member"])

    def distance(self, feats: np.ndarray) -> np.ndarray:
        """Min over classes of sqrt(Mahalanobis distance); feats: (N, D)."""
        feats = np.atleast_2d(feats).astype(np.float64)
        d = [np.einsum("nd,de,ne->n", feats - m, self.precision, feats - m) for m in self.means]
        return np.sqrt(np.maximum(np.min(d, axis=0), 0))

    def score(self, dist: float) -> float:
        """Map distance to 0..1 (1 = typical chest X-ray, 0.5 exactly at the cut-off)."""
        return float(1 / (1 + (dist / self.threshold) ** 4))

    def decide(self, img: Image.Image, feats: np.ndarray | None) -> dict:
        ok, reason, stats = heuristics(img)
        if not ok:
            return {"is_xray": False, "score": 0.0, "reason": reason}
        dist = float(self.distance(feats)[0])
        s = self.score(dist)
        if dist > self.threshold:
            return {"is_xray": False, "score": s,
                    "reason": "This does not look like a chest X-ray (it is unlike the X-rays
the model was trained on)."}
        return {"is_xray": True, "score": s, "reason": "Looks like a chest X-ray."}


# ---------------------------------------------------------------------------- fitting

def non_xray_images() -> dict[str, Image.Image]:
    """Validation set of things that are NOT chest X-rays (all free to use)."""
    from PIL import ImageDraw
    from skimage import data

    imgs = {}
    for n in ["astronaut", "chelsea", "coffee", "rocket", "text", "camera", "moon", "page",
"coins",
              "horse", "clock", "hubble_deep_field", "immunohistochemistry", "retina", "brick",
              "grass", "gravel", "cell", "checkerboard", "cat", "colorwheel", "logo",
"motorcycle_left"]:
        try:
            arr = getattr(data, n)()
            if arr.dtype == bool:
                arr = arr.astype(np.uint8) * 255
            imgs[f"skimage_{n}"] = Image.fromarray(arr)
```

```
        except Exception as e:  # some images need an optional download
            print("skip", n, e)
    rng = np.random.default_rng(0)
    imgs["noise_gray"] = Image.fromarray(rng.integers(0, 256, (512, 512), dtype=np.uint8))
    imgs["noise_rgb"] = Image.fromarray(rng.integers(0, 256, (512, 512, 3), dtype=np.uint8))
    for c in [(0, 0, 0), (255, 255, 255), (128, 128, 128), (220, 30, 30), (30, 90, 200)]:
        imgs[f"plain_{c}"] = Image.new("RGB", (400, 300), c)
    imgs["gradient"] = Image.fromarray(np.tile(np.linspace(0, 255, 512, dtype=np.uint8), (512,
1)))
    for k in range(3):  # synthetic documents: black text on white paper
        doc = Image.new("L", (850, 1100), 255)
        d = ImageDraw.Draw(doc)
        for i in range(40):
            words = " ".join("".join(chr(97 + rng.integers(26)) for _ in range(rng.integers(2,
9)))
                             for _ in range(10))
            d.text((60, 40 + i * 25 + k * 3), words, fill=0)
        if k == 2:
            doc = Image.fromarray(255 - np.asarray(doc))  # inverted (white text on black)
        imgs[f"document_{k}"] = doc
    return imgs


def main() -> None:
    from sklearn.covariance import LedoitWolf
    from torch.utils.data import DataLoader

    from ml.datasets import XrayDataset, eval_transform
    from ml.inference import PneumoniaModel

    pm = PneumoniaModel(MODELS_DIR, gate=False)       # loads the shipped ensemble weights
    member = pm.members[0]
    print("gate feature extractor:", member["name"])

    def feats_for(split):
        dl = DataLoader(XrayDataset(split, eval_transform(member["res"])), batch_size=32,
num_workers=4)
        out, ys = [], []
        for x, y in dl:
            out.append(pm.features(x))
            ys.append(y.numpy())
        return np.concatenate(out), np.concatenate(ys)

    ftr, ytr = feats_for("train")
    lw = LedoitWolf().fit(np.concatenate([ftr[ytr == c] - ftr[ytr == c].mean(0) for c in (0,
1)]))
    means = np.stack([ftr[ytr == c].mean(0) for c in (0, 1)])
    np.savez(MODELS_DIR / "xray_gate.npz", means=means,
precision=lw.precision_.astype(np.float32),
             threshold=1.0, member=member["name"])
    gate = XrayGate()
    dval = gate.distance(feats_for("val")[0])
    thr = float(np.quantile(dval, THRESH_QUANTILE) * THRESH_MARGIN)
    np.savez(MODELS_DIR / "xray_gate.npz", means=means,
precision=lw.precision_.astype(np.float32),
             threshold=thr, member=member["name"])
    pm.gate = XrayGate()

    # Validate end-to-end with check_image() on real files (test X-rays) and non-X-rays.
    import pandas as pd

    from ml.common import RAW_DIR, SPLIT_CSV
    df = pd.read_csv(SPLIT_CSV)
    test_rows = df[df.split == "test"]
    acc = [pm.check_image(Image.open(RAW_DIR / r)) for r in test_rows.raw_path]
    rejected_test = [r for r, c in zip(test_rows.raw_path, acc) if not c["is_xray"]]
    imgs = non_xray_images()
    nx = {k: pm.check_image(v) for k, v in imgs.items()}
    wrongly_accepted = [k for k, c in nx.items() if c["is_xray"]]
    # Harder variant: grayscale copies of the colour images, so only the feature-distance test
can catch them.
    gray = {k: pm.check_image(v.convert("L")) for k, v in imgs.items() if v.mode != "L"}
    gray_accepted = [k for k, c in gray.items() if c["is_xray"]]
```

```
    res = {"feature_member": member["name"], "threshold_distance": thr,
           "threshold_rule": f"{THRESH_MARGIN} x {THRESH_QUANTILE:.1%} quantile of validation
distances",
           "val_distance_quantiles": {q: float(np.quantile(dval, q)) for q in (0.5, 0.9, 0.99,
0.995, 1.0)},
           "test_xrays_accepted": f"{len(acc) - len(rejected_test)}/{len(acc)}",
           "test_xray_accept_rate": (len(acc) - len(rejected_test)) / len(acc),
           "test_xrays_rejected": rejected_test,
           "non_xray_rejected": f"{len(nx) - len(wrongly_accepted)}/{len(nx)}",
           "non_xray_wrongly_accepted": wrongly_accepted,
           "grayscale_copies_of_colour_images_rejected": f"{len(gray) -
len(gray_accepted)}/{len(gray)}",
           "grayscale_copies_wrongly_accepted": gray_accepted,
           "non_xray_details": {k: {"is_xray": c["is_xray"], "score": round(c["score"], 4),
"reason": c["reason"]}
                                for k, c in nx.items()}}
    (MODELS_DIR / "xray_gate_metrics.json").write_text(json.dumps(res, indent=2))
    mpath = MODELS_DIR / "metrics.json"
    if mpath.exists():
        m = json.loads(mpath.read_text())
        m["xray_gate"] = res
        mpath.write_text(json.dumps(m, indent=2))
    print(json.dumps({k: v for k, v in res.items() if k != "non_xray_details"}, indent=2))


if __name__ == "__main__":
    main()
```

A.8 ml/inference.py

PneumoniaModel inference API used by the application.

```
"""PneumoScan AI inference API (the contract used by the app, see docs/PROJECT_PLAN.md).

    from ml.inference import PneumoniaModel
    model = PneumoniaModel("models")          # loads weights once (GPU if available)
    model.check_image(pil_img)                # {"is_xray", "score", "reason"}
    model.predict(pil_img)                    # label, calibrated probability, risk, Grad-CAM
PNGs...
    model.info()                              # {"metrics": metrics.json, "model_card":
model_card.json}

CLI self-test:  python -m ml.inference sample_xrays/pneumonia_01.jpeg
"""
from __future__ import annotations

import base64
import io
import json
import threading
import time
from pathlib import Path

import numpy as np
import torch
from PIL import Image

from ml.common import CLASSES, MODELS_DIR, build_model, canonical_view, to_gray_uint8
from ml.datasets import eval_transform
from ml.gradcam import GradCAM, colorize, overlay

DISPLAY_LONG_SIDE = 768   # longest side (px) of input_png / overlay / heatmap PNGs
OVERLAY_OPACITY = 0.6     # heatmap opacity used for gradcam_overlay_png


def load_image(path: str | Path) -> Image.Image:
    """Open JPG/PNG/BMP/TIFF with PIL, or a DICOM (.dcm) with pydicom (returned as 32-bit
'F')."""
    path = Path(path)
    if path.suffix.lower() == ".dcm":
```

```
        import pydicom
        ds = pydicom.dcmread(str(path))
        arr = ds.pixel_array.astype(np.float32)
        arr = arr * float(getattr(ds, "RescaleSlope", 1)) + float(getattr(ds,
"RescaleIntercept", 0))
        if getattr(ds, "PhotometricInterpretation", "") == "MONOCHROME1":
            arr = arr.max() - arr  # MONOCHROME1 stores bones dark; flip to the usual look
        if arr.ndim == 3:
            arr = arr[..., :3].mean(-1) if arr.shape[-1] in (3, 4) else arr[0]
        return Image.fromarray(np.ascontiguousarray(arr, dtype=np.float32))  # mode "F"
    img = Image.open(path)
    img.load()
    return img


def _png_b64(arr: np.ndarray) -> str:
    buf = io.BytesIO()
    Image.fromarray(arr).save(buf, format="PNG")
    return base64.b64encode(buf.getvalue()).decode("ascii")


class PneumoniaModel:
    def __init__(self, models_dir: str = "models", device: str | None = None, gate: bool =
True):
        self.dir = Path(models_dir) if Path(models_dir).is_absolute() else MODELS_DIR.parent /
models_dir
        if not self.dir.exists():
            self.dir = MODELS_DIR
        self.device = torch.device(device or ("cuda" if torch.cuda.is_available() else "cpu"))
        self.amp = self.device.type == "cuda"
        self.cfg = json.loads((self.dir / "ensemble.json").read_text())
        self.members, self._lock = [], threading.RLock()
        for m in self.cfg["members"]:
            ck = torch.load(self.dir / m["file"], map_location="cpu", weights_only=False)
            net = build_model(ck["arch"], pretrained=False)
            net.load_state_dict({k: v.float() if v.is_floating_point() else v for k, v in
ck["state_dict"].items()})
            net = net.to(self.device).eval()
            for p in net.parameters():
                p.requires_grad_(False)  # Grad-CAM only needs gradients w.r.t. activations
            self.members.append({**m, "net": net, "cam": GradCAM(net), "tf":
eval_transform(m["res"])})
        self._pooled = None
        self.members[0]["net"].avgpool.register_forward_hook(lambda _m, _i, o: setattr(self,
"_pooled", o))
        self.gate = None
        if gate:
            from ml.xray_gate import XrayGate
            self.gate = XrayGate(self.dir / "xray_gate.npz")
        metrics_path = self.dir / "metrics.json"
        self.version = json.loads(metrics_path.read_text()).get("model_version", "unknown") \
            if metrics_path.exists() else "unknown"

    # ------------------------------------------------------------------ helpers
    def _tensor(self, member: dict, view: Image.Image) -> torch.Tensor:
        return member["tf"](view).unsqueeze(0).to(self.device)

    @torch.no_grad()
    def features(self, x: torch.Tensor) -> np.ndarray:
        """Pooled layer4 features of the first member for a normalised batch (used by the
gate)."""
        x = x.to(self.device)
        with torch.autocast("cuda", dtype=torch.float16, enabled=self.amp):
            self.members[0]["net"](x)
        return self._pooled.flatten(1).float().cpu().numpy()

    def _risk(self, p: float) -> str:
        for level, (lo, hi) in self.cfg["risk_bands"].items():
            if lo <= p < hi or (level == "High" and p >= lo):
                return level
        return "High"

    # ------------------------------------------------------------------ public API
```

```
    def check_image(self, img: Image.Image) -> dict:
        """Is this a chest X-ray? {"is_xray": bool, "score": 0..1, "reason": str}"""
        from ml.xray_gate import heuristics
        with self._lock:
            ok, reason, _ = heuristics(img)
            if not ok:
                return {"is_xray": False, "score": 0.0, "reason": reason}
            if self.gate is None:
                return {"is_xray": True, "score": 1.0, "reason": "gate disabled"}
            feats = self.features(self._tensor(self.members[0], canonical_view(img)))
            return self.gate.decide(img, feats)

    def predict(self, img: Image.Image, with_gradcam: bool = True) -> dict:
        t0 = time.perf_counter()
        check = self.check_image(img)
        if not check["is_xray"]:
            return {"rejected": True, "xray_check": check,
                    "inference_ms": (time.perf_counter() - t0) * 1000, "model_version":
self.version}
        with self._lock:
            gray = to_gray_uint8(img)
            view = canonical_view(img)
            margins, explained = [], []
            for m in self.members:
                x = self._tensor(m, view)
                with torch.autocast("cuda", dtype=torch.float16, enabled=self.amp):
                    if with_gradcam:
                        acts, grads, lg = m["cam"].explain(x)
                        explained.append((acts, grads, x.shape[-2:]))
                    else:
                        with torch.no_grad():
                            lg = m["net"](x)
                    views = [lg]
                    if self.cfg.get("tta_hflip", True):
                        with torch.no_grad():
                            views.append(m["net"](torch.flip(x, dims=[3])))
                margins.append(float(np.mean([float(v[0, 1] - v[0, 0]) for v in views])))
            z = float(np.mean(margins))
            prob = float(1 / (1 + np.exp(-z / self.cfg["temperature"])))
            thr = float(self.cfg["threshold"])
            label = CLASSES[int(prob >= thr)]

            # Display images keep the ORIGINAL aspect ratio; the square-view heatmap is
            # stretched back onto it, so all three PNGs line up pixel for pixel.
            h, w = gray.shape
            s = DISPLAY_LONG_SIDE / max(h, w)
            dw, dh = max(1, round(w * s)), max(1, round(h * s))
            disp = np.asarray(Image.fromarray(gray).resize((dw, dh), Image.Resampling.LANCZOS))
            out = {"label": label, "prob_pneumonia": prob,
                   "confidence": prob if label == "PNEUMONIA" else 1 - prob,
                   "threshold": thr, "risk_level": self._risk(prob),
                   "input_png": _png_b64(disp), "xray_check": check}
            if with_gradcam:
                # Explain the ENSEMBLE's decision: every member's map for the final label,
averaged.
                target = CLASSES.index(label)
                maps = [GradCAM.cam(acts, grads, target, tuple(size)) for acts, grads, size in
explained]
                avg = np.mean([np.asarray(Image.fromarray(c.astype(np.float32)).resize((dw, dh),
Image.Resampling.BILINEAR))
                               for c in maps], axis=0)
                avg = avg / avg.max() if avg.max() > 0 else avg
                heat = colorize(avg)
                out["gradcam_overlay_png"] = _png_b64(overlay(disp, heat, OVERLAY_OPACITY))
                out["gradcam_heatmap_png"] = _png_b64(heat)
            else:
                out["gradcam_overlay_png"] = out["gradcam_heatmap_png"] = ""
        out["inference_ms"] = (time.perf_counter() - t0) * 1000
        out["model_version"] = self.version
        return out

    def info(self) -> dict:
        read = lambda n: json.loads((self.dir / n).read_text()) if (self.dir / n).exists() else
{}
```

```
        return {"metrics": read("metrics.json"), "model_card": read("model_card.json")}


def _cli() -> None:
    import argparse
    ap = argparse.ArgumentParser(description="Run PneumoScan AI on image files")
    ap.add_argument("images", nargs="+")
    ap.add_argument("--save", help="folder to write the Grad-CAM overlay PNGs")
    ap.add_argument("--cpu", action="store_true")
    a = ap.parse_args()
    model = PneumoniaModel(device="cpu" if a.cpu else None)
    for path in a.images:
        res = model.predict(load_image(path))
        short = {k: (f"<{len(v)} base64 chars>" if k.endswith("_png") else v) for k, v in
res.items()}
        print(Path(path).name, json.dumps(short, indent=2))
        if a.save and "gradcam_overlay_png" in res:
            Path(a.save).mkdir(parents=True, exist_ok=True)
            (Path(a.save) /
f"{Path(path).stem}_gradcam.png").write_bytes(base64.b64decode(res["gradcam_overlay_png"]))


if __name__ == "__main__":
    _cli()
```

A.9 app/backend/model_service.py

Back-end model loading service.

```
"""Loads the real ResNet model once (in a background thread), or falls back to the mock.

Real model: used when ml/inference.py and models/ensemble.json exist and load without error.
Mock model: used otherwise; health reports mode "mock" and the UI shows a DEMO MODE banner.
"""
from __future__ import annotations

import json
import logging
import sys
import threading
import time
import traceback
from typing import Any

from PIL import Image

from . import config
from .errors import AppError

log = logging.getLogger("pneumoscan.model")


def real_model_files_present() -> bool:
    return (config.ML_DIR / "inference.py").exists() and (config.MODELS_DIR /
"ensemble.json").exists()


class ModelService:
    def __init__(self) -> None:
        self.model: Any = None
        self.mode: str | None = None  # "real" | "mock"
        self.state = "idle"  # idle | loading | ready | error
        self.error: str | None = None
        self.mock_reason: str | None = None
        self.device: str | None = None
        self.version: str | None = None
        self._lock = threading.Lock()  # one prediction at a time (GPU memory, thread safety)
        self._thread: threading.Thread | None = None
        self.loaded_at: float | None = None
```

```
    # ------------------------------------------------------------------ loading
    def start(self, background: bool = True) -> None:
        if self.state in ("loading", "ready"):
            return
        self.state = "loading"
        if background:
            self._thread = threading.Thread(target=self._load, name="model-loader", daemon=True)
            self._thread.start()
        else:
            self._load()

    def wait(self, timeout: float = 120.0) -> None:
        if self._thread is not None:
            self._thread.join(timeout)

    def _load_mock(self, reason: str) -> None:
        from .mock_inference import PneumoniaModel as MockModel

        self.model = MockModel(str(config.MODELS_DIR))
        self.mode = "mock"
        self.mock_reason = reason
        self.device = "cpu (mock)"
        self.version = self.model.version

    def _load(self) -> None:
        try:
            want = config.MODEL_MODE
            if want == "mock":
                self._load_mock("Mock model selected with PNEUMOSCAN_MODEL=mock.")
            elif real_model_files_present():
                try:
                    self._load_real()
                except Exception as exc:  # fall back, but say why
                    log.error("Real model failed to load:\n%s", traceback.format_exc())
                    if want == "real":
                        raise
                    self._load_mock(f"The trained model could not be loaded
({type(exc).__name__}: {exc}).")
            else:
                if want == "real":
                    raise RuntimeError("Trained model files were not found in the models
folder.")
                self._load_mock("Trained model files were not found in the models folder.")
            self.loaded_at = time.time()
            self.state = "ready"
            log.info("Model ready: mode=%s device=%s version=%s", self.mode, self.device,
self.version)
        except Exception as exc:
            self.state = "error"
            self.error = f"{type(exc).__name__}: {exc}"
            log.error("Model loading failed:\n%s", traceback.format_exc())

    def _load_real(self) -> None:
        root = str(config.ROOT)
        if root not in sys.path:
            sys.path.insert(0, root)
        from ml.inference import PneumoniaModel  # type: ignore[import-not-found]

        model = PneumoniaModel(str(config.MODELS_DIR))
        dev = getattr(model, "device", None)
        device = str(dev) if dev is not None else "cpu"
        try:
            import torch

            if device.startswith("cuda") and torch.cuda.is_available():
                device = f"cuda ({torch.cuda.get_device_name(0)})"
        except Exception:
            pass
        self.model = model
        self.mode = "real"
        self.mock_reason = None
        self.device = device
        self.version = getattr(model, "version", None) or self._version_from_metrics()
        if not config.SKIP_WARMUP:
```

```
            self._warm_up()

    def _version_from_metrics(self) -> str:
        try:
            return json.loads((config.MODELS_DIR / "metrics.json").read_text(encoding="utf-
8")).get("model_version", "unknown")
        except Exception:
            return "unknown"

    def _warm_up(self) -> None:
        """Run one prediction so the first real scan in class is fast."""
        try:
            candidates = sorted(config.SAMPLES_DIR.glob("*.jp*g")) +
sorted(config.SAMPLES_DIR.glob("*.png"))
            img = Image.open(candidates[0]) if candidates else Image.new("L", (512, 512), 90)
            img.load()
            with self._lock:
                self.model.predict(img, with_gradcam=True)
        except Exception:
            log.warning("Warm-up prediction failed (ignored):\n%s", traceback.format_exc())

    # ------------------------------------------------------------------ use
    def health(self) -> dict:
        status = {"ready": "ok", "error": "error"}.get(self.state, "warming_up")
        return {
            "status": status,
            "ready": self.state == "ready",
            "mode": self.mode,
            "device": self.device,
            "model_version": self.version,
            "mock_reason": self.mock_reason,
            "error": self.error,
            "app_version": config.APP_VERSION,
        }

    def require_ready(self) -> None:
        if self.state == "ready":
            return
        if self.state == "error":
            raise AppError(503, "model_error", "The model failed to load. Check the start.bat
window for details.")
        raise AppError(503, "warming_up", "The model is still warming up. Try again in a few
seconds.")

    def predict(self, img: Image.Image) -> dict:
        self.require_ready()
        try:
            with self._lock:
                return self.model.predict(img, with_gradcam=True)
        except AppError:
            raise
        except Exception as exc:
            log.error("Prediction failed:\n%s", traceback.format_exc())
            raise AppError(500, "inference_failed", "The model could not analyse this image. Try
another X-ray.") from exc

    def info(self) -> dict:
        """metrics.json + model_card.json (read from disk so they are shown even in mock
mode)."""
        out: dict = {"metrics": None, "model_card": None}
        for key, name in (("metrics", "metrics.json"), ("model_card", "model_card.json")):
            p = config.MODELS_DIR / name
            if p.exists():
                try:
                    out[key] = json.loads(p.read_text(encoding="utf-8"))
                except Exception:
                    log.warning("Could not read %s", p)
        return out


service = ModelService()
```

A.10 app/backend/main.py

FastAPI application and REST endpoints.

```
"""PneumoScan AI backend: FastAPI app serving the JSON API and the built React frontend.

Run from the project root:
    .venv\\Scripts\\python -m uvicorn app.backend.main:app --port 8000
"""
from __future__ import annotations

import logging
import re
from contextlib import asynccontextmanager
from pathlib import Path

from fastapi import FastAPI, File, Form, Query, Request, UploadFile
from fastapi.exceptions import RequestValidationError
from fastapi.responses import FileResponse, HTMLResponse, JSONResponse, Response
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel, ConfigDict
from starlette.concurrency import run_in_threadpool
from starlette.exceptions import HTTPException as StarletteHTTPException

from . import config, report, samples
from .errors import AppError
from .imaging import check_size, decode_image
from .model_service import ModelService
from .storage import IMAGE_KINDS, Store

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(name)s:
%(message)s")
log = logging.getLogger("pneumoscan")

SCAN_ID_RE = re.compile(r"^PS-\d{8}-[0-9A-F]{6}$")
SOURCES = {"scan", "sample", "triage"}
LIMITS = {"name": 120, "sex": 20, "patient_id": 64, "symptoms": 2000, "doctor_notes": 5000}

FIGURE_TITLES = {
    "confusion_matrix": "Confusion matrix (test set)",
    "roc_curve": "ROC curve (test set)",
    "roc": "ROC curve (test set)",
    "pr_curve": "Precision–recall curve (test set)",
    "training_curves": "Training curves",
    "architecture": "Model architecture",
    "architecture_diagram": "Model architecture",
    "residual_block_diagram": "Residual (bottleneck) block",
    "sample_images": "Sample X-rays from the dataset",
    "confusion_matrix_test": "Confusion matrix (test set)",
    "roc_curve_test": "ROC curve (test set)",
    "pr_curve_test": "Precision–recall curve (test set)",
    "probability_histogram_test": "Predicted probabilities (test set)",
    "gradcam_examples": "Grad-CAM examples",
    "sample_grid": "Sample images from the dataset",
    "class_distribution": "Class distribution",
    "calibration": "Calibration (reliability) curve",
}


# ---------------------------------------------------------------------------- helpers
def _err(status: int, code: str, message: str) -> JSONResponse:
    return JSONResponse({"error": {"code": code, "message": message}}, status_code=status)


def _clean_text(value: object, field: str) -> str:
    s = "" if value is None else str(value)
    s = s.replace("\r\n", "\n").replace("\r", "\n").replace("\x00", "").strip()
    limit = LIMITS[field]
    if len(s) > limit:
        nice = field.replace("_", " ")
        raise AppError(422, "invalid_field", f"The {nice} is too long (max {limit}
characters).")
```

```
    return s


def _clean_age(value: object) -> int | None:
    if value is None:
        return None
    s = str(value).strip()
    if s == "":
        return None
    try:
        age = float(s)
    except ValueError:
        raise AppError(422, "invalid_field", "Age must be a number of years, for example 45.")
from None
    if not (0 <= age <= 130) or age != age:
        raise AppError(422, "invalid_field", "Age must be between 0 and 130 years.")
    return int(age)


def _clean_sex(value: object) -> str:
    s = ("" if value is None else str(value)).strip().lower()
    mapping = {"": "", "f": "Female", "female": "Female", "m": "Male", "male": "Male", "o":
"Other", "other": "Other"}
    if s not in mapping:
        raise AppError(422, "invalid_field", "Sex must be Female, Male or Other (or left
empty).")
    return mapping[s]


def _patient_from_form(name: str, age: str, sex: str, patient_id: str, symptoms: str) -> dict:
    return {
        "name": _clean_text(name, "name"),
        "age": _clean_age(age),
        "sex": _clean_sex(sex),
        "patient_id": _clean_text(patient_id, "patient_id"),
        "symptoms": _clean_text(symptoms, "symptoms"),
    }


async def _read_upload(file: UploadFile) -> bytes:
    data = await file.read(config.MAX_UPLOAD_BYTES + 1)
    check_size(len(data))
    return data


FIGURE_ORDER = ("architecture", "roc", "confusion", "pr_curve", "probability", "calibration",
"gradcam", "training_curves",
                "class_distribution", "sample", "residual")


def _figures() -> list[dict]:
    root = config.FIGURES_DIR
    if not root.exists():
        return []
    out = []
    for p in root.glob("*.png"):
        stem = p.stem.lower()
        title = FIGURE_TITLES.get(stem)
        if title is None:
            key = next((k for k in FIGURE_TITLES if stem.startswith(k + "_")), None)
            if key:
                rest = p.stem[len(key) + 1:].replace("_", " ")
                title = f"{FIGURE_TITLES[key]} · {rest}" if rest else FIGURE_TITLES[key]
        rank = next((i for i, k in enumerate(FIGURE_ORDER) if stem.startswith(k)),
len(FIGURE_ORDER))
        out.append((rank, stem, {"name": p.stem, "title": title or p.stem.replace("_", "
").capitalize(),
                                 "url": f"/figures/{p.name}"}))
    return [f for _, _, f in sorted(out, key=lambda t: (t[0], t[1]))]


class ScanPatch(BaseModel):
    model_config = ConfigDict(extra="ignore")
```

```
    name: str | None = None
    age: int | float | str | None = None
    sex: str | None = None
    patient_id: str | None = None
    symptoms: str | None = None
    doctor_notes: str | None = None


# ---------------------------------------------------------------------------- app
def create_app() -> FastAPI:
    model = ModelService()
    store = Store(config.STORAGE_DIR)

    @asynccontextmanager
    async def lifespan(_app: FastAPI):
        model.start(background=True)
        yield

    app = FastAPI(title="PneumoScan AI", version=config.APP_VERSION, lifespan=lifespan,
                  docs_url="/api/docs", redoc_url=None, openapi_url="/api/openapi.json")
    app.state.model = model
    app.state.store = store

    # ------------------------------------------------------------- errors
    @app.exception_handler(AppError)
    async def _app_error(_req: Request, exc: AppError):
        return _err(exc.status, exc.code, exc.message)

    @app.exception_handler(StarletteHTTPException)
    async def _http_error(_req: Request, exc: StarletteHTTPException):
        if exc.status_code == 404:
            return _err(404, "not_found", "This page or item does not exist.")
        if exc.status_code == 405:
            return _err(405, "method_not_allowed", "This action is not allowed here.")
        detail = exc.detail if isinstance(exc.detail, str) else "The request could not be
processed."
        return _err(exc.status_code, f"http_{exc.status_code}", detail)

    @app.exception_handler(RequestValidationError)
    async def _validation_error(_req: Request, exc: RequestValidationError):
        errs = exc.errors()
        first = errs[0] if errs else {}
        loc = [str(x) for x in first.get("loc", []) if x not in ("body", "query", "path")]
        field = loc[0] if loc else ""
        if field in ("file", "files") and first.get("type") == "missing":
            return _err(400, "no_file", "No file was uploaded. Choose a chest X-ray image and
try again.")
        msg = first.get("msg", "Invalid request.")
        return _err(422, "invalid_request", f"{field + ': ' if field else ''}{msg}")

    @app.exception_handler(Exception)
    async def _unexpected(_req: Request, exc: Exception):
        log.exception("Unexpected error: %s", exc)
        return _err(500, "internal_error", "Something went wrong on the server. Please try
again.")

    # ---------------------------------------------------------------- core
    @app.get("/api/health")
    def health():
        return model.health()

    @app.get("/api/model-info")
    def model_info():
        info = model.info()
        h = model.health()
        return {"mode": h["mode"], "ready": h["ready"], "model_version": h["model_version"],
                "metrics": info["metrics"], "model_card": info["model_card"], "figures":
_figures()}

    def _analyse(data: bytes, filename: str, *, source: str, patient: dict, notes: str,
                 ground_truth: str | None) -> dict:
        dec = decode_image(data, filename)
        result = model.predict(dec.image)
```

```
        check = result.get("xray_check") or {}
        check = {"is_xray": bool(check.get("is_xray", False)), "score": float(check.get("score")
or 0.0),
                 "reason": str(check.get("reason") or "This does not look like a chest X-ray.")}
        if result.get("rejected"):
            return {"rejected": True, "saved": False, "filename": filename, "xray_check": check}
        result = {**result, "xray_check": check}
        scan = store.save_scan(result, filename=filename, source=source, mode=model.mode or
"mock",
                               patient=patient, notes=notes, ground_truth=ground_truth,
                               width=dec.width, height=dec.height)
        return {"rejected": False, "saved": True, "scan": scan}

    @app.post("/api/predict")
    async def predict(
        file: UploadFile = File(...),
        name: str = Form(""),
        age: str = Form(""),
        sex: str = Form(""),
        patient_id: str = Form(""),
        symptoms: str = Form(""),
        notes: str = Form(""),
        source: str = Form("scan"),
        ground_truth: str = Form(""),
    ):
        model.require_ready()
        patient = _patient_from_form(name, age, sex, patient_id, symptoms)
        notes_clean = _clean_text(notes, "doctor_notes")
        src = source if source in SOURCES else "scan"
        gt = ground_truth.strip().upper() or None
        if gt not in (None, "NORMAL", "PNEUMONIA"):
            gt = None
        data = await _read_upload(file)
        fname = Path(file.filename or "upload").name
        return await run_in_threadpool(_analyse, data, fname, source=src, patient=patient,
                                       notes=notes_clean, ground_truth=gt)

    @app.post("/api/batch")
    async def batch(files: list[UploadFile] = File(...)):
        model.require_ready()
        if len(files) > config.MAX_BATCH_FILES:
            raise AppError(413, "too_many_files", f"Upload at most {config.MAX_BATCH_FILES} X-
rays at a time.")
        items = []
        for f in files:
            fname = Path(f.filename or "upload").name
            item = {"filename": fname, "ok": False, "rejected": False, "error": None, "scan":
None, "xray_check": None}
            try:
                data = await _read_upload(f)
                res = await run_in_threadpool(_analyse, data, fname, source="triage",
patient={}, notes="",
                                              ground_truth=None)
                if res["rejected"]:
                    item.update(rejected=True, xray_check=res["xray_check"],
                                error={"code": "not_xray", "message": (res["xray_check"] or
{}).get("reason")
                                       or "This does not look like a chest X-ray."})
                else:
                    item.update(ok=True, scan=res["scan"])
            except AppError as exc:
                item["error"] = exc.to_dict()
            except Exception:
                log.exception("Batch item failed: %s", fname)
                item["error"] = {"code": "internal_error", "message": "This file could not be
processed."}
            items.append(item)

        def rank(it: dict):
            if it["ok"]:
                return (0, -it["scan"]["prob_pneumonia"])
            return (1 if it["rejected"] else 2, 0)

        items.sort(key=rank)
```

```
        return {"items": items, "count": len(items)}

    # ------------------------------------------------------------ history
    def _get_or_404(scan_id: str) -> dict:
        if not SCAN_ID_RE.match(scan_id):
            raise AppError(404, "not_found", "This scan does not exist. It may have been
deleted.")
        scan = store.get(scan_id)
        if scan is None:
            raise AppError(404, "not_found", "This scan does not exist. It may have been
deleted.")
        return scan

    @app.get("/api/history")
    def history(
        q: str = Query("", max_length=200),
        page: int = Query(1, ge=1, le=100000),
        page_size: int = Query(24, ge=1, le=100),
        label: str = Query(""),
        source: str = Query(""),
    ):
        lab = label.strip().upper() or None
        if lab not in (None, "NORMAL", "PNEUMONIA"):
            raise AppError(422, "invalid_request", "label must be NORMAL or PNEUMONIA.")
        src = source.strip().lower() or None
        if src is not None and src not in SOURCES:
            raise AppError(422, "invalid_request", "source must be scan, sample or triage.")
        return store.list(q.strip(), page, page_size, lab, src)

    @app.get("/api/history/{scan_id}")
    def history_item(scan_id: str):
        return _get_or_404(scan_id)

    @app.get("/api/history/{scan_id}/image/{kind}")
    def history_image(scan_id: str, kind: str):
        _get_or_404(scan_id)
        if kind not in IMAGE_KINDS:
            raise AppError(404, "not_found", "Unknown image type.")
        p = store.image_path(scan_id, kind)
        if not p.exists() and kind == "heatmap":
            p = store.image_path(scan_id, "overlay")
        if not p.exists():
            raise AppError(404, "not_found", "This image is missing.")
        return FileResponse(p, headers={"Cache-Control": "private, max-age=86400"})

    @app.patch("/api/history/{scan_id}")
    def history_update(scan_id: str, body: ScanPatch):
        _get_or_404(scan_id)
        fields: dict = {}
        sent = body.model_dump(exclude_unset=True)
        for key, val in sent.items():
            if key == "age":
                fields["age"] = _clean_age(val)
            elif key == "sex":
                fields["sex"] = _clean_sex(val)
            else:
                fields[key] = _clean_text(val, key)
        scan = store.update(scan_id, fields)
        if scan is None:
            raise AppError(404, "not_found", "This scan does not exist. It may have been
deleted.")
        return scan

    @app.delete("/api/history/{scan_id}")
    def history_delete(scan_id: str):
        _get_or_404(scan_id)
        store.delete(scan_id)
        return {"deleted": 1}

    @app.delete("/api/history")
    def history_clear():
        return {"deleted": store.clear()}
```

```
    @app.get("/api/report/{scan_id}.pdf")
    def report_pdf(scan_id: str):
        scan = _get_or_404(scan_id)
        try:
            pdf = report.build_report(scan, model.info(), store.image_path)
        except Exception as exc:
            log.exception("Report failed")
            raise AppError(500, "report_failed", "The PDF report could not be created. Please
try again.") from exc
        return Response(pdf, media_type="application/pdf", headers={
            "Content-Disposition": f'attachment; filename="PneumoScan_Report_{scan_id}.pdf"',
            "Cache-Control": "no-store",
        })

    # ------------------------------------------------------------- samples + figures
    @app.get("/api/samples")
    def samples_list():
        return {"items": samples.list_samples()}

    @app.get("/api/samples/thumb/{rel:path}")
    def sample_thumb(rel: str):
        p = samples.sample_path(rel)
        if p is None:
            raise AppError(404, "not_found", "Sample not found.")
        try:
            data = samples.thumbnail(p)
        except Exception as exc:
            raise AppError(422, "corrupt_image", "This sample image could not be read.") from
exc
        return Response(data, media_type="image/jpeg", headers={"Cache-Control": "public, max-
age=3600"})

    @app.api_route("/api/{rest:path}", methods=["GET", "POST", "PUT", "PATCH", "DELETE"],
include_in_schema=False)
    def api_404(rest: str):
        path = f"/api/{rest}"
        for route in app.routes:
            if (getattr(route, "name", "") != "api_404" and str(getattr(route, "path",
"")).startswith("/api/")
                    and route.path_regex.match(path)):
                raise AppError(405, "method_not_allowed", "This action is not allowed here.")
        raise AppError(404, "not_found", f"Unknown API endpoint: {path}")

    app.mount("/samples", StaticFiles(directory=str(config.SAMPLES_DIR), check_dir=False),
name="samples")
    app.mount("/figures", StaticFiles(directory=str(config.FIGURES_DIR), check_dir=False),
name="figures")

    # ------------------------------------------------------------- frontend (SPA)
    dist = config.FRONTEND_DIST

    @app.get("/{full_path:path}", include_in_schema=False)
    def spa(full_path: str):
        if dist.exists():
            if full_path:
                target = (dist / full_path).resolve()
                try:
                    target.relative_to(dist.resolve())
                    if target.is_file():
                        cache = "public, max-age=31536000, immutable" if
full_path.startswith("assets/") else "no-cache"
                        return FileResponse(target, headers={"Cache-Control": cache})
                except ValueError:
                    pass
                if full_path.startswith("assets/"):
                    raise AppError(404, "not_found", "File not found.")
            index = dist / "index.html"
            if index.exists():
                return FileResponse(index, headers={"Cache-Control": "no-cache"})
        return HTMLResponse(
            "<h1>PneumoScan AI</h1><p>The frontend has not been built yet. Run "
            "<code>npm run build</code> in app/frontend.</p>", status_code=503)
```

```
    return app


app = create_app()
```

APPENDIX B: HOW TO RUN THE PROJECT

B.1 Running the Application (Classroom Demo)

1. Double-click **start.bat** in the project folder (C:\PersonalProjects\pneumoscan-ai).
2. Wait a few seconds. The browser opens at http://127.0.0.1:8000/ (or the next free port up to 8010).
3. Keep the black start.bat window open while using the application; close it (or press Ctrl+C) to stop.
4. The status pill in the top bar shows **Model ready** once the weights are loaded. No internet connection is needed.

```
cd C:\PersonalProjects\pneumoscan-ai
start.bat
```

B.2 First-Time Setup

Only needed if the .venv folder or the built web interface is missing. start.bat also builds the web interface automatically the first time if Node.js is installed.

```
cd C:\PersonalProjects\pneumoscan-ai
py -3.11 -m venv .venv
.venv\Scripts\python -m pip install -r app\backend\requirements.txt
REM plus PyTorch 2.11 + torchvision 0.26 (CUDA 12.8 build) for the real model

cd app\frontend
npm install
npm run build
```

B.3 Reproducing the Machine-Learning Pipeline

The dataset (ChestXRay2017.zip, Mendeley Data DOI 10.17632/rscbjbr9sj.2) must be extracted to data/raw/chest_xray. The commands below reproduce every step; the --val-only run never touches the test split, and the --export run scores the test split once and writes the files in models/.

```
set PY=.venv\Scripts\python.exe
%PY% -m ml.prepare_data
%PY% -m ml.train --name r50_384_s1 --arch resnet50 --res 384 --epochs 20 --bs 24 --seed 1
%PY% -m ml.train --name r34_384_s2 --arch resnet34 --res 384 --epochs 20 --bs 32 --seed 2
%PY% -m ml.train --name r50_448_s3 --arch resnet50 --res 448 --epochs 20 --bs 16 --seed 3
%PY% -m ml.evaluate --members r50_384_s1 r34_384_s2 r50_448_s3 --val-only
%PY% -m ml.evaluate --members r50_384_s1 r34_384_s2 r50_448_s3 --export
%PY% -m ml.xray_gate
%PY% -m ml.make_figures
%PY% -m pytest ml/test_inference.py -q
```

B.4 Analysing a Single Image from the Command Line

```
.venv\Scripts\python.exe -m ml.inference sample_xrays\pneumonia_01.jpeg --save out_cam
```

B.5 Running the Tests

```
.venv\Scripts\python -m pytest ml\test_inference.py -q
.venv\Scripts\python -m pytest app\backend\tests -q
cd app\frontend && npm run lint && npm run build
```

Interactive API documentation is available at http://127.0.0.1:8000/api/docs while the server is running. All patient details, images and notes are stored only in app/backend/storage/ on the local computer.